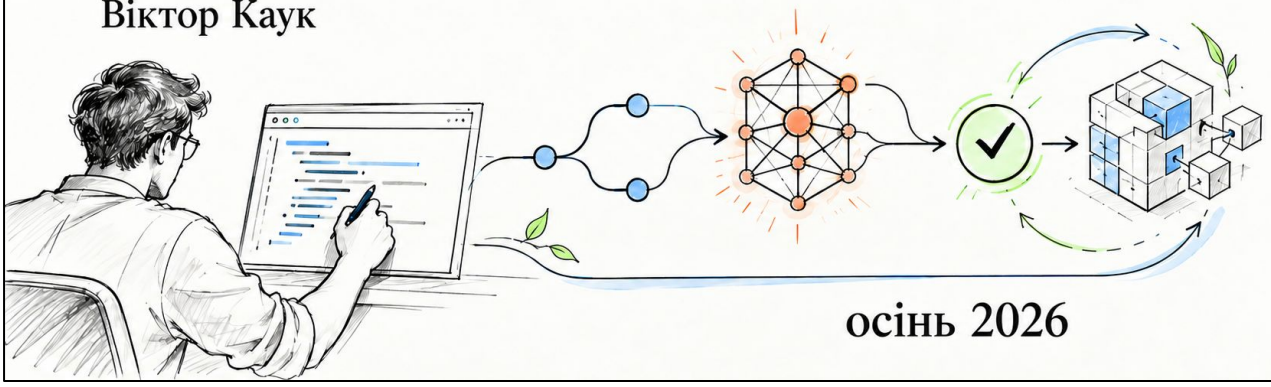
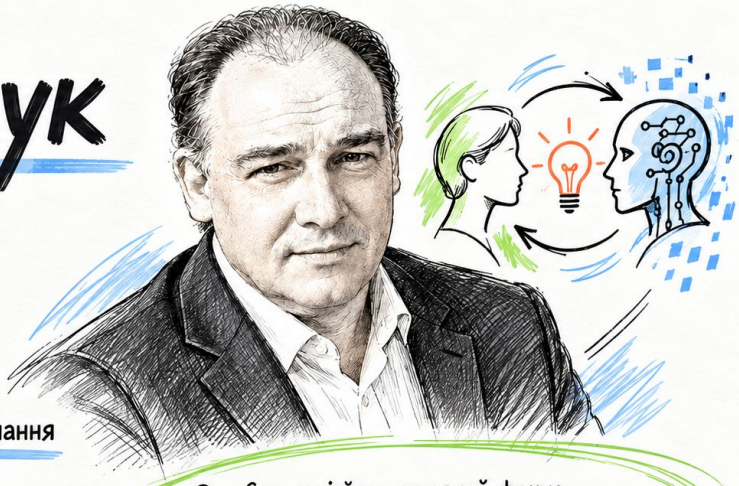
Штучний інтелект у програмній інженерії

Віктор Каук



осінь 2026

Віктор Каук



кандидат технічних наук

доцент кафедри програмної інженерії ХНУРЕ

науковий керівник Центру технологій дистанційного навчання

«Амбасадор ІТ освіти» EPAM Ukraine

Понад 30 років працює на перетині ІТ, освіти й технологічних інновацій.

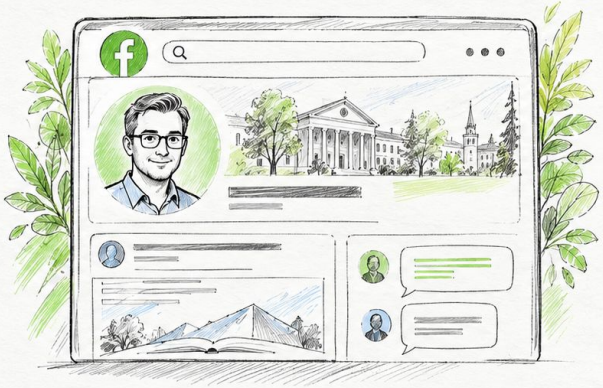
Сьогодні його основний фокус — генеративний ШІ, AI-асистенти, computational creativity, дизайн-мислення та нові способи взаємодії людини й штучного інтелекту.

Інфоресурси від мене

Telegram-група

ШІ для викладача та науковця

@AI_for_teachers_researchers



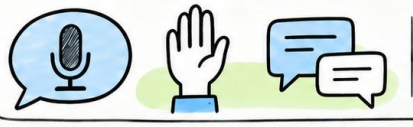
Facebook

https://www.facebook.com/victor.kauk

Правила спілкування

Не забуваємо про відвідування – відмічайтесь (є проблеми – кажіть)

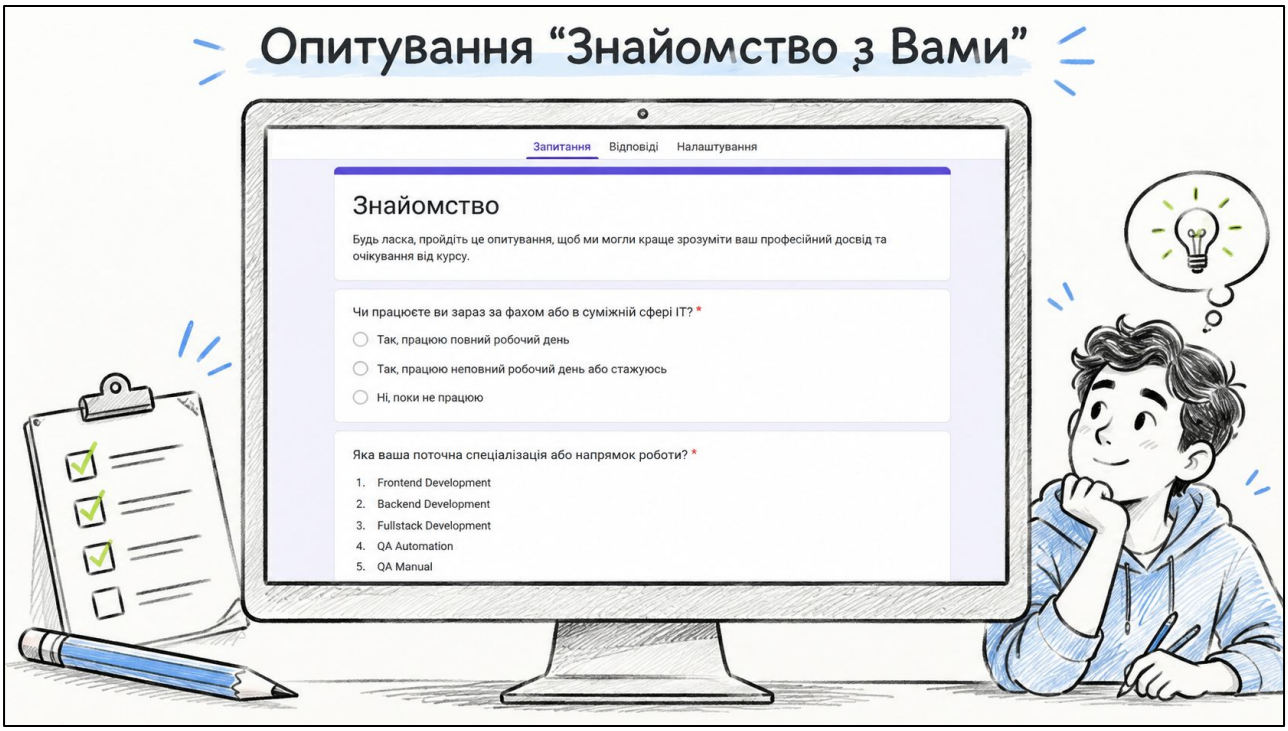
Камера за можливістю



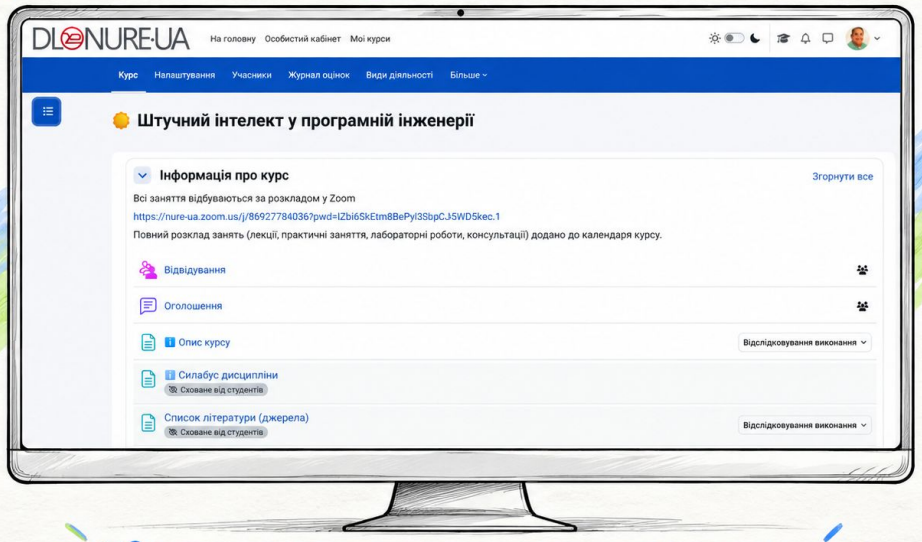
Питання одразу: голосом, піднята рука, у чаті

Будьте активними!

Не буває незручних або простих питань,
погано коли їх зовсім немає!


Опитування “Знайомство з Вами”
Запитання
Відповіді
Налаштування
Знайомство
Будь ласка, пройдіть це опитування, щоб ми могли краще зрозуміти ваш професійний досвід та очікування від курсу.
Чи працюєте ви зараз за фахом або в суміжній сфері IT? *
Так, працюю повний робочий день
Так, працюю неповний робочий день або стажуюсь
Ні, поки не працюю
Яка ваша поточна спеціалізація або напрямок роботи? *
1. Frontend Development
2. Backend Development
3. Fullstack Development
4. QA Automation
5. QA Manual

Дистанційний курс



https://dl.nure.ua/course/view.php?id=32205

Правила появи матеріалів на курсі

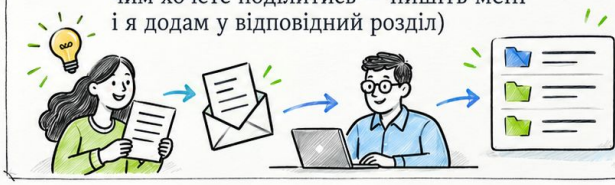
1 У вас будуть всі необхідні матеріали: конспекти, вебпокликання, методичні рекомендації, відеозаписи

2 Усі матеріали стають доступними після заняття (лекції, ПЗ, ЛР)

3 Інформація щодо самостійної роботи буде надана після 3 лекції

4 Прийом завдань з ПЗ та ЛР (від дати першого заняття з будь-якою групою до завершення семестру)

5 Вітається поява додаткових матеріалів від студентів (знайшли щось цікаве, чим хочете поділитись — пишіть мені і я додам у відповідний розділ)



Зміст курсу

Лекцій — 8

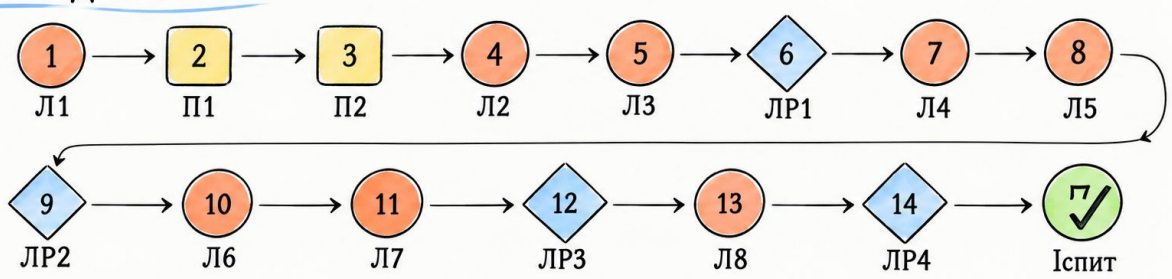
Практичні заняття — 2 (підряд)

Лабораторні роботи — 4

Самостійна робота

Іспит

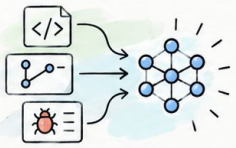
Послідовність



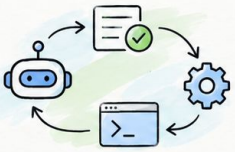
Самостійна робота — упродовж курсу

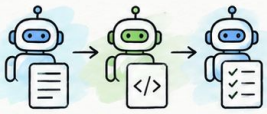
Теоретична частина

1.
Вступ до AI у програмній інженерії

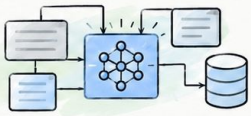
2. 
Програмні артефакти як дані для AI

3.
Мовні моделі для коду

4. 
Агентні архітектури в інженерії ПЗ

5. 
Мультиагентні workflow в SE

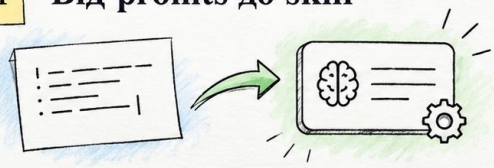
6.
AI для тестування, налагодження та безпеки

7. 
Побудова систем з AI-компонентами

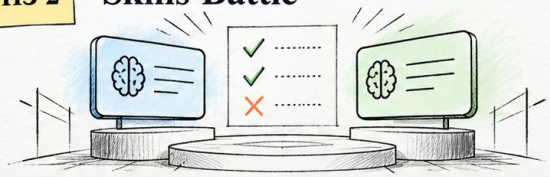
8.
Оцінювання, експлуатація, відповідальне використання

Практична частина

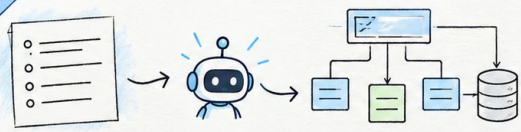
ПЗ 1 Від promts до skill



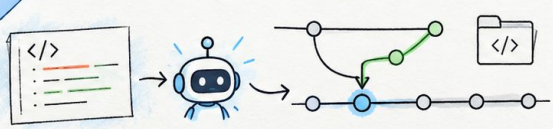
ПЗ 2 Skills Battle



ЛР 1 AI-assisted requirements and design

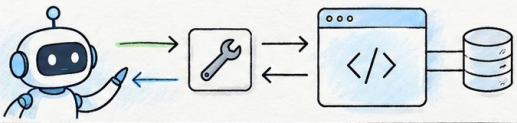


ЛР 2 AI-assisted implementation у наявному репозиторії



ЛР 3 AI для тестування, налагодження та безпеки

ЛР 4 Інженерія малого агентного AI-enabled компонента



Як отримати оцінку

Увага! Обов'язково виконати всі ПЗ та ЛР

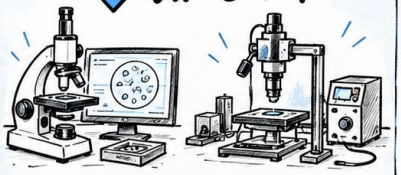
ПЗ 1 і 2

2 × 5 = 10 балів

ЛР 1 і 2

2 × 12 = 24 бали

ЛР 3 і 4



2 × 13 = 26 балів

10 + 24 + 26 = **60** балів за семестр

\+

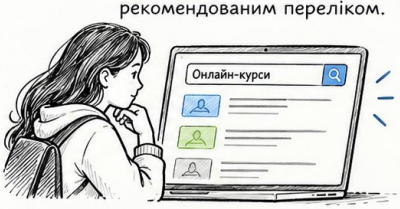
Іспит — максимум **40** балів

60 + 40 = 100 балів

Неформальна освіта — сертифікати за зовнішні курси

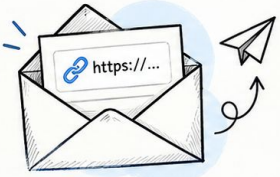
1 Знайти курс

Можна і поза рекомендованим переліком.



2 Написати викладачу

Надіслати вебпокликання на курс.

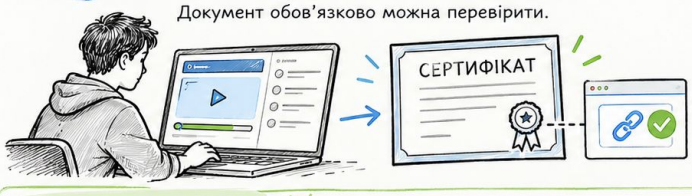


3 Отримати згоду на перезарахування

Частина курсу: ПЗ або ЛР; увесь курс — виняток.

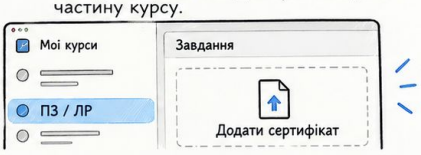
4 Пройти курс і отримати сертифікат

Документ обов'язково можна перевірити.



5 Подати сертифікат

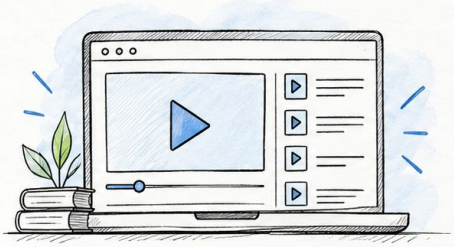
Як відповідь на погоджену викладачем частину курсу.



На що звертати увагу

- Тематика чітко відповідає курсу.
- Кількість годин — не менша за відповідну частину курсу.
- Сертифікат перевіряється за вебпокликанням (Udemy, Coursera).

Не забуваємо про можливості

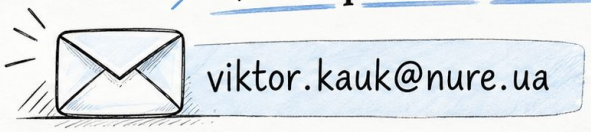


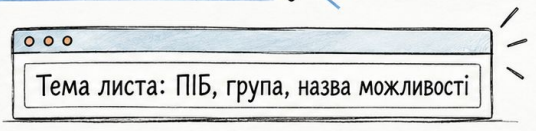
UDEMY

Grammarly

***

Щоб отримати, напишіть мені на пошту



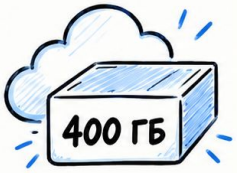


https://one.google.com/ai-student

Пропозиції Google AI для студентів ЗВО

Google AI Plus

0 грн/міс.
на 12 місяців
Потім — 224,99 грн/міс.



- Gemini і Gemini Notebook
- До 200 сторінок конспектів
- Створення тексту й контенту

Google AI Pro + YouTube Premium

309,99 грн/міс.
Знижка 71% — до 4 років
Потім — 909,99 грн/міс.

- До 1500 сторінок конспектів
- Розширені можливості Gemini
- Інструменти для розробки
- YouTube Premium

Умови та доступність перевіряйте за посиланням.

Використання ШІ

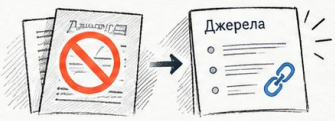
Використання генеративного AI дозволяється, **якщо завдання не позначено як індивідуальне контрольне без AI.**

Заборонено:

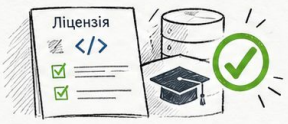
Передавати зовнішнім AI-сервісам персональні дані, закритий вихідний код, паролі, API-ключі та іншу конфіденційну інформацію.

Фабрикувати результати запусків або тестів.

Подавати AI-вивід без перевірки й розуміння.



Приховувати джерела.



Ігнорувати умови ліцензій програмного коду та навчальних даних.

Конвергенція штучного інтелекту та програмної інженерії знаменує фундаментальну зміну парадигми в сучасних обчислювальних системах. У центрі цієї трансформації залишається інженер-людина, координуючи двосторонній зв'язок із інтелектуальними системами.

З одного боку, ми застосовуємо автономні механізми та статистичні підходи — AI4SE та ML4SE — для оптимізації рутинних робочих процесів, розширеної предиктивної аналітики та автоматизованого аналізу коду.

Завдяки стрімкому розвитку великих мовних моделей (LLM4SE), генеративні алгоритми безпосередньо залучаються до синтезу коду, розбору специфікацій природною мовою та інтелектуального рефакторингу.

З іншого боку, вектор зміщується в площину SE4AI, де завданням інженера стає проєктування надійних, масштабованих архітектур, побудова конвеєрів обробки даних, мінімізація затримок та забезпечення детермінованого контролю над імовірнісними компонентами.

У результаті постає ключове архітектурне та етичне питання: де саме закінчується внесок автоматизованого інструмента й починається безпосередня відповідальність інженера?

Сьогодні ми закладаємо понятійний фундамент нашого навчального курсу, що визначає системний перехід від абстрактного сприйняття штучного інтелекту до його практичної інтеграції в інженерію програмного забезпечення. Наша мета полягає не просто в огляді технологій, а в побудові чіткої таксономії результатів навчання, де кожна теоретична концепція безпосередньо верифікується контрольними питаннями самоперевірки.

Насамперед ми проводимо суворе методологічне розмежування між векторами взаємодії: застосуванням штучного інтелекту, класичного машинного навчання та великих мовних моделей для інженерних завдань — AI4SE, ML4SE та LLM4SE — проти проєктування інфраструктури, архітектурної надійності та конвеєрів для самих AI-систем, тобто парадигми SE4AI. Наступним кроком є точне відображення генеративних LLM-інструментів на кожен окремий етап життєвого циклу розробки програмного забезпечення (SDLC) — від аналізу вимог і проєктування до рефакторингу, тестування й розгортання.

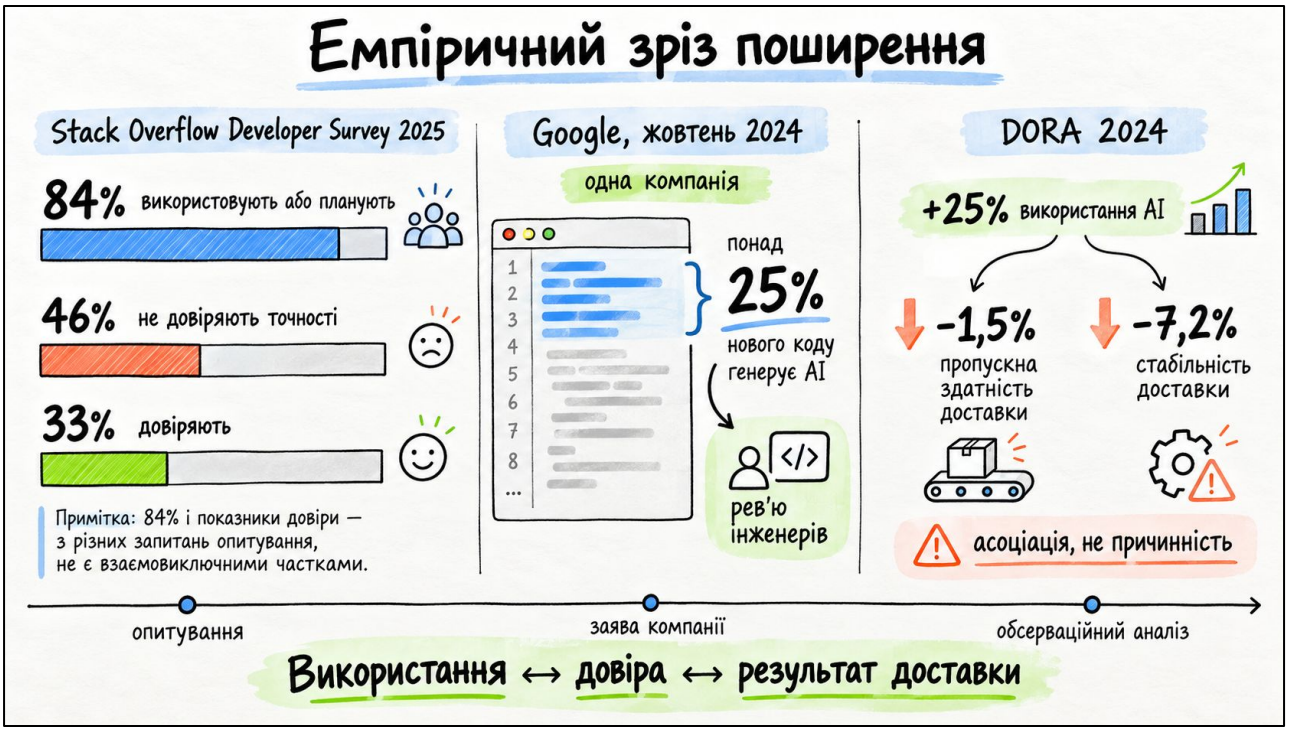
Глибший інженерний аналіз вимагає класифікації інструментів за критичними осями: рівнем автономності агентів, обсягом і керованістю контексту та детермінованістю результатів. Ми детально з'ясуємо природу недетермінованості сучасних генеративних моделей, обумовлену стохастичними процесами семплювання. Це підводить нас до завершальної та найважливішої

компетенції: формуванню суворого підходу до верифікації, валідації та забезпечення якості всіх генерованих штучним інтелектом артефактів, що гарантує надійність, безпеку та детерміновану відповідність фінального програмного продукту корпоративним стандартам.

Протягом усієї історії розвитку програмної інженерії масштабні зміщення парадигм неодноразово переосмислювали базову одиницю роботи інженера. На зламі 1980–1990-х років перехід від процедурного підходу до об'єктно-орієнтованого програмування підняв рівень абстракції від окремих функцій до повноцінних класів, що поєднують поля та методи.

Паралельно контроль версій пройшов шлях від SCCS і CVS до Git, змінивши фокус відстеження з ізольованого файлу на цілісний коміт у розподіленому дереві змін. Початок 2000-х років ознаменувався появою безперервної інтеграції (CI), яка витіснила нестабільні нічні збірки й запровадила автоматизований конвеєр перевірки для кожного окремого коміту. Сьогодні, в часовому проміжку 2021–2026 років, генеративні LLM-інструменти формують новий фундаментальний зсув.

Механічне написання коду перестає бути головним змістом інженерної праці. Новою одиницею інженерної відповідальності стає точне специфікування вимог, формалізація намірів та багаторівнева верифікація синтезованих артефактів.

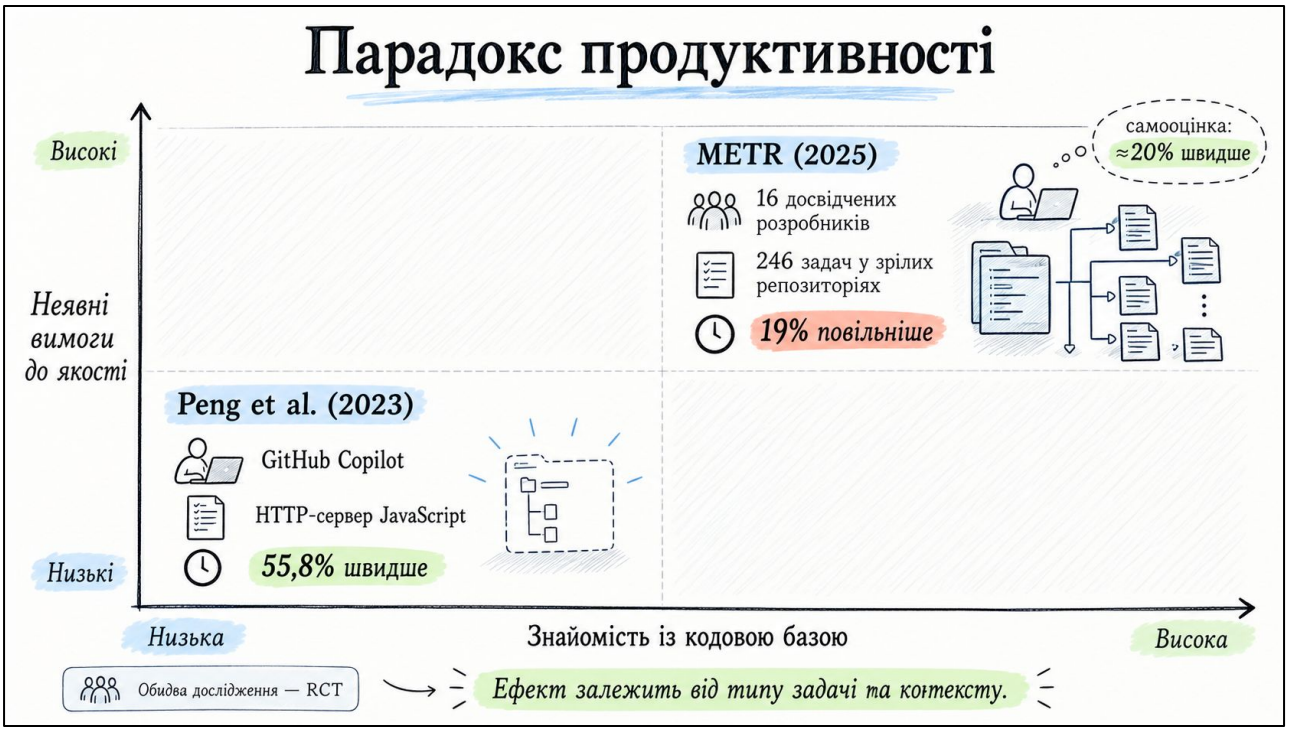


Аналіз емпіричного зрізу впровадження генеративного штучного інтелекту розкриває суттєвий розрив між масштабами адопції та реальними виробничими метриками надійності систем. Згідно з масштабним опитуванням Stack Overflow Developer Survey 2025, вісімдесят чотири відсотки фахівців уже застосовують або планують інтегрувати AI-інструменти в розробку. Водночас ми стикаємося з глибоким дефіцитом довіри: сорок шість відсотків інженерів не довіряють точності згенерованих результатів, і лише тридцять три відсотки висловлюють упевненість у синтезованих артефактах. Методологічно ці показники репрезентують різні виміри сприйняття технології, а не взаємовиключні частки.

На корпоративному рівні публічні прецеденти демонструють стрімке нарощування генеративних обсягів. Зокрема, за заявою Google у жовтні 2024 року, понад двадцять п'ять відсотків нового коду компанії вже генерується штучним інтелектом. Проте критичним запобіжником тут виступає обов'язкове експертне рецензування інженерами-людьми, яке гарантує збереження архітектурної цілісності та верифікацію коду перед його інтеграцією в кодову базу.

Фундаментальний обсерваційний аналіз звіту DORA 2024 підкреслює неоднозначність практичного впливу автоматизації на процеси доставки. Зростання інтенсивності використання AI на двадцять п'ять відсотків статистично

асоціюється зі зниженням пропускної здатності доставки на півтора відсотка та спадом стабільності релізів на 7,2 відсотка. Важливо наголосити: тут зафіксовано статистичну асоціацію, а не прямий причинно-наслідковий зв'язок. Побудована траєкторія — від опитувань розробників через заяви окремих компаній до обсерваційного аналізу — доводить, що взаємозв'язок між використанням інструментів, рівнем довіри інженера та фінальною якістю поставки програмного забезпечення потребує суворої інженерної дисципліни та системної валідації.



Взаємозв'язок між генеративним штучним інтелектом та швидкістю розробки оголює фундаментальний парадокс продуктивності, зумовлений природою завдань і системним контекстом. Аналіз рандомізованих контрольованих досліджень демонструє діаметрально протилежні результати залежно від граничних умов проєктування. У простих сценаріях із низькими неявними вимогами до якості — як це зафіксовано в класичному дослідженні Пена та співавторів двадцять третього року — інженери, які використовували GitHub Copilot для реалізації ізольованого JavaScript HTTP-сервера, продемонстрували прискорення розробки на п'ятдесят п'ять і вісім десятих відсотка. За низької попередньої обізнаності з кодовою базою та вузької архітектурної ізоляції генеративні підказки дають відчутний виграш у швидкості.

Проте емпіричні випробування в умовах складних, сформованих кодових баз виявляють значний супротив імовірнісних інструментів. Дослідження організації METR двадцять п'ятого року, що охопило шістнадцять досвідчених фахівців під час виконання двохсот сорока шести завдань у зрілих репозиторіях, виявило реальне сповільнення роботи на дев'ятнадцять відсотків. При цьому виникає показовий когнітивний дисонанс: самі інженери суб'єктивно оцінювали свою роботу як швидшу приблизно на двадцять відсотків. У щільно зв'язаних архітектурах із високими неявними стандартами витрати на осмислення, контекстну інтеграцію, налагодження та верифікацію згенерованого коду починають переважати час його прямого написання людиною. Емпіричний

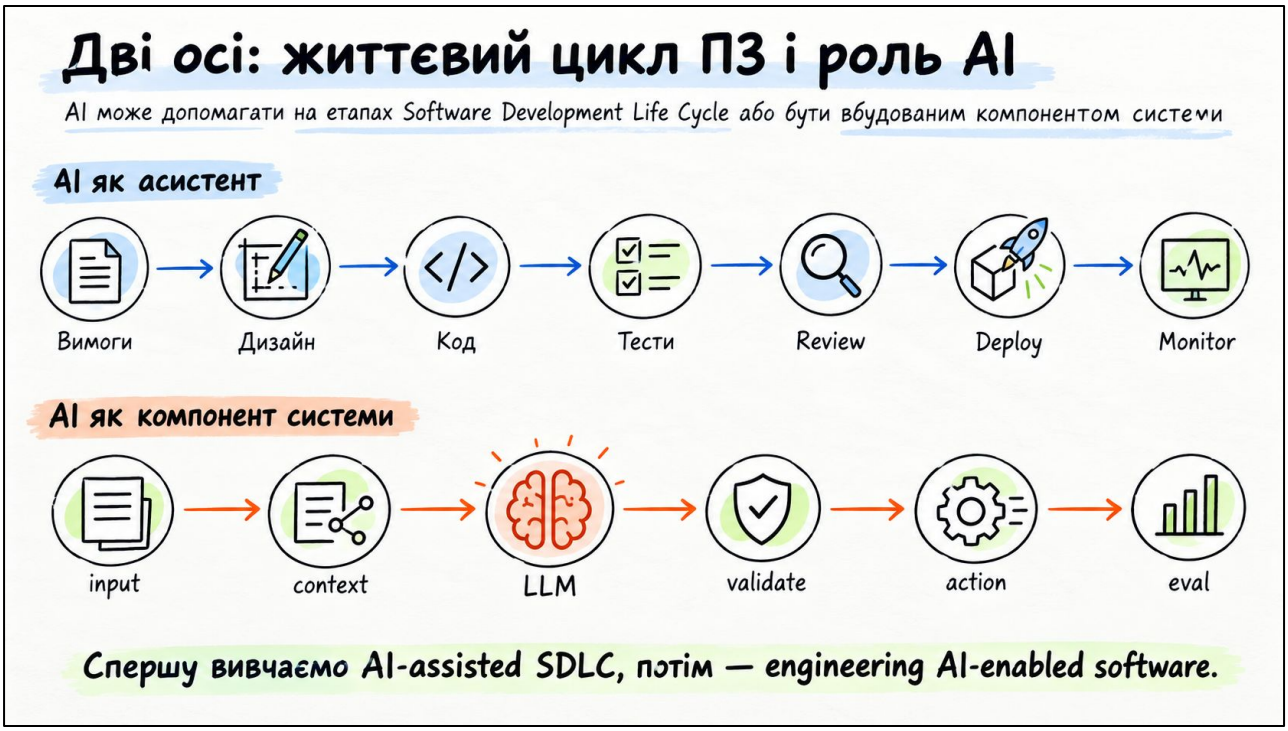
підсумок однозначний: ефект продуктивності не є абсолютним і прямо залежить від типу задачі, де надмірне довіряння синтезованим артефактам у складному контексті генерує архітектурний борг замість очікуваного прискорення.

Фундаментальна теза Фреда Брукса про відсутність «срібної кулі» встановлює ключове розмежування між суттєвою та випадковою складністю програмної інженерії.

Суттєва складність безпосередньо випливає з самої предметної області: вона охоплює узгодження неоднозначних вимог зацікавлених сторін, математичний вибір незмінних інваріантів системи та балансування складних компромісів між продуктивністю, масштабованістю, безпекою й вартістю архітектури.

У цьому вимірі штучний інтелект виконує виключно дорадчу функцію генерації варіантів, тоді як людина-інженер зберігає абсолютну відповідальність за остаточний вибір та системні наслідки.

Натомість генеративні інструменти спрямовані на подолання випадкової складності, спричиненої технічною реалізацією: синтез шаблонного коду, інтеграцію знайомих інтерфейсів прикладного програмування та форматування й трансляцію синтаксису. Звідси випливає наша робоча гіпотеза: що вища частка суттєвої складності в інженерній задачі, то меншим є реальний приріст продуктивності від застосування моделей штучного інтелекту.



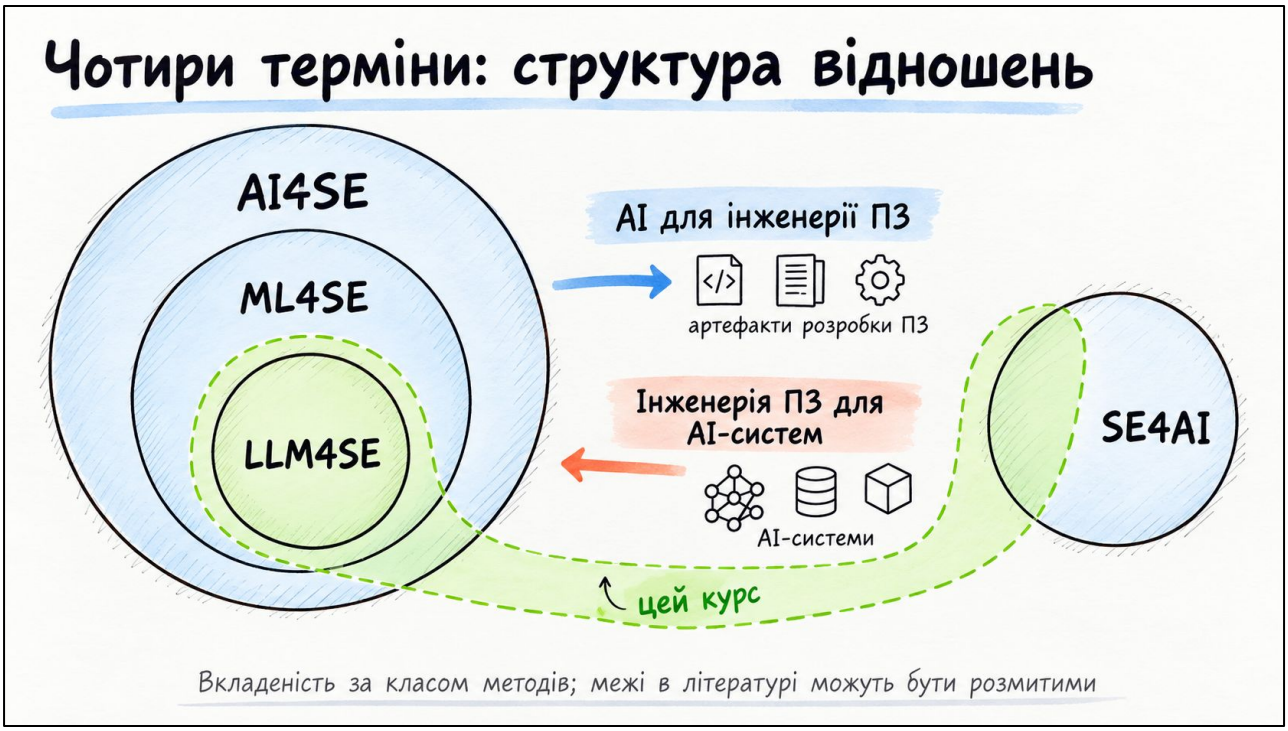
Інтеграція штучного інтелекту в інженерію програмного забезпечення вимагає системної декомпозиції вздовж двох фундаментальних осей: ШІ як зовнішній інструмент прискорення інженерного циклу та ШІ як внутрішній, виконуваний компонент самої архітектури системи.

Перший вимір — ШІ як асистент — передбачає органічне впровадження інтелектуальних інструментів у класичний життєвий цикл розробки програмного забезпечення (SDLC). Моделі оптимізують формалізацію вимог, проєктування архітектури, синтез вихідного коду, генерацію модульних тестів, автоматизований код-рев'ю, процеси деплою та моніторинг інфраструктури. У цій площині програмний продукт залишається детермінованим, проте суттєво зростає швидкість та ефективність інженерної діяльності.

Другий вимір — ШІ як компонент системи — повністю змінює архітектурну модель застосунку. Програма приймає вхідні дані, збагачує їх динамічним контекстом і передає у ядро великої мовної моделі. З огляду на стохастичну природу нейромережевих відповідей, обов'язковим стає впровадження суворого шару валідації результату перед виконанням будь-якої дії, а також безперервний конвеєр оцінювання (eval) у процесі експлуатації. Саме тому коректна навчальна методологія передбачає послідовний перехід: спершу ми ґрунтовно опановуємо AI-assisted SDLC, і лише після цього переходимо до проєктування та надійного інженерного забезпечення систем із вбудованим штучним інтелектом.

Фундаментальна архітектура програмної інженерії пройшла через три окремі епохи розвитку, які кардинально змінили спосіб опису логіки, її виконання, версіонування та верифікації. У класичній парадигмі Software 1.0 системи базувалися на явному, детермінованому синтаксисі. Його виконавцем виступав комп'ютер, версіонувався безпосередньо вихідний код, а надійність гарантувалася детермінованими модульними та інтеграційними програмними тестами. У 2017 році Андрій Карпати формалізував концепцію Software 2.0, де логіка змістилася від явних інструкцій до оптимізованих ваг нейронних мереж. Тут виконавцем стає навчена модель, об'єктом версіонування — дані, ваги та пайплайни навчання, а засобом валідації — статистичні тестові набори та цільові метрики точності.

Станом на 2025 рік Карпати окреслив появу парадигми Software 3.0, у якій ключовим носієм програми стає промпт природною мовою, а виконавчим рушієм — велика мовна модель. Версіонування тепер охоплює системні промпти, контексти та конфігурації мультиагентних систем. Відповідно, підходи до перевірки трансформуються у комплексні оцінювальні бенчмарки (evals) та поведінкові тести для контролю недетермінованих реакцій. Ця системна еволюція чітко показує: процеси рев'ю та тестування послідовно масштабувалися — від аналізу коду до верифікації даних і ваг, а сьогодні — до оцінювання промптів та агентної взаємодії.



Побудова чіткої таксономії є критично важливою для аналізу перетину штучного інтелекту та програмної інженерії. Ми визначаємо ієрархічну структуру вкладеності за класом методів: штучний інтелект для інженерії програмного забезпечення, або AI4SE, виступає найширшим доменом, що охоплює евристичні алгоритми, символьні методи та експертні системи, орієнтовані на артефакти розробки ПЗ — вихідний код, документацію й конфігурації. Усередині нього розташовується машинне навчання для інженерії ПЗ, або ML4SE, яке додає статистичні моделі, аналіз залежностей та предиктивну аналітику до інженерного життєвого циклу. Подальша спеціалізація виокремлює великі мовні моделі для програмної інженерії, або LLM4SE, як генеративний напрям, зосереджений на семантичному аналізі, синтезі коду й контекстних трансформаціях.

Паралельно існує ортогональний вектор — інженерія програмного забезпечення для AI-систем, або SE4AI. Тут спрямованість кардинально змінюється: класичні інженерні практики — модульність, масштабованість конвеєрів даних, відмовостійкість та надійність архітектури — застосовуються для проєктування й супроводу самих AI-систем. Фокус нашого курсу зосереджений саме на стику LLM4SE та SE4AI, поєднуючи генеративні можливості мовних моделей із надійним інженерним каркасом. Хоча термінологічні межі в літературі часто є розмитими, розуміння цієї структури забезпечує детермінований контроль, точне відстеження артефактів та системну

дисципліну в сучасній розробці.

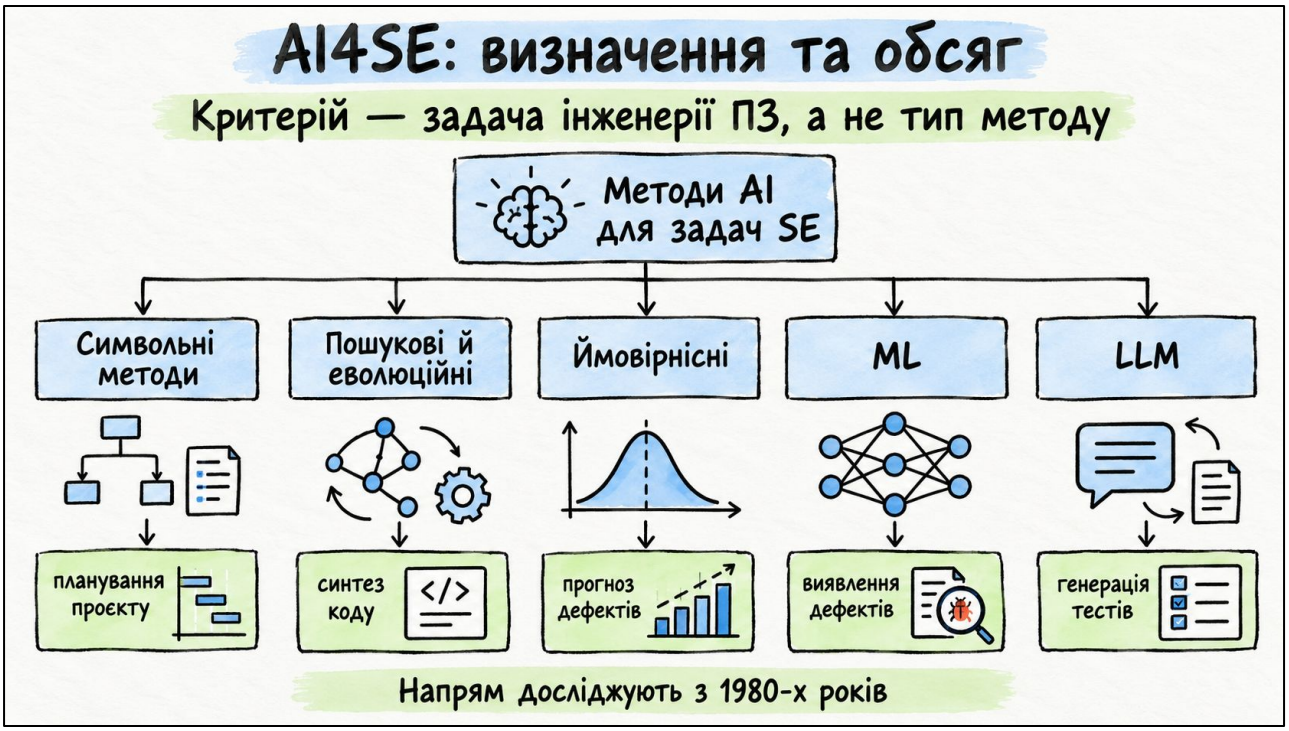
Фундаментальне розуміння взаємодії штучного інтелекту та програмної інженерії вимагає суворого розмежування двох векторів відображення: що виступає інструментом перетворення, а що є його безпосереднім об'єктом.

У першому напрямі — AI4SE — штучний інтелект виступає перетворювальною силою, спрямованою на вдосконалення процесів розробки. Тут методи машинного навчання, оптимізаційні алгоритми, евристичні моделі та механізми інференсу діють як функціональне відображення над класичними інженерними артефактами: вимогами, архітектурними схемами, вихідним кодом, тест-кейсами та конфігураціями. Результатом є покращені або якісно нові інженерні артефакти: синтезований код, згенерована документація чи оптимізовані системні компоненти. Типовим прикладом є сценарій LLM4SE, коли мовна модель аналізує специфікацію бізнес-логіки та генерує набір тестових сценаріїв для платіжного сервісу.

Протилежний вектор — SE4AI — розглядає інженерію програмного забезпечення як дисципліну, що формує керовану AI-систему. Тут усталені інженерні практики, формальні стандарти, методики контролю якості та процеси верифікації застосовуються до систем, які містять недетерміновані компоненти машинного навчання. Фокус зосереджується на надійності архітектури, безпеці інтерфейсів, інтеграційних шлюзах, конвеєрах даних і спостережуваності. Метою є трансформація стохастичної моделі у надійну, безпечну та повністю керовану виробничу систему через безперервний моніторинг, регресійне тестування чат-

ботів і контроль деградації.

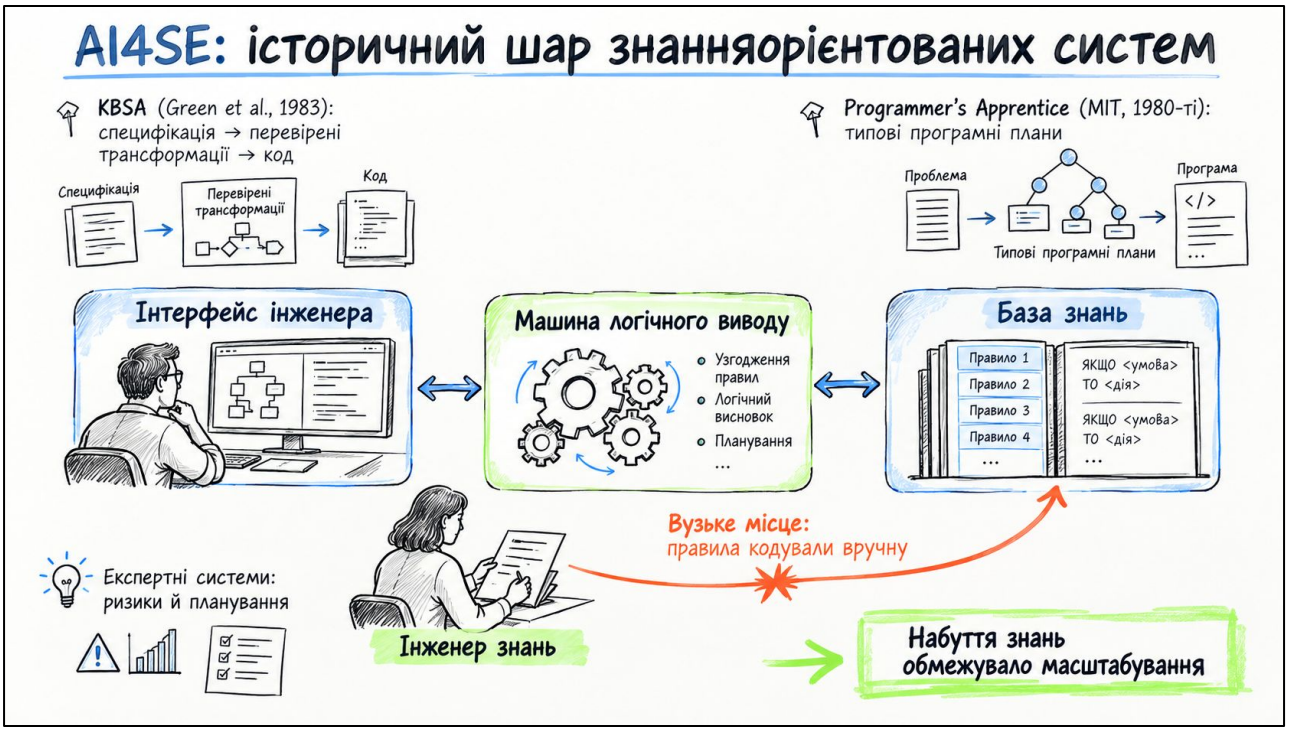
Кульмінацією цього дуалізму є циклічні контури зворотного зв'язку, де обидва напрями сходяться: наприклад, коли велика мовна модель синтезує оцінювальні набори (evals), які потім застосовуються для валідації та тестування надійності самого LLM-застосунку. Ключове інженерне питання полягає в усвідомленні ролей: що саме перетворюється і що виступає інструментом перетворення.



Визначення штучного інтелекту для програмної інженерії ґрунтується на фундаментальному методологічному критерії: визначальною межею є сама інженерна задача, а не конкретний клас чи тип застосованого методу. У межах цього підходу методи штучного інтелекту розглядаються як широкий спектр формальних обчислювальних механізмів, спрямованих на оптимізацію інженерних процесів розробки програмного забезпечення. Важливо підкреслити, що це не новий тренд останніх років; наукові дослідження цього напряму системно тривають ще з 1980-х років.

Таксономічний простір AI4SE охоплює цілу низку обчислювальних парадигм. Символьні методи та системи представлення знань формалізують бізнес-правила й обмеження предметної області для автоматизації календарного планування проєктів і розподілу ресурсів. Пошукові та еволюційні алгоритми досліджують комбінаторні простори станів для задач синтезу коду, автоматизованого виправлення дефектів та оптимізації архітектури. Ймовірнісні моделі й баєсівські мережі оцінюють статистичні невизначеності для прогнозування появи дефектів та оцінки ризиків відмов. Класичне машинне навчання аналізує історичні метрики, шаблони репозиторіїв та абстрактні синтаксичні дерева для виявлення дефектів, вразливостей і кодових запахів. Нарешті, великі мовні моделі формують сучасний генеративний фронтир, інтерпретуючи специфікації природною мовою для автоматизованої генерації комплексних наборів тестів. Незалежно від того, чи це детермінований

символьний логічний вивід, чи генеративна нейромережева модель, інженерна сутність лишається незмінною: використання методів інтелектуального аналізу для подолання складності програмних систем.



Витоки напряму штучного інтелекту для програмної інженерії сягають парадигми знанняорієнтованих систем 1980-х років.

Фундаментальні концепції того періоду, зокрема модель Knowledge-Based Software Assistant (KBSA), запропонована Гріном у 1983 році, розглядали процес розробки як суворе формальне перетворення: перехід від абстрактної специфікації через верифіковані трансформації безпосередньо до виконуваного коду.

Паралельно в проєкті Массачусетського технологічного інституту Programmer's Apprentice здійснювалося відображення структури проблеми на типові програмні плани для напівавтоматичної побудови програм.

Архітектурно ці платформи спиралися на зв'язку інженерного інтерфейсу та машини логічного виводу, яка виконувала узгодження правил, дедуктивний вивід та планування на основі бази знань із продукційними правилами вигляду «якщо — то».

Попри успішне застосування експертних систем для оцінки ризиків і планування, підхід зіткнувся з критичним вузьким місцем: ручним кодуванням правил інженерами знань. Саме складність набуття знань стала нездоланним бар'єром, що обмежив масштабування таких систем у промисловій інженерії.

Фундаментальна передумова застосування штучного інтелекту в інженерії програмного забезпечення полягає у формалізації розробницьких артефактів як структурованих даних. Будь-яка програмна система є багатовимірним об'єктом, і штучний інтелект сприймає її не просто як абстрактний задум, а через призму строго визначених інформаційних репрезентацій.

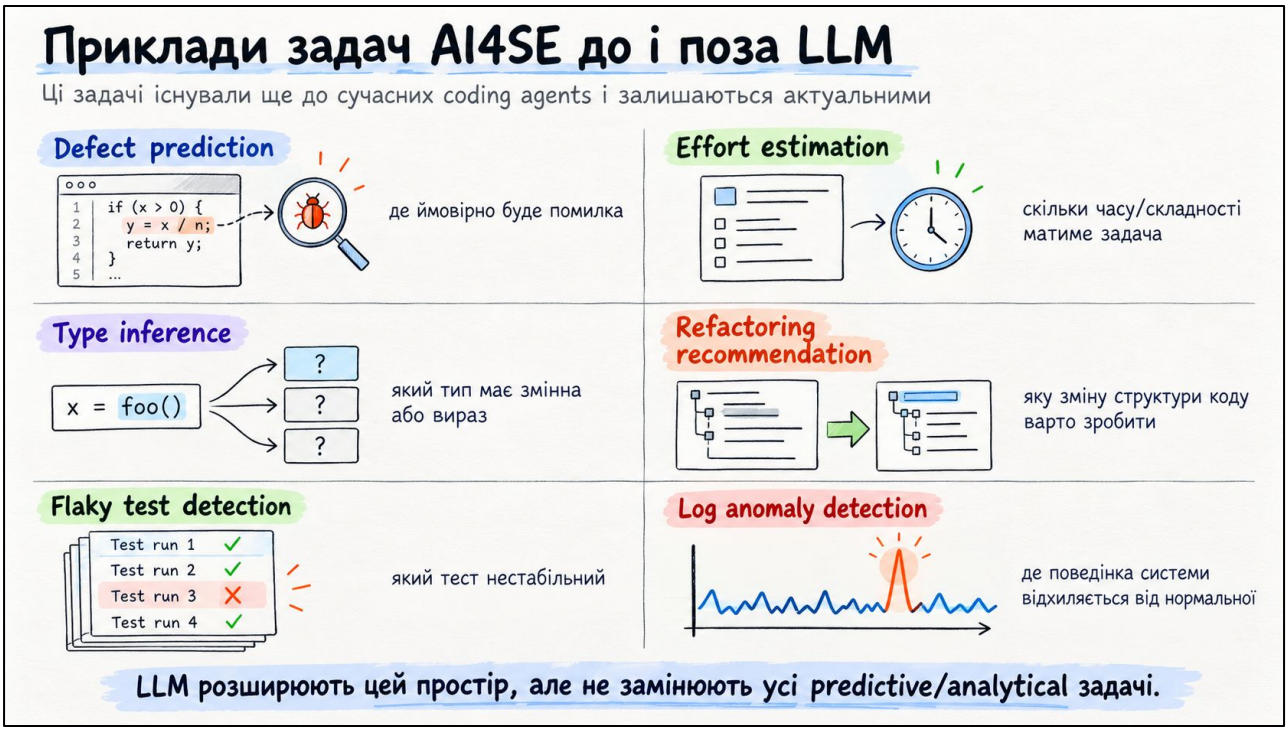
Першим базовим виміром є власне код. На найпростішому рівні моделі інтерпретують його як послідовність лексичних токенів, проте глибокий семантичний аналіз вимагає перетворення синтаксису в абстрактні синтаксичні дерева (AST), графи залежностей і потоки керування. Це дозволяє моделям оперувати як локальною структурою коду, так і глобальною топологією проєкту.

Другий вимір охоплює задачі та формалізацію інженерних намірів: публічні issues, баг-репорти та користувацькі історії (user stories). Вони формують контекст вимог і транслюють бізнес-логіку у формальні інженерні специфікації, поєднуючи природну мову з технічними вимогами.

Третя площина відображає еволюцію системи в часі через репозиторій версій. Фіксації змін (commits), топологія гілок та обговорення в межах pull requests фіксують не просто статичний стан, а вектор архітектурного розвитку, динаміку перетворень і колективні інженерні рішення команди.

Четвертим критичним пластом є динаміка виконання: структуровані логи, розподілені трейси, верифікаційні протоколи тестів і телеметричні метрики. Це сигнали поведінки системи в реальному часі, які замкнені на фазу експлуатації.

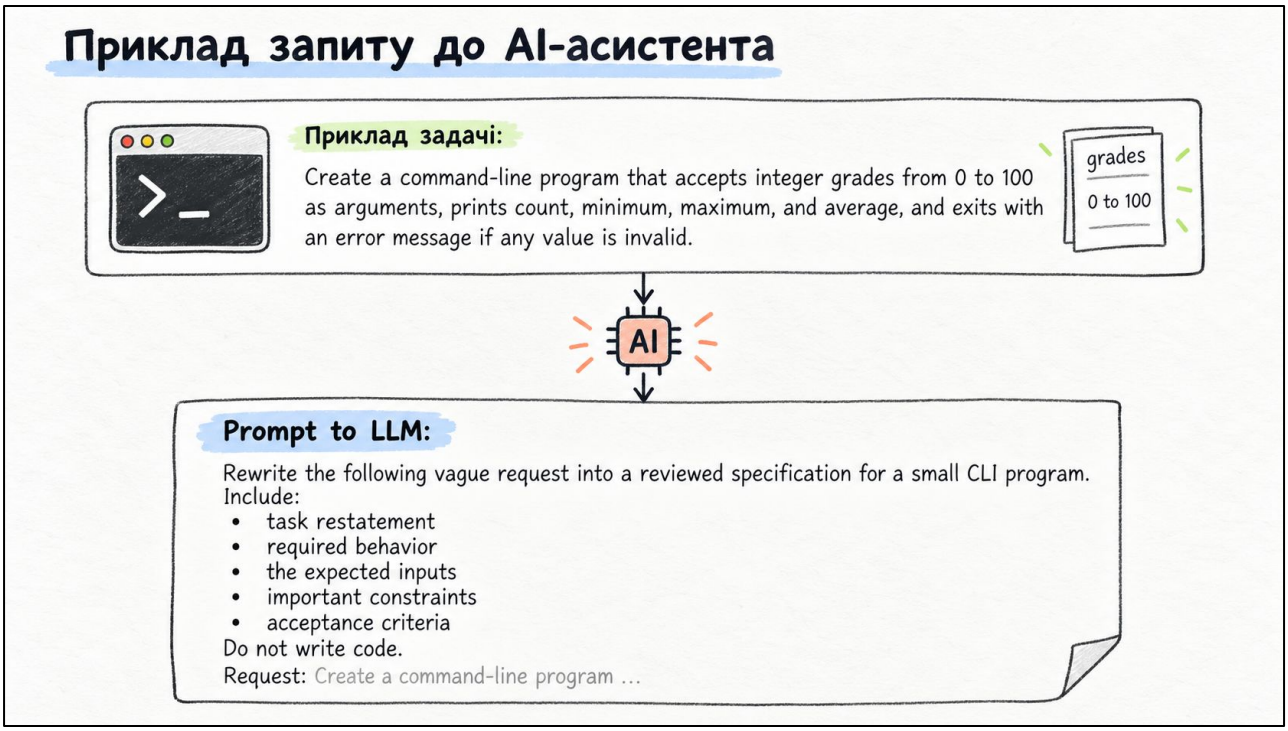
Таким чином, той самий програмний проєкт для моделей штучного інтелекту постає водночас і текстом, і графом, і послідовністю еволюційних змін, і неперервним потоком сигналів виконання. Саме синергія цих чотирьох репрезентацій створює повноцінний контекст для автономного аналізу, надійного синтезу та автоматизованого супроводу програмного забезпечення.



Застосування штучного інтелекту в інженерії програмного забезпечення спирається на ґрунтовний аналітичний фундамент, що виник задовго до появи сучасних генеративних кодових агентів і виходить далеко за межі авторегресійних мовних моделей. Спеціалізовані предиктивні та аналітичні моделі залишаються критично актуальними для вирішення широкого спектра системних інженерних задач. Зокрема, у забезпеченні якості алгоритми прогнозування дефектів аналізують синтаксичні конструкції та виявляють потенційні помилки — наприклад, невалідоване ділення в умовних виразах — ще на етапі статичного коду. Одночасно моделі оцінювання трудовитрат оптимізують проєктне планування, трансформуючи складність декомпонованих задач у часові оцінки виконання.

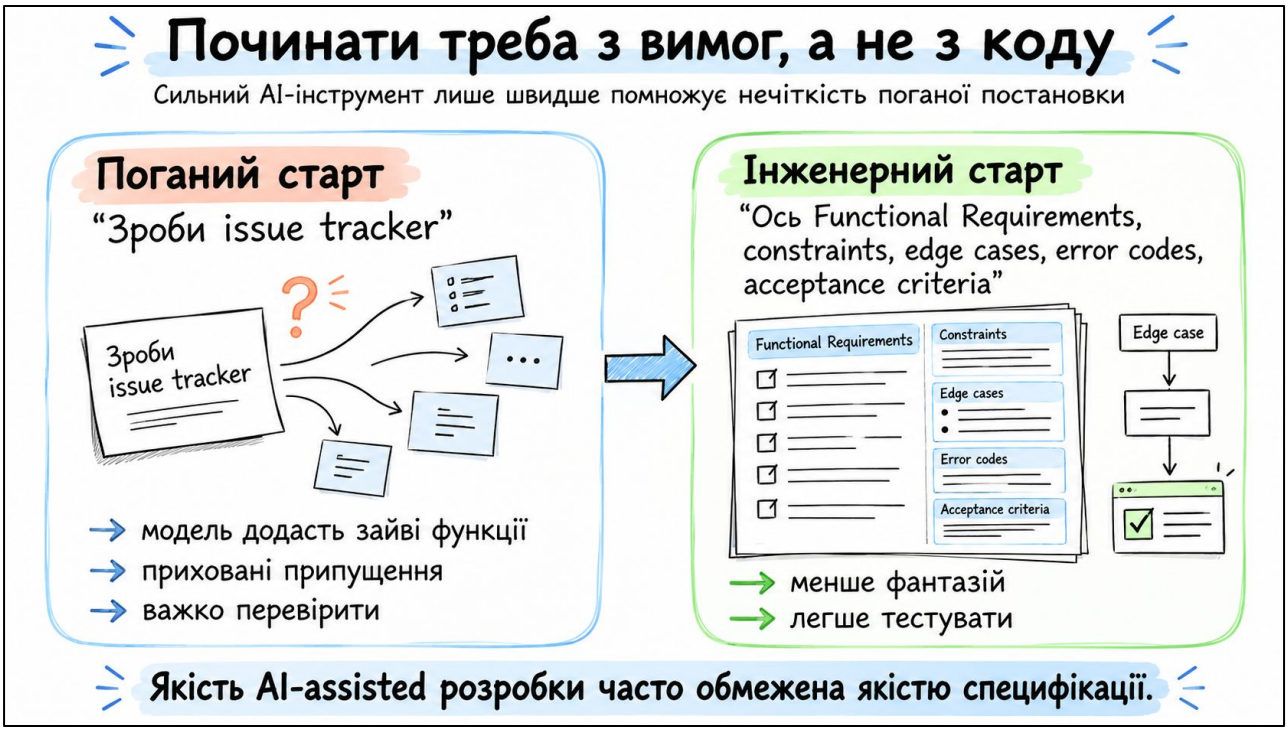
На рівні компіляторів та мовних інструментів алгоритми виведення типів ефективно реконструюють типи змінних і виразів у динамічних мовах, істотно підвищуючи надійність систем. Системи автоматизованих рекомендацій щодо рефакторингу досліджують топологію вихідного коду та пропонують структурні покращення для зниження технічного боргу. У межах тестування алгоритми аналізують результати прогонів для виявлення нестабільних, або так званих flaky-тестів, що порушують детермінованість CI/CD-конвеєрів. Нарешті, на етапі експлуатації моделі детекції аномалій у логах аналізують поведінкові часові ряди та сигналізують про відхилення системи від нормального функціонування. Великі мовні моделі якісно розширюють можливості розробки, проте вони не замінюють

і не скасовують спеціалізовані аналітичні та предиктивні інженерні задачі.



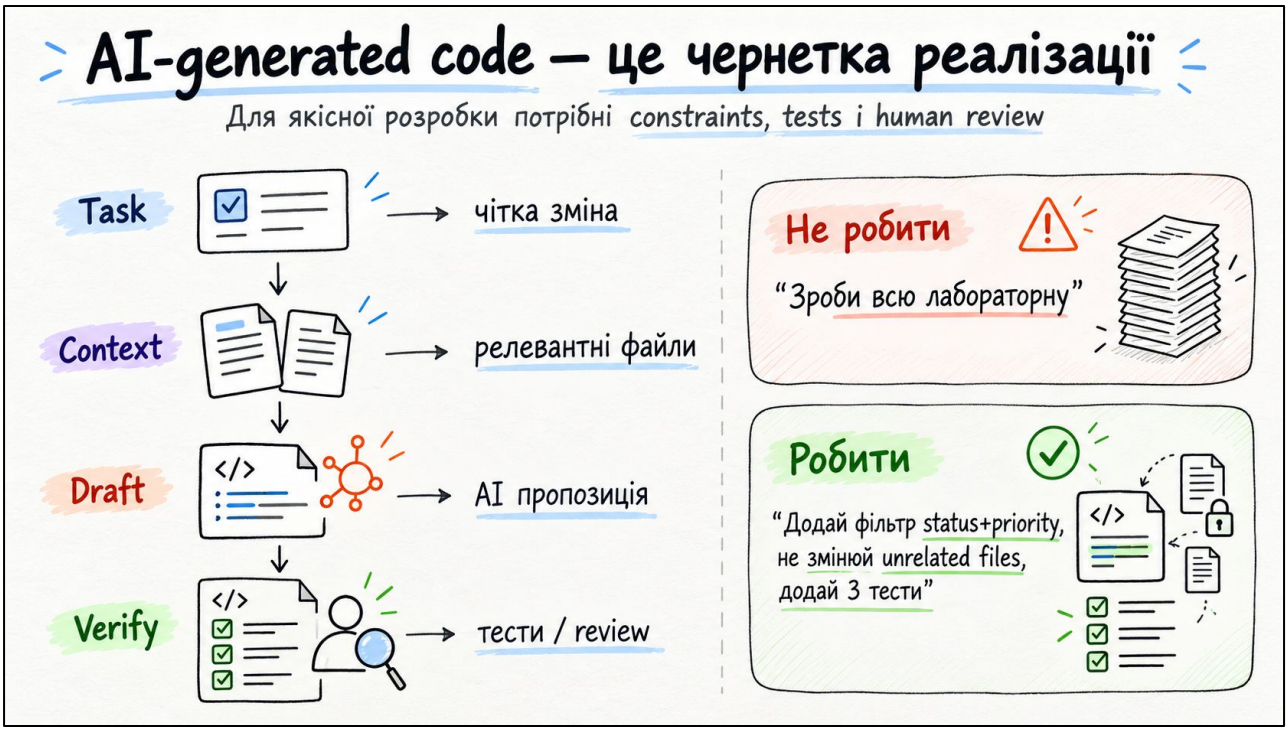
У професійній інженерії програмного забезпечення спроба безпосереднього синтезу коду на основі неформалізованих описів природною мовою неминуче призводить до архітектурних розбіжностей, пропущених граничних випадків та появи вразливих рішень. Коли ми стикаємося з первинною інженерною задачею — наприклад, розробкою консольної утиліти для статистичного аналізу масиву цілочислових оцінок від нуля до ста — першим кроком не повинна бути миттєва генерація коду. Початкова фаза взаємодії з асистентом має зосереджуватися на суворій інженерії вимог.

Шляхом формування мета-промпту для уточнення специфікації ми змушуємо велику мовну модель діяти в ролі системного аналітика, а не звичайного автодоповнювача синтаксису. Моделі ставиться завдання трансформувати неструктурований запит у верифіковану технічну специфікацію для CLI-застосунку. Цей документ повинен містити формалізоване переформулювання задачі, чіткий опис обов'язкової поведінки системи, специфікацію очікуваних вхідних параметрів із валідацією діапазонів, критичні інженерні обмеження та вичерпні критерії прийнятності. При цьому критичною вимогою є пряма заборона на написання коду на цьому етапі. Такий дисциплінований підхід до контекст-інженерії гарантує, що подальша реалізація спиратиметься на детерміновану й узгоджену специфікацію, мінімізуючи дефекти ще до початку написання вихідного коду.



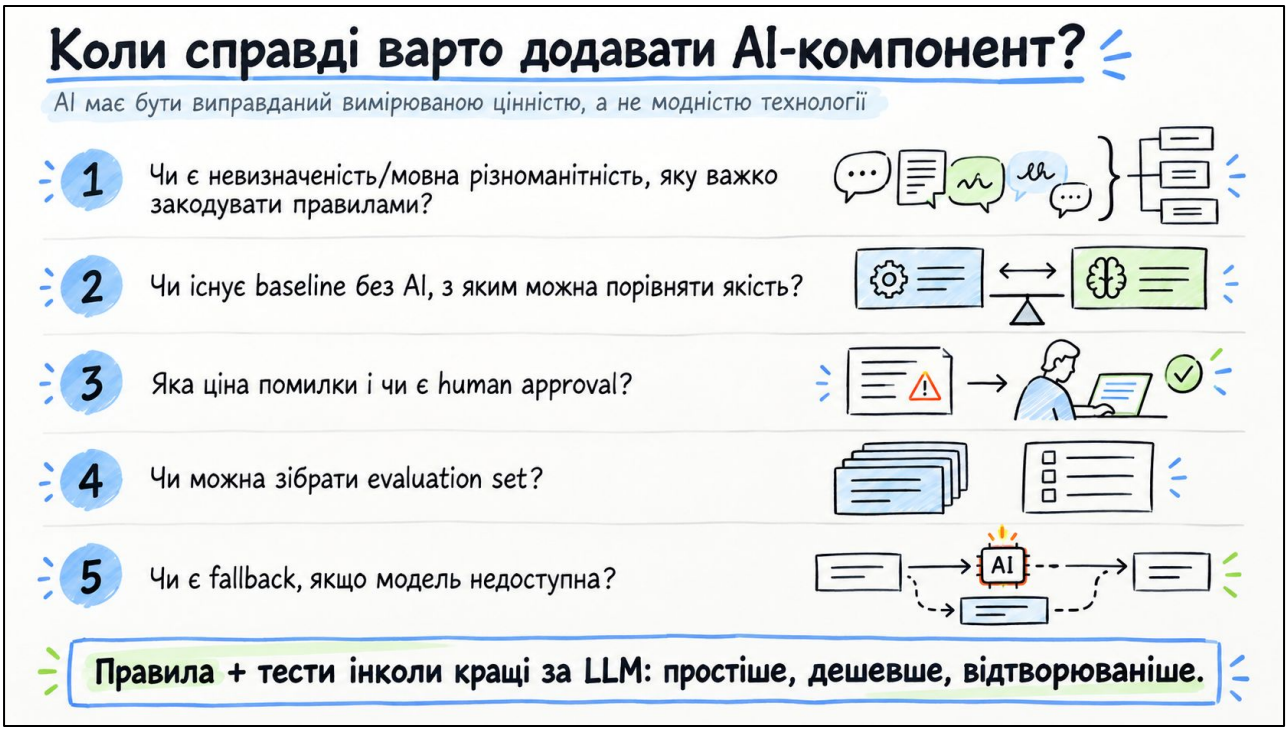
Фундаментальний принцип системної інженерії програмного забезпечення стверджує, що розробка має беззастережно починатися з формалізації вимог, а не з передчасного написання коду. Потужний інструмент штучного інтелекту не усуває концептуальну невизначеність — навпаки, він лише експоненційно примножує нечіткість слабкої або некоректної постановки задачі. Коли інженер починає роботу з абстрактного та неформалізованого запиту — наприклад, «зроби issue tracker», — мовна модель вимушено заповнює смисловий вакуум власними стохастичними припущеннями. Це неминуче веде до розростання надлишкової функціональності, впровадження прихованих архітектурних здогадок і створення складної системи, яку практично неможливо детерміновано протестувати чи верифікувати.

На противагу цьому, зрілий інженерний підхід перетворює взаємодію з асистентом на роботу зі строгою технічною специфікацією. Формування вичерпних функціональних вимог, чітких системних обмежень, граничних випадків, стандартизованих кодів помилок та вимірних критеріїв прийнятності дозволяє жорстко обмежити простір можливих рішень моделі. Така декомпозиція мінімізує імовірність неконтрольованих галюцинацій та фантазій алгоритму, забезпечуючи високу тестованість і надійність кодової бази. У підсумку кінцева якість AI-assisted розробки прямо обмежена глибиною та строгістю первинної інженерної специфікації.



У професійній інженерії програмного забезпечення штучний інтелект у жодному разі не повинен сприйматися як повноцінний автономний автор системного коду: згенерований ШІ код — це виключно чернетка реалізації. Досягнення високої інженерної якості в умовах AI-assisted розробки категорично вимагає трьох взаємопов'язаних компонентів: чітких системних обмежень (constraints), суцільного покриття тестами (tests) та обов'язкового експертного код-рев'ю людиною (human review).

Цей дисциплінований робочий процес розгортається через чотири послідовні фази: формалізацію задачі у вигляді чіткої та атомарної зміни коду, точне формування контексту шляхом ізоляції виключно релевантних файлів, отримання попередньої чернетки-пропозиції від асистента та фінальну верифікацію результату через прогін автоматичних тестів і детальне інспектування. Відповідно, інженерна культура взаємодії з моделлю вимагає уникнення монолітних і розмитих директив, таких як спроба згенерувати всю лабораторну роботу за один крок, що неминуче спричиняє галюцинації та архітектурну деградацію. Натомість правильний інженерний патерн полягає в точкових постановках із чіткими межами: реалізувати конкретний фільтр за статусом і пріоритетом, суворо заборонити модифікацію непов'язаних файлів кодової бази та обов'язково розробити супровідні тести для верифікації нової логіки. Лише сприйняття коду як чорнового варіанта, що підлягає системній валідації, гарантує надійність створюваного програмного продукту.

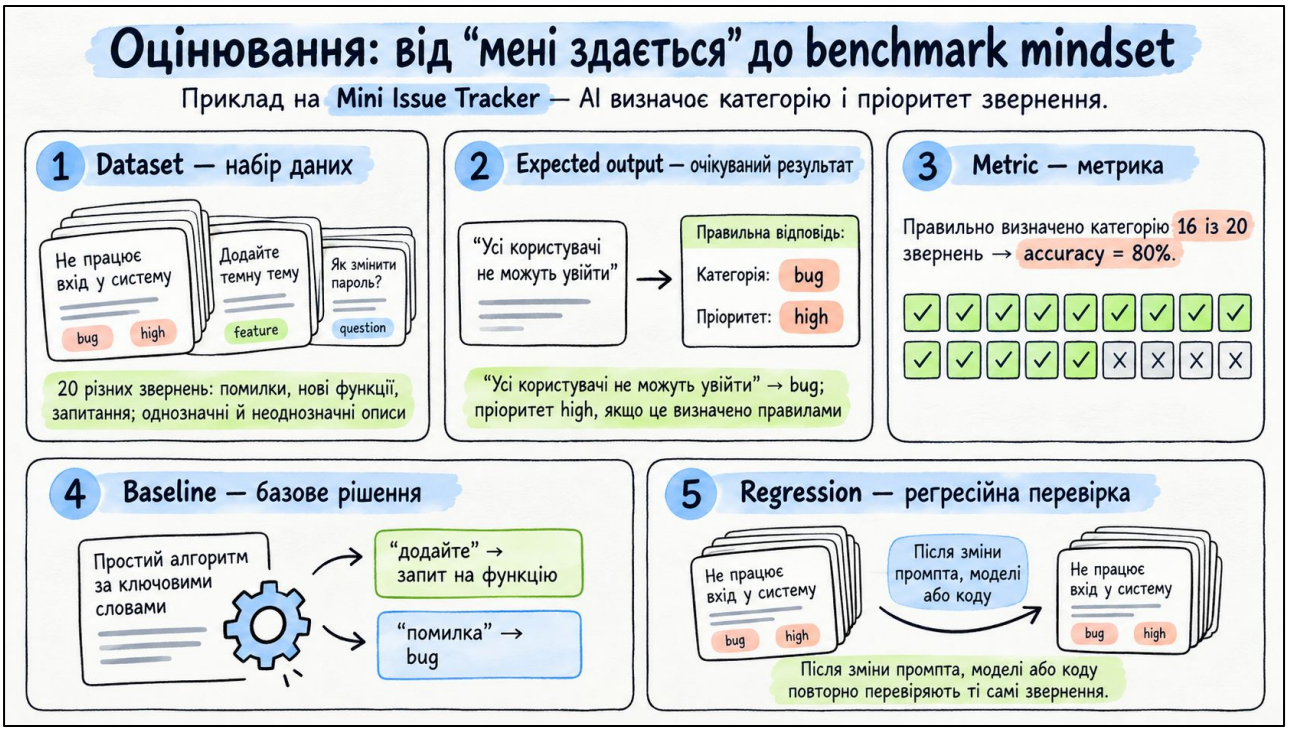


Архітектурне рішення щодо інтеграції штучного інтелекту в програмну систему має базуватися виключно на вимірюваній інженерній та бізнесовій цінності, а не на технологічному хайпі чи маркетингових трендах. Впровадження стохастичного компонента виправдане лише тоді, коли система стикається з високим рівнем невизначеності або складною мовною варіативністю, яку неможливо ефективно та масштабовано описати детермінованими правилами. Якщо задачу можна вирішити класичними алгоритмами або регулярними виразами, використання генеративних моделей є надлишковим.

Обов'язковою передумовою є наявність чіткого baseline без застосування ШІ, що дає змогу об'єктивно зіставити якість, обчислювальні витрати та надійність рішень. Без такого порівняльного базису неможливо довести реальну ефективність нової архітектури. Водночас критичним фактором залишається ціна помилки: у доменах із високими ризиками незворотних наслідків архітектура зобов'язана передбачати обов'язковий етап human approval для верифікації згенерованих дій.

На інженерному рівні життєздатність AI-компонента залежить від можливості зібрати репрезентативний evaluation set для систематичного тестування та моніторингу регресій. Нарешті, забезпечення відмовостійкості вимагає проектування детермінованого fallback-механізму на випадок недоступності або затримки зовнішніх API моделей. Класичні правила та вичерпні тести часто виявляються значно ефективнішими за LLM: вони простіші

в підтримці, дешевші в експлуатації та забезпечують стовідсоткову відтворюваність поведінки системи.



Перехід від суб'єктивного сприйняття до системного мислення на основі бенчмарків є фундаментальною передумовою надійної інтеграції штучного інтелекту в інженерію програмного забезпечення. Інтуїтивні оцінки за принципом «мені здається, що працює» мають бути повністю витіснені формалізованими, відтворюваними експериментальними конвеєрами оцінювання. Розглянемо це на практичному прикладі сервісу класифікації та пріоритизації звернень у системі трекінгу задач, де модель автоматично визначає категорію і рівень критичності кожного тікета.

Процес побудови бенчмарку починається з формування фіксованого набору даних. Референтний датасет містить двадцять варіативних запитів: повідомлення про збої, пропозиції нових функцій та типові запитання користувачів, поєднуючи як чіткі, так і граничні, неоднозначні описи ситуацій. Для кожного прикладу формується еталонний очікуваний результат: звернення про неможливість входу в систему для всіх користувачів за правилами доменної логіки має детерміновано маркуватися як дефект із найвищим пріоритетом.

Якість функціонування системи квантифікується за допомогою строгої метрики: якщо модель коректно класифікувала шістнадцять звернень із двадцяти, точність дорівнює вісімдесяти відсоткам. При цьому нейромережеве рішення обов'язково зіставляється з базовим рівнем — класичним алгоритмом пошуку за ключовими словами, де маркери «додайте» чи «помилка» скеровують потік завдань без залучення складних обчислень. Це дозволяє довести

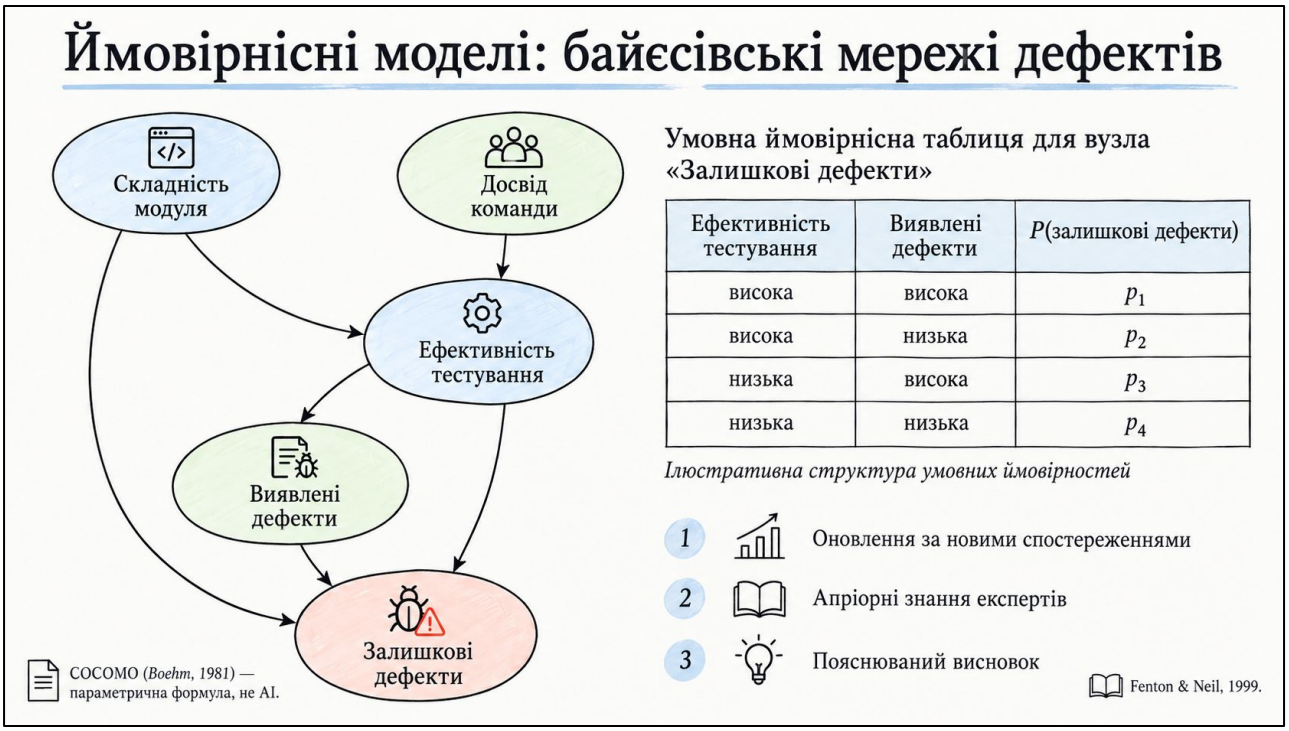
виправданість використання генеративної моделі. Завершальним етапом стає неперервна регресійна перевірка: будь-яка модифікація системного промпту, заміна версії моделі чи зміна коду вимагає обов'язкового повторного прогону фіксованого датасету для запобігання деградації поведінки системи в продуктивному контурі.

Парадигма пошукової інженерії програмного забезпечення (Search-Based Software Engineering, SBSE), концептуалізована Марком Германом та Філом Джонсом у 2001 році, переосмислює класичні інженерні виклики як формальні задачі дискретної та комбінаторної оптимізації. Фундаментом цього підходу виступає неподільна тріада: представлення простору кандидатних рішень, визначення цільової функції пристосованості (fitness function) та вибір ефективного метаевристичного пошукового алгоритму.

У контексті автоматизованої генерації тестових наборів обчислювальний процес розгортається за канонічним генетичним циклом. Алгоритм стартує з випадкової генерації початкової популяції тестових наборів. На кожній ітерації здійснюється суворе кількісне оцінювання пристосованості кожної особини, де реалізується багатокритеріальна оптимізація: алгоритм максимізує метрику покриття гілок коду (branch coverage), одночасно накладаючи регуляризаційний штраф за довжину тестового набору через ваговий коефіцієнт лямбда. Після цього на основі турнірного відбору формується пул батьківських особин, до яких застосовуються генетичні оператори кросоверу та мутації для синтезу нової генерації рішень.

Цикл виконується ітеративно до моменту повного вичерпання виділеного обчислювального бюджету часу або ресурсів процесора, повертаючи в результаті глобально субоптимальний, найкращий верифікований тестовий набір. Практична реалізація SBSE знаходить своє втілення у промислових

інструментах на кшталт EvoSuite для автоматичної генерації юніт-тестів, алгоритмах динамічної пріоритезації регресійних тестів у конвеєрах неперервної інтеграції, а також у задачах оптимізації вибору вимог під час планування релізів програмного забезпечення.



Кількісна оцінка надійності програмного забезпечення вимагає точного моделювання невизначеності між взаємопов'язаними інженерними факторами, виходячи за межі детермінованих параметричних формул на кшталт класичної моделі COCOMO Боема.

Запропонований Фентоном і Нілом у 1999 році підхід на основі байєсівських мереж довіри формує суворий імовірнісний каркас для моделювання причинно-наслідкових залежностей при оцінюванні дефектів.

У межах орієнтованого ациклічного графа структурні атрибути, такі як складність модуля, та організаційні параметри, зокрема досвід команди, безпосередньо обумовлюють ефективність тестування та кількість виявлених дефектів.

Ці проміжні вузли передають імовірнісний розподіл до цільового стану — залишкових дефектів.

За допомогою умовних ймовірнісних таблиць система формалізує багатовимірні сценарії, розраховуючи ймовірність прихованих дефектів для різних комбінацій якості тестування та кількості знайдених помилок. Такий підхід забезпечує три ключові переваги: динамічне оновлення ймовірностей на основі нових спостережень, систематичну інтеграцію апріорних експертних знань за умов дефіциту емпіричних даних та прозорий, пояснюваний логічний вивід для проактивного управління архітектурними ризиками перед релізом.

Фундаментальний парадокс автоматизованого виправлення програм полягає в тому, що проходження тестового набору не є еквівалентом семантичної коректності коду. Коли Клер Ле Гуес та колеги презентували систему GenProg на конференції ICSE 2012 року, генетичні алгоритми продемонстрували автоматизоване усунення 55 зі 105 дефектів за середньої вартості хмарних обчислень близько восьми доларів на дефект.

Проте подальший емпіричний аналіз Ці та співавторів на ISSTA 2015 року виявив критичну проблему: значна частка згенерованих патчів лише обходила слабкі тести, а не розв'язувала дефект по суті.

У просторі всіх можливих змін коду правдоподібні патчі проходять наявні тести, проте лише невелика їх частина є по-справжньому коректною й відповідає вихідній специфікації, зберігаючи необхідну функціональність.

Значна кількість рішень просто видаляє корисну бізнес-логіку — наприклад, усуває обов'язкову перевірку прав адміністратора, щоб примусово повернути очікуване значення, внаслідок чого всі юніт-тести лишаються «зеленими».

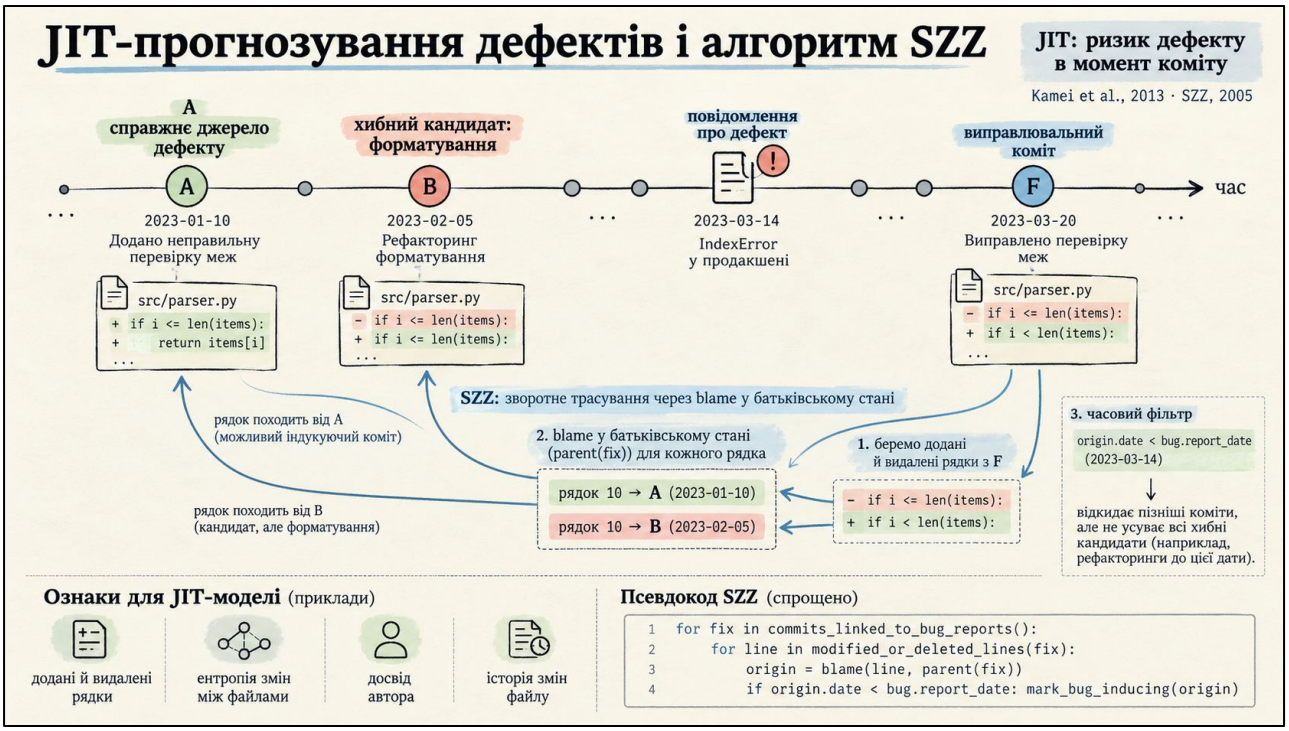
Точно такий самий ризик характерний і для сучасної LLM-генерації коду: мовні моделі здатні синтезувати поверхнево успішні патчі, які формально задовольняють вузькі тестові твердження, але непомітно руйнують архітектурну цілісність та інваріанти безпеки системи.

Парадигма машинного навчання для програмної інженерії, або ML4SE, позначає фундаментальний відхід від евристичних експертних систем завдяки формуванню емпіричного контуру зворотного зв'язку на основі репозиторіїв розробки.

Замість ручного кодування жорстких правил знання системно виводяться безпосередньо з історії програмного проєкту. Цей підхід спирається на видобування даних із репозиторіїв програмного забезпечення (MSR) для отримання первинної телеметрії: комітів, міжфайлових дифів, задач у трекерах, журналів безперервної інтеграції та метрик вихідного коду. На етапі екстракції ознак ці гетерогенні дані перетворюються на абстрактні синтаксичні дерева, графові зв'язки та матричні структури.

Після розмітки за категоріями помилок, функціональності чи рефакторингу відбувається тренування, валідація та тестування моделей, що розв'язують формальні задачі класифікації, регресії, кластеризації або ранжування.

Навчена модель інтегрується безпосередньо в робочий простір інженера — середовище розробки (IDE) або конвеєр CI — видаючи рекомендації щодо виправлення багів, доповнення тестів чи оптимізації коду. Завершальним критичним елементом є контур зворотного зв'язку: фіксація реакцій інженера на рекомендації постачає нові зразки й уточнені мітки для повторного навчання, забезпечуючи безперервну адаптацію моделей до динаміки кодової бази.



Прогнозування дефектів у момент здійснення коміту (Just-in-Time, JIT) докорінно трансформує підхід до забезпечення якості, оцінюючи прихований ризик внесення помилки безпосередньо під час фіксації змін у репозиторії.

Спираючись на фундаментальне дослідження Камеї та співавторів 2013 року, цей підхід безпосередньо залежить від ретроспективної розмітки індукуючих комітів — завдання видобування даних, яке вирішує класичний алгоритм SZZ, запропонований у 2005 році.

Механіка SZZ ґрунтується на зворотному трасуванні змін через аналіз blame у батьківському стані системи контролю версій. Після реєстрації повідомлення про дефект та появи виправлювального коміту алгоритм виділяє додані, модифіковані й видалені рядки. Виконуючи операцію blame відносно батьківського стану виправлення, він відстежує історію походження цих рядків у глибину репозиторію. Проте наївне відстеження створює суттєвий шум: вторинний рефакторинг та форматування коду формують хибних кандидатів. Хоча часовий фільтр відсікає всі коміти, створені після офіційної дати фіксації дефекту, він не здатен автоматично усунути всі попередні хибні спрацьовування, зумовлені суто синтаксичними змінами.

Отриманий після фільтрації масив істинно індукуючих комітів стає базою для навчання класифікаторів JIT-моделей. Ці статистичні моделі аналізують багатовимірні ознаки коміту: обсяг доданих і видалених рядків, ентропію розподілу змін між файлами, досвід автора та історію змін модифікованого

файлу. Це дозволяє проактивно блокувати регресії безпосередньо в конвеєрах неперервної інтеграції до моменту їхнього потрапляння у виробниче середовище.

Інші задачі ML4SE

Задача	Тип ML	Вхідні дані	Вихід	Метрика
Triage issues	класифікація	текст issue, історія	відповідальний	F1
Дублікати баг-репортів	ранжування	тексти звітів	схожі звіти	MAP
Класифікація комітів	класифікація	diff, повідомлення	fix / feature / refactor	F1
Оцінка трудомісткості	регресія	вимоги, історія задач	story points / години	MAE
Прогноз flaky tests	класифікація	історія прогонів CI	ризик нестабільності	AUC

Flaky test: результат змінюється без змін коду

Застосування машинного навчання в інженерії програмного забезпечення виходить далеко за межі прогнозування дефектів, охоплюючи широкий спектр операційних задач сучасного життєвого циклу розробки.

В управлінні задачами автоматизований тріаж (issue triage) застосовує класифікацію тексту та історії звернень для точного призначення найбільш кваліфікованого відповідального інженера, орієнтуючись на максимізацію F1-міри.

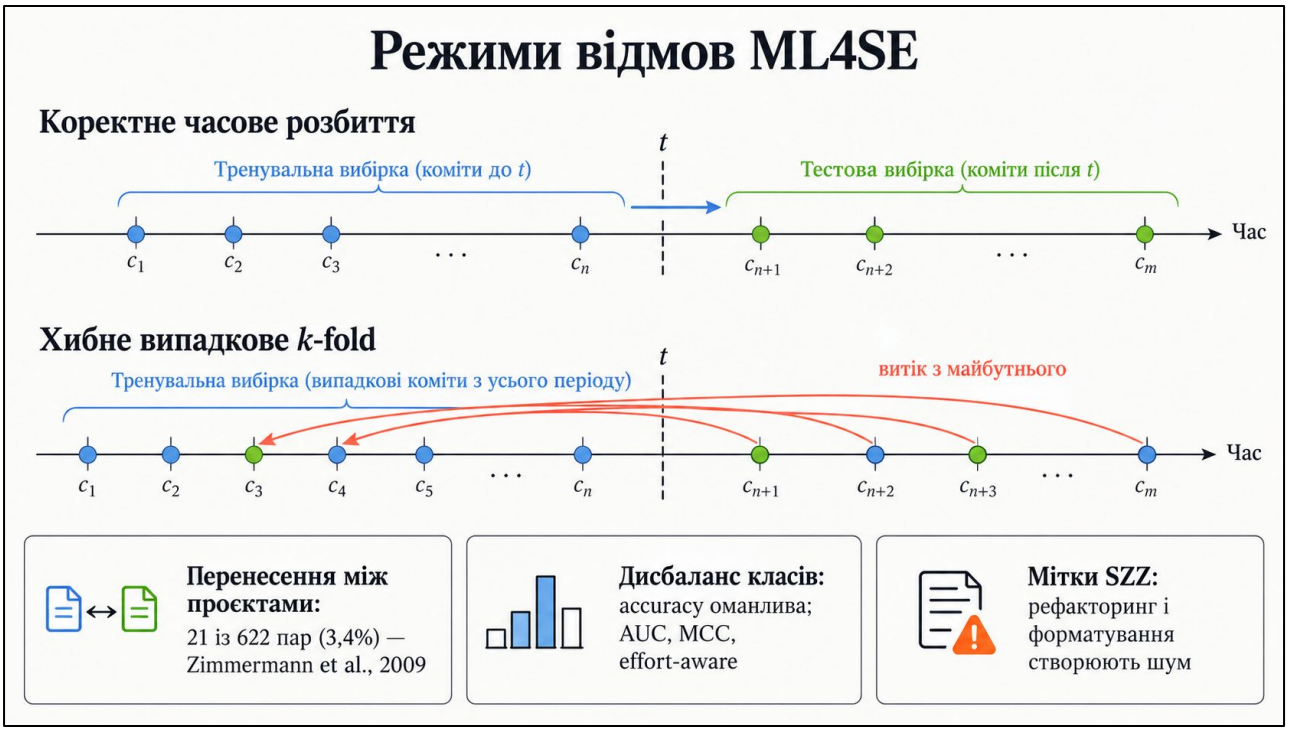
Виявлення дублікатів баг-репортів формалізується як задача ранжування, де на основі семантичного аналізу текстів звітів формується ранжований перелік схожих інцидентів, якість якого вимірюється метрикою середньої точності (MAP).

У конвеєрах контролю версій класифікація комітів аналізує кодові дифи та супровідні повідомлення для категоризації змін на виправлення дефектів, нову функціональність або рефакторинг, що оцінюється збалансованою F1-метрикою.

Для планування розробки моделі регресії обробляють вимоги та ретроспективну історію задач для точної оцінки трудомісткості у story points або інженерних годинах, мінімізуючи середню абсолютну похибку (MAE).

Нарешті, у конвеєрах безперервної інтеграції класифікаційні моделі досліджують історію прогонів CI для завчасного прогнозування flaky-тестів — нестабільних тестів, результат виконання яких змінюється без внесення змін у вихідний код, що валідується за допомогою метрики площі під кривою (AUC).

Таке системне відображення інженерних викликів на класичні парадигми машинного навчання трансформує емпіричні репозиторні дані на керовану виробничу аналітику.



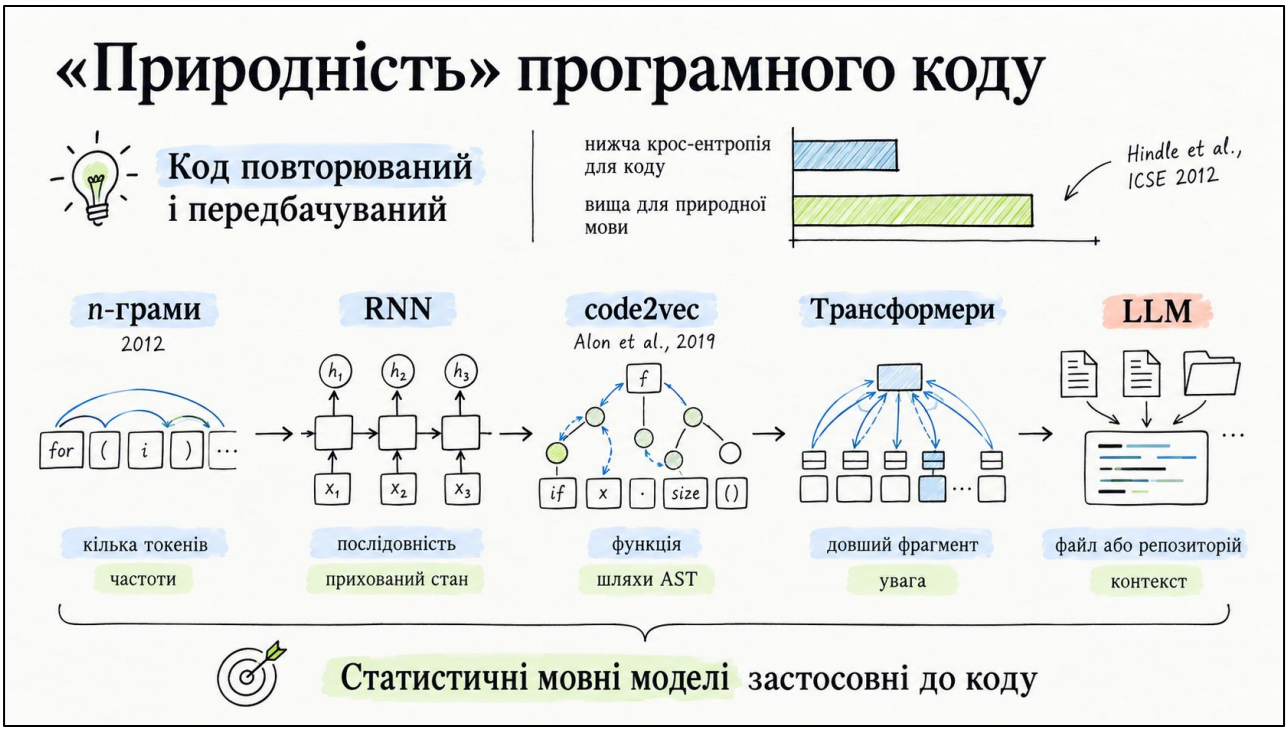
Надійність емпіричних моделей машинного навчання для програмної інженерії безпосередньо залежить від своєчасного виявлення та запобігання типовим методологічним режимам відмов. Фундаментальним аспектом тут виступає суворе збереження хронологічної послідовності під час розбиття даних.

За коректного часового спліту тренувальна вибірка повинна містити виключно ретроспективні коміти до моменту часу t, тоді як тестовий набір ізолюється подіями, зафіксованими суворо після цієї межі. Застосування наївного випадкового k-fold розбиття є критичною помилкою: довільне перемішування комітів по всій часовій шкалі призводить до витоку даних із майбутнього в минуле, формуючи оманливо завищені оцінки якості моделі.

Окрім порушення часового розбиття, надійність рішень підриває низка системних інженерних пасток. Перенесення навчених моделей між різними проєктами демонструє вкрай низьку стабільність; класичне дослідження Ціммерманна та колег 2009 року засвідчило успішне міжпроєктне прогнозування лише для двадцяти однієї з шестисот двадцяти двох пар репозиторіїв, що становить мізерні 3,4%.

Ситуацію ускладнює суттєвий дисбаланс класів, при якому стандартна точність (accuracy) стає абсолютно оманливою, вимагаючи переходу до метрик AUC, коефіцієнта кореляції Метьюза (MCC) та підходів з урахуванням витрачених зусиль (effort-aware). Нарешті, автоматична розмітка за алгоритмом SZZ вносить суттєвий шум через хибне включення рефакторингу й

форматування коду до навчальної вибірки. Без усунення цих викривлень перехід від експериментів до стійких промислових конвеєрів залишається недосяжним



Фундаментальною концепцією сучасного машинного навчання для вихідного коду є гіпотеза «природності», експериментально обґрунтована Хіндлом та колегами на конференції ICSE 2012 року.

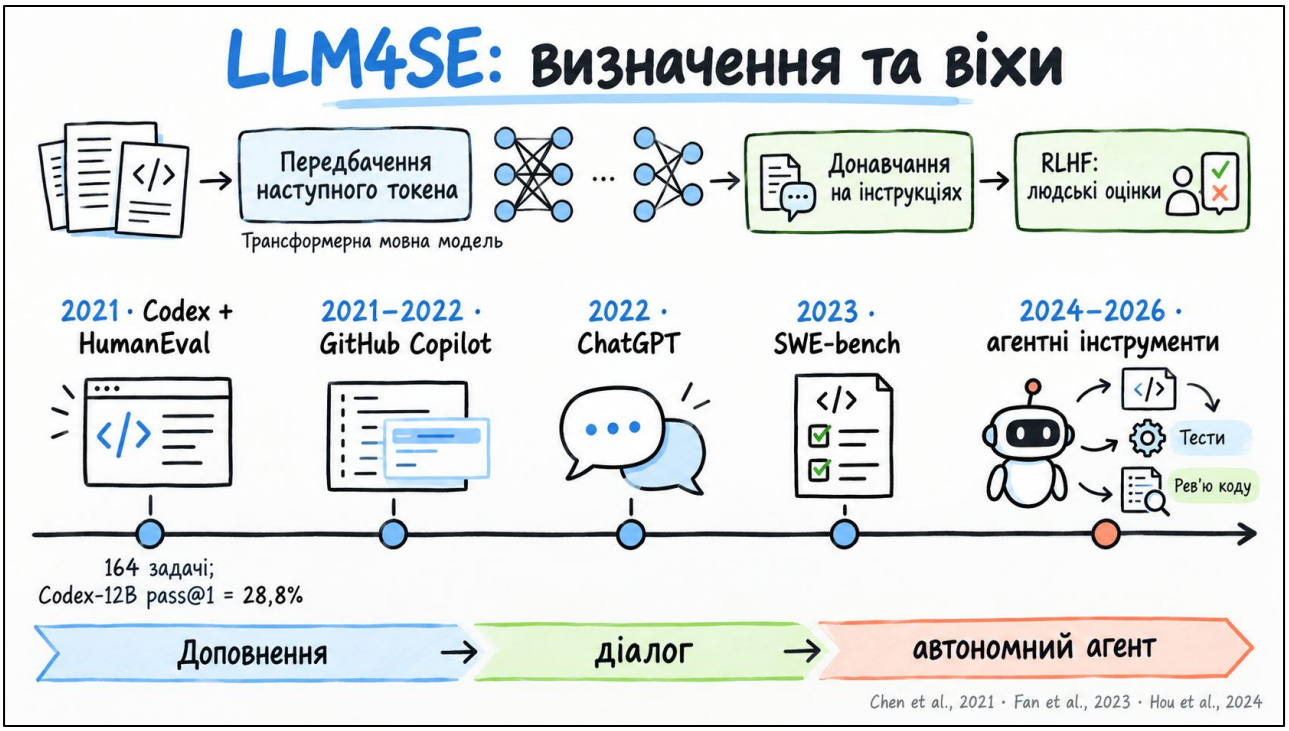
Програмне забезпечення, створюване людьми для вираження інженерного задуму, характеризується високою повторюваністю та системною передбачуваністю структури. Емпіричні дослідження показують, що вихідний код має значно нижчу крос-ентропію порівняно з природною людською мовою, підтверджуючи його вищу статистичну повторюваність. Саме ця закономірність створює математичну базу для прямого застосування статистичних мовних моделей до кодових артефактів.

Еволюція архітектур моделювання коду демонструє невпинне масштабування контексту та ускладнення структурних репрезентацій.

У 2012 році базові підходи обмежувалися n-грамами, які відстежували частоту кількох послідовних токенів. Рекурентні нейронні мережі додали приховані стани для аналізу послідовностей змінних і викликів.

У 2019 році Алон та співавтори презентували модель code2vec, що інтегрувала синтаксичні знання через векторизацію шляхів в абстрактних синтаксичних деревах (AST) на рівні функцій. Поява трансформерів дозволила за допомогою механізму уваги охоплювати довгі кодові фрагменти. На сучасному етапі великі мовні моделі здатні оперувати контекстом цілих файлів і розподілених репозиторіїв, забезпечуючи глибинний семантичний аналіз

системної архітектури.



Парадигма великих мовних моделей для програмної інженерії (LLM4SE) сформувалася завдяки поєднанню авторегресійного навчання базових моделей, донавчання на інструкціях та вирівнювання за допомогою навчання з підкріпленням на основі людських оцінок (RLHF).

На базовому рівні трансформерна мовна модель обробляє програмний і природномовний текст через задачу передбачення наступного токена, формуючи стійкі синтаксичні та семантичні представлення. Далі модель проходить етап донавчання на інструкціях для виконання інженерних завдань і калібрується через оцінки людей для мінімізації галюцинацій та дотримання стандартів якості коду.

Цей технологічний процес пройшов чіткі історичні віхи. У 2021 році випуск моделі Codex та бенчмарку HumanEval зафіксував базовий рівень метрики pass@1 у 28,8% на 164 функціональних задачах, започаткувавши еру синтезу коду. Протягом 2021–2022 років поява GitHub Copilot перетворила цю технологію на практичний інструмент автодоповнення коду в IDE. Наприкінці 2022 року ChatGPT відкрив можливість інтерактивного діалогу для рефакторингу, аналізу помилок та обговорення архітектурних рішень. Поява бенчмарку SWE-bench у 2023 році перенесла фокус тестування з окремих функцій на розв'язання реальних репозиторних задач і дефектів.

Нарешті, період 2024–2026 років знаменує утвердження автономних агентних інструментів, здатних самостійно модифікувати код між кількома

файлами, запускати тестові конвеєри та проводити рев'ю коду. Ця траєкторія чітко відображає концептуальну зміну ролі моделі: перехід від локального автодоповнення через інтерактивний діалог до повноцінного автономного агента в життєвому циклі розробки.

LLM4SE: спектр задач

	Код	Тести	Документація	Вимоги	Конфігурації / IaC
Генерація	нова функція	тест-кейси	README	user stories	Terraform-модуль
Пояснення	розбір функції	причина падіння	стислий виклад	тлумачення правила	пояснення YAML
Трансформація	Java 8 → 17	оновлення тестів	зміна формату	розбиття історії	міграція CI
Аналіз	code review	прогалини покриття	неузгодженості	суперечності	ризики конфігурації

ML4SE:
вузька модель для задачі

→

LLM4SE:
загальна модель + промпт

Промпт і контекст теж версіонують і перевіряють

Функціональний простір великих мовних моделей для програмної інженерії (LLM4SE) розгортається як багатовимірна матриця, що проєктує базові когнітивні операції на ключові інженерні артефакти.

По горизонталі цей спектр охоплює вихідний код, тести, технічну документацію, специфікації вимог та конфігурації інфраструктури як коду (IaC). По вертикалі операційні можливості структуруються на чотири рівні: генерацію, пояснення, трансформацію та аналіз.

На рівні генерації моделі забезпечують синтез нових функцій, створення тест-кейсів, підготовку документації README, декомпозицію user stories та побудову модулів Terraform. У площині пояснення модель здійснює семантичний розбір кодових функцій, визначає першопричини падіння тестів, формує стислий виклад складної документації, тлумачить бізнес-правила й пояснює синтаксис конфігурацій YAML.

Рівень трансформації відповідає за масштабну модернізацію: міграцію застарілого синтаксису з Java 8 на Java 17, оновлення тестових тверджень, зміну форматів даних, розбиття користувацьких історій та рефакторинг конвеєрів неперервної інтеграції. Нарешті, аналітичний блок фокусується на автоматизованому code review, виявленні прогалин у покритті коду, усуненні неузгодженостей у документації, пошуку суперечностей у вимогах та аудиті ризиків безпеки в конфігураціях.

Ця архітектурна структура фіксує фундаментальний парадигмальний

зсув: перехід від класичного підходу ML4SE, де під кожну вузьку задачу навчалася окрема модель, до парадигми LLM4SE, де універсальна мовна модель адаптується через формалізацію промпту.

Відповідно, сам промпт та сформований контекст набувають статусу повноцінних інженерних артефактів, які підлягають суворому версіонуванню, контролю змін та систематичній верифікації.

Фундаментальне дослідження Скаллі та колег на конференції NeurIPS 2015 року формалізує ключову архітектурну реальність: у реальних виробничих системах машинного навчання безпосередній математичний код моделі становить лише мізерну частку загального обсягу інженерного рішення.

Переважна більшість системної складності зосереджена у навколишній інфраструктурі інженерії програмного забезпечення.

Надійне функціонування вимагає побудови конвеєрів збору даних, шлюзів верифікації даних, декларативного управління конфігураціями, оркестрації апаратних ресурсів, безперервного моніторингу та інфраструктури обслуговування.

Така масштабна інфраструктурна обв'язка породжує критичний прихований технічний борг через нелінійну зв'язність компонентів. Показовим є принцип CACE — зміна однієї ознаки впливає на всі інші, коли найменша корекція вхідного сигналу лавиноподібно дестабілізує весь граф ознак та підриває передбачуваність результатів. Ситуацію ускладнюють неоголошені споживачі, які неявно спираються на виходи моделі, а також приховані петлі зворотного зв'язку, коли згенеровані моделлю дані повертаються у тренувальні вибірки наступних ітерацій. В епоху генеративних систем цей ландшафт технічного боргу суттєво розширюється. Недетерміновані промпти, динамічні шаблони та інструменти агентів формують нові джерела крихкості, які вимагають суворої інженерної дисципліни для забезпечення стійкості всієї системи.

Процес інженерії програмного забезпечення для систем штучного інтелекту — SE4AI — є складним, циклічним життєвим циклом із множинними контурами зворотного зв'язку, формалізованим у дослідженні Амерші та колег 2019 року. Розробка розпочинається з формалізації вимог до моделі: визначення бізнес-цілей, цільових користувачів, ресурсних обмежень та чітких критеріїв успіху. Далі розгортається конвеєр підготовки даних: ідентифікація репрезентативних джерел, аудит прав доступу, глибоке очищення від шумів і дублікатів, а також регламентована розмітка із залученням людини в циклі (human-in-the-loop) та суворим контролем міжоцінювальної узгодженості.

Сформовані масиви переходять на етап конструювання та селекції текстових і статистичних ознак із залученням знань предметної області. Після цього здійснюється безпосереднє навчання моделі, що охоплює експерименти з архітектурою, тюнінг гіперпараметрів і масштабування обчислень. Критичним шлюзом виступає етап оцінювання, де аналіз помилок, розрахунок цільових метрик і порівняння з базовими лініями визначають готовність системи до запуску, формуючи ітеративні повернення до збору даних, інженерії ознак або уточнення вихідних вимог. За успішного проходження валідації модель розгортається у виробничу інфраструктуру у вигляді API-сервісу з контролем версій та політиками безпеки. Фінальна фаза моніторингу відстежує деградацію якості в реальному часі, дрейф даних (data drift), аномалії та зворотний зв'язок користувачів, за потреби ініціюючи глобальний перегляд системних вимог.

В епоху сучасних генеративних застосунків цей класичний цикл доповнюється специфічними інженерними рівнями контролю. Інженерія промптів вимагає суворого версіонування шаблонів, ведення реєстру змін та проведення A/B-тестування. Недетермінованість поведінки контролюється через автоматизовані конвеєри безперервного оцінювання (evals) для запобігання поведінковій регресії. Периметр безпеки гарантується інтеграцією вхідних і вихідних бар'єрів (guardrails), які виконують санітизацію промптів, фільтрацію небажаного контенту та аудит політик. Нарешті, експлуатаційна стабільність вимагає прецизійного обліку вартості токенів через динамічну оптимізацію запитів, бюджетні ліміти та системи сповіщень.

В інженерії систем штучного інтелекту каскади даних (data cascades) становлять один із найнебезпечніших відкладених режимів відмов, спричинених накопиченням прихованих проблем якості вхідних даних. Дослідження Самбасіван та колег на CHI 2021 року показало, що дев'яносто два відсотки опитаних практиків стикалися з такими системними каскадними збоями в промислових середовищах. Ланцюг деградації розпочинається з первинних дефектів даних: пропусків, некоректних діапазонів, статистичних зміщень або неоголошених змін у джерелах надходження. Якщо інфраструктура позбавлена суворих шлюзів валідації даних, які контролюють схеми, межі значень та аномалії, пошкоджені записи безперешкодно потрапляють у конвеєр інференсу.

Таке неконтрольоване надходження інформації призводить до непомітної деградації моделі. Зміна розподілів ознак у продакшні відносно тренувальних даних спричиняє коваріаційний зсув (дрейф), знижуючи узагальнюючу здатність алгоритмів і стабільно погіршуючи прогнози без генерації явних програмних винятків. Класичним прецедентом став інцидент із Google Flu Trends у сезоні 2012–2013 років, коли неврахований дрейф даних призвів до переоцінки піку епідемії майже вдвічі порівняно з реальними показниками.

Для усунення таких ризиків архітектура вимагає безперервного моніторингу розподілів — зокрема розрахунку індексу стабільності популяції (PSI) та дивергенції Кульбака — Лейблера з налаштуванням алертів на відхилення. Відсутність такого контролю неминуче переростає в

повномасштабний бізнес-інцидент: помилкові автоматизовані рішення, прямі фінансові збитки, зниження конверсії та втрату довіри клієнтів.

Забезпечення комплексної готовності системи до промислової експлуатації вимагає впровадження підходів на зразок ML Test Score, що формалізують багаторівневий аудит даних, моделей та інфраструктури моніторингу.

Порівняння чотирьох напрямів

Режими відмов

Параметр	AI4SE	ML4SE	LLM4SE	SE4AI
Об'єкт	процес / артефакт SE	артефакти як дані	код і текст	AI-система
Метод	будь-який AI	ML	LLM	практики SE
Вихід	план, тест, прогноз	класифікація, оцінка	згенерований артефакт	керована система
Метрика	залежить від задачі	AUC, MCC, MAE	pass@k, % issues	SLO, дрейф, evals
Домінантний ризик	знання / fitness	витік / перенесення	галюцинації / варіативність	техборг / дрейф
Період	з 1980-х	з 2000-х	з 2021	з 2015

Порівняльний аналіз чотирьох напрямів перетину штучного інтелекту та програмної інженерії формує чітку таксономію архітектурних підходів і властивих їм режимів відмов. Базовий напрям AI4SE, історія якого розпочалася ще у 1980-х роках, розглядає інженерні процеси та артефакти розробки через призму широкого спектра методів штучного інтелекту — від символьного виводу до еволюційних алгоритмів.

Виходом тут виступають плани проєктів, тестові набори та прогнози дефектів, де метрики визначаються конкретною задачею, а домінантними ризиками є складність набуття знань та формалізація цільових функцій пристосованості.

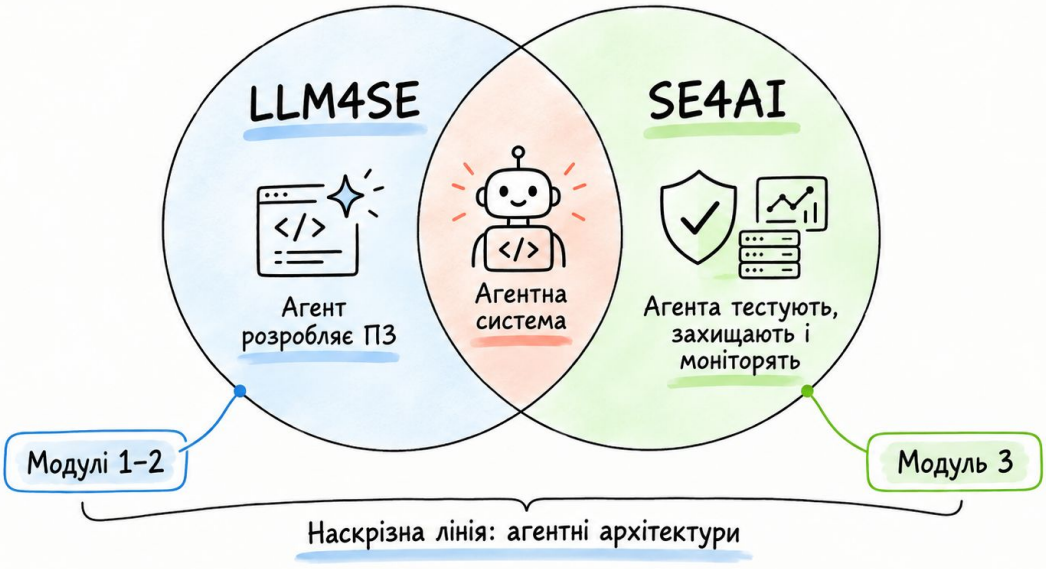
Починаючи з 2000-х років, парадигма ML4SE переосмислила репозиторні артефакти як емпіричні навчальні дані. Застосування класичного машинного навчання дозволило отримати точні класифікації та регресійні оцінки, верифіковані метриками AUC, MCC та MAE, проте зіткнулося з критичними режимами відмов: витоком даних із майбутнього та вкрай слабким міжпроєктним перенесенням навчених моделей.

Сучасний період, започаткований у 2021 році, ознаменований розвитком LLM4SE, де об'єктом маніпуляцій стає контекст вихідного коду та документації природною мовою. Використання великих мовних моделей забезпечує синтез повноцінних кодових артефактів, якість яких оцінюється за бенчмарками pass@k та часткою розв'язаних задач у репозиторіях. Водночас ключовим джерелом

відмов тут стають недетермінованість, галюцинації та висока варіативність генерації.

Нарешті, парадигма SE4AI, що набула системної ваги з 2015 року, кардинально змінює вектор: тут сама AI-система виступає об'єктом застосування класичних практик програмної інженерії для побудови надійного та керованого виробничого комплексу. Якість таких систем вимірюється через SLO, метрики дрейфу та набори evals, а основними ризиками залишаються прихований технічний борг, дрейф даних та небезпечні каскадні збої інфраструктури.

Позиціонування курсу



Математичний ландшафт генеративного моделювання структурується на чотири фундаментальні парадигми залежно від способу наближення або обчислення базової функції густини ймовірності.

Моделі з латентними змінними оптимізують нижню оцінку правдоподібності через проєктування високорозмірних спостережень у стиснений прихований простір, охоплюючи лінійний імовірнісний аналіз головних компонентів (PPCA) та нелінійні варіаційні автоенкодери (VAE).

Дифузійні та score-based архітектури реалізують зворотний рух стохастичного процесу зашумлення у неперервних або дискретних просторах, формуючи сучасну базу для структурної генерації.

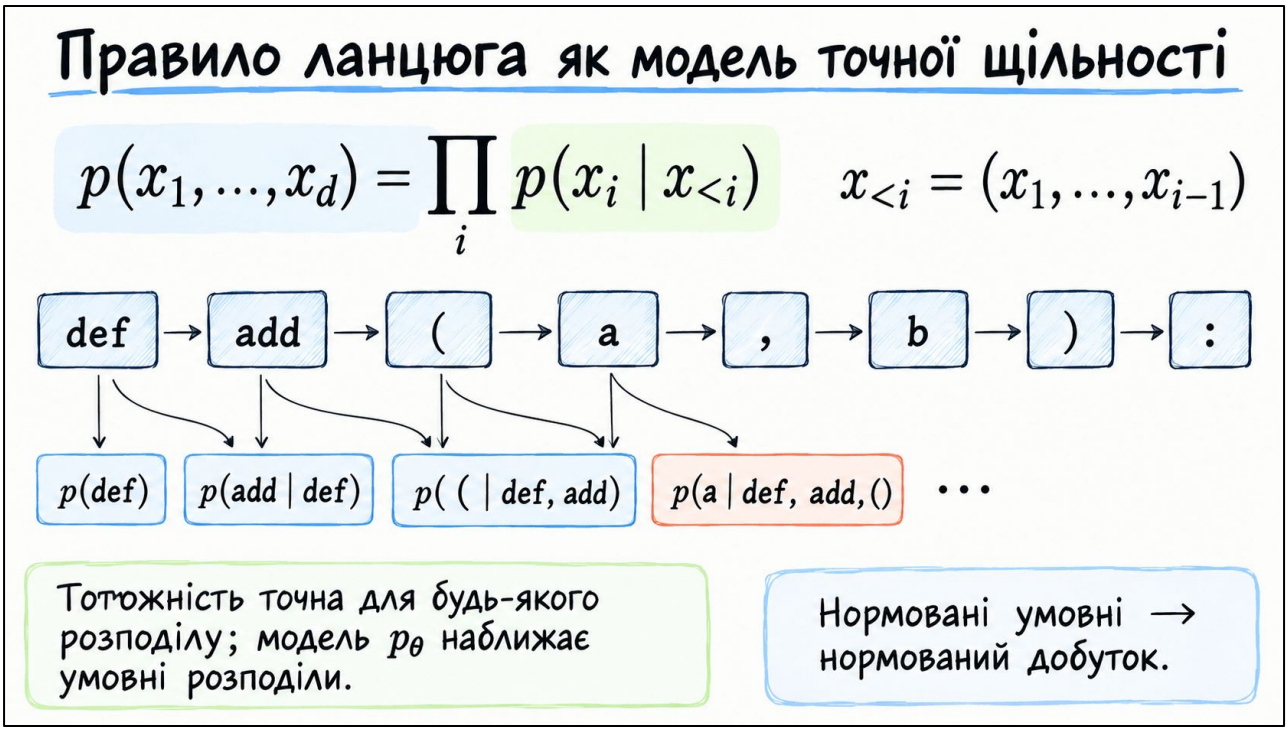
Окремий клас утворюють підходи поза прямою правдоподібністю, зокрема генеративно-змагальні мережі (GAN) та енергетичні моделі (EBM), де розподіл апроксимується через мінімаксну оптимізацію або ненормовані енергетичні рельєфи.

На противагу їм, моделі точної щільності спираються на аналітично розв'язувані розподіли.

Якщо нормалізувальні потоки застосовують інвертовані бієктивні відображення з обчислюваним якобіаном, то авторегресійні моделі розкладають сумісну ймовірність послідовності на добуток умовних імовірностей за правилом ланцюга. Саме ця авторегресійна факторизація становить математичний і концептуальний фундамент сучасних трансформерних великих мовних моделей

(LLM).

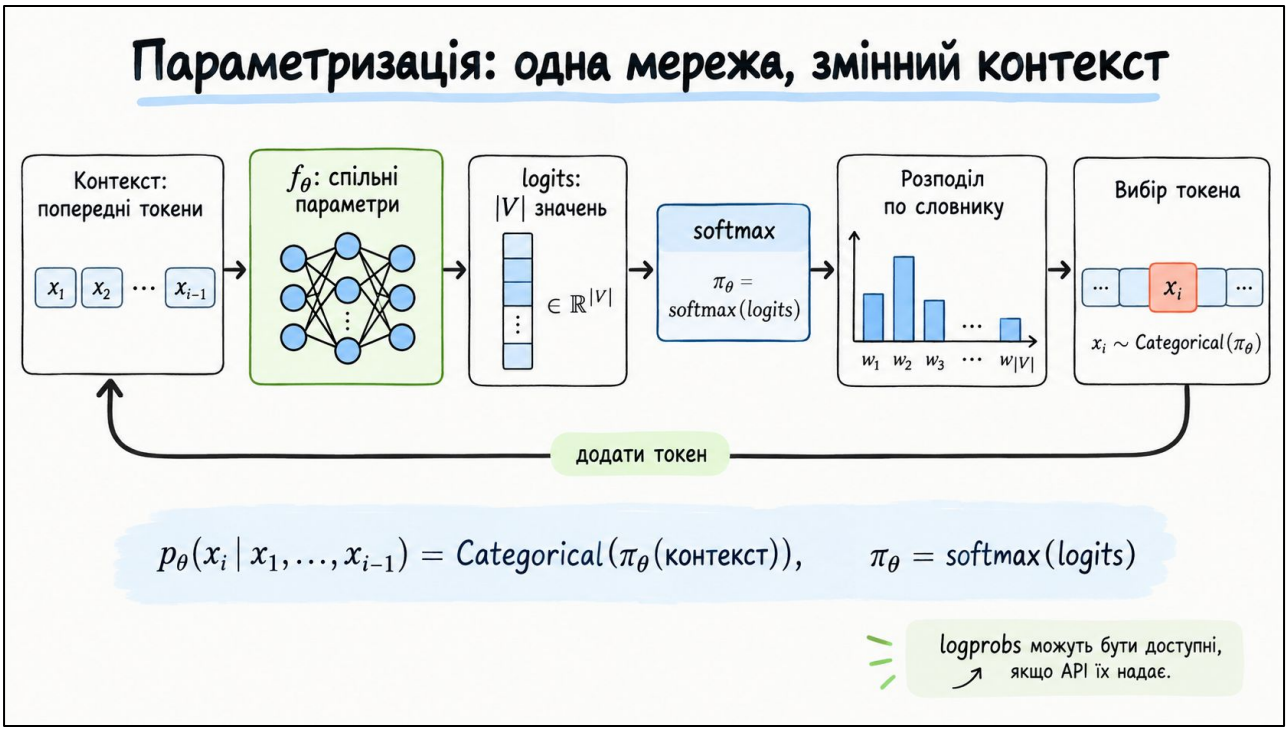
Завдяки послідовному обчисленню ймовірності кожного наступного токена на основі попереднього контексту, моделі забезпечують точний розрахунок правдоподібності без апроксимаційних припущень, що гарантує стабільне навчання та прецизійне захоплення довготривалих семантичних зв'язків у кодових і текстових структурах.



Математичний фундамент сучасного генеративного моделювання спирається на ймовірнісне правило ланцюга, яке формує суворий аналітичний базис для оцінювання точної щільності. Замість безпосереднього наближення високорозмірної та обчислювально складної сумісної ймовірності всієї послідовності, правило ланцюга розкладає сумісний розподіл p(x1, …, xd) на точний добуток одновимірних умовних імовірностей, де кожен наступний токен xi розраховується відносно всього префікса попередніх токенів x{<i}.

У контексті інженерії програмного забезпечення це означає представлення синтаксису коду як структурованого стохастичного процесу. Під час синтезу сигнатури функції початкова ймовірність ключового слова p(text{def}) задає базовий контекст; надалі ідентифікатор функції, відкривальна дужка та формальні параметри обчислюються послідовно, де ймовірність появи аргументу a безпосередньо обумовлена накопиченою історією токенів text{def}, text{add} та дужкою.

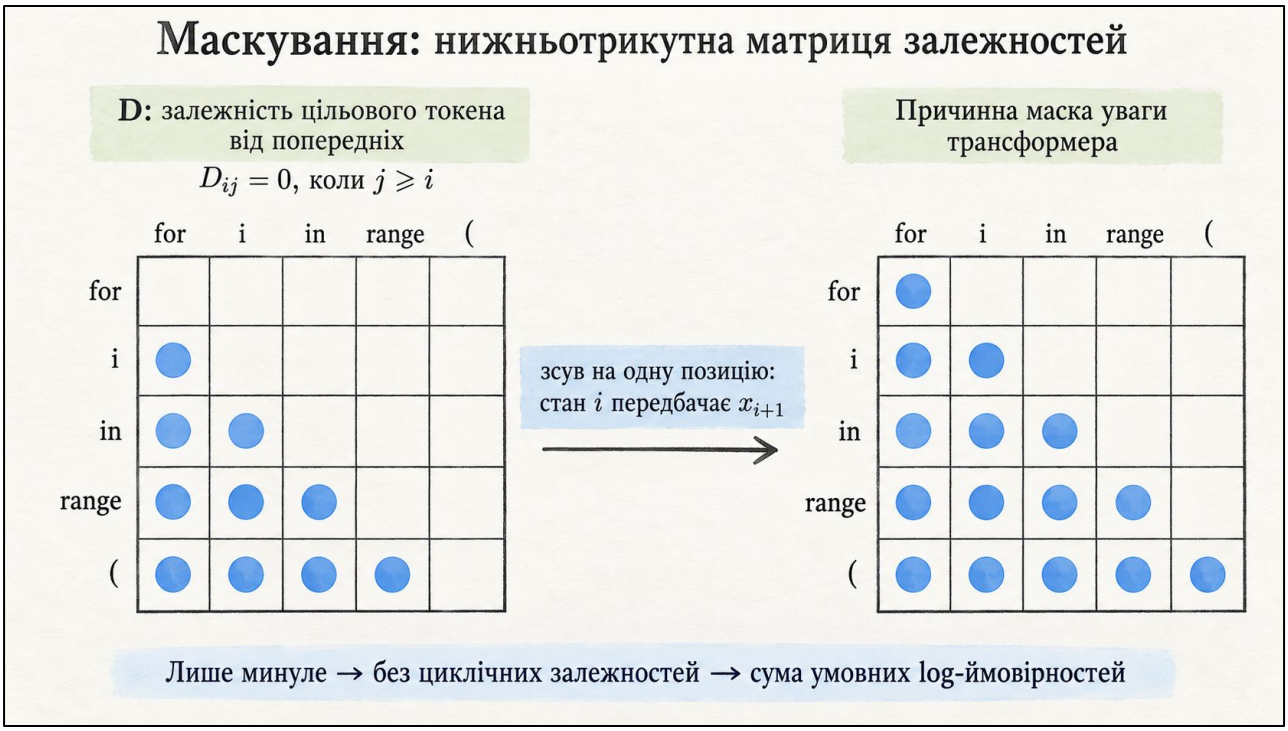
Ця тотожність є абсолютно точною для будь-якого розподілу й не потребує спрощувальних припущень щодо незалежності. Параметризована нейромережева модель апроксимує виключно ці локальні умовні розподіли. Оскільки кожен умовний розподіл нормалізований за простором словника, їхній авторегресійний добуток автоматично гарантує отримання строго нормованого сумісного розподілу ймовірностей для всієї кодової послідовності.



Обчислювальний механізм авторегресійної генерації базується на інваріантній архітектурі: єдиній параметризованій нейронній мережі, що оперує динамічно змінним контекстним вікном. На кроці і система отримує накопичену послідовність попередніх токенів від x1 до x{i-1} як вхідний контекст.

Цей вхід обробляється мережею , набір ваг якої залишається спільним і фіксованим для кожного кроку генерації. У результаті обчислень формується вектор ненормованих логітів у просторі дійсної розмірності, що точно відповідає повному розміру словника моделі.

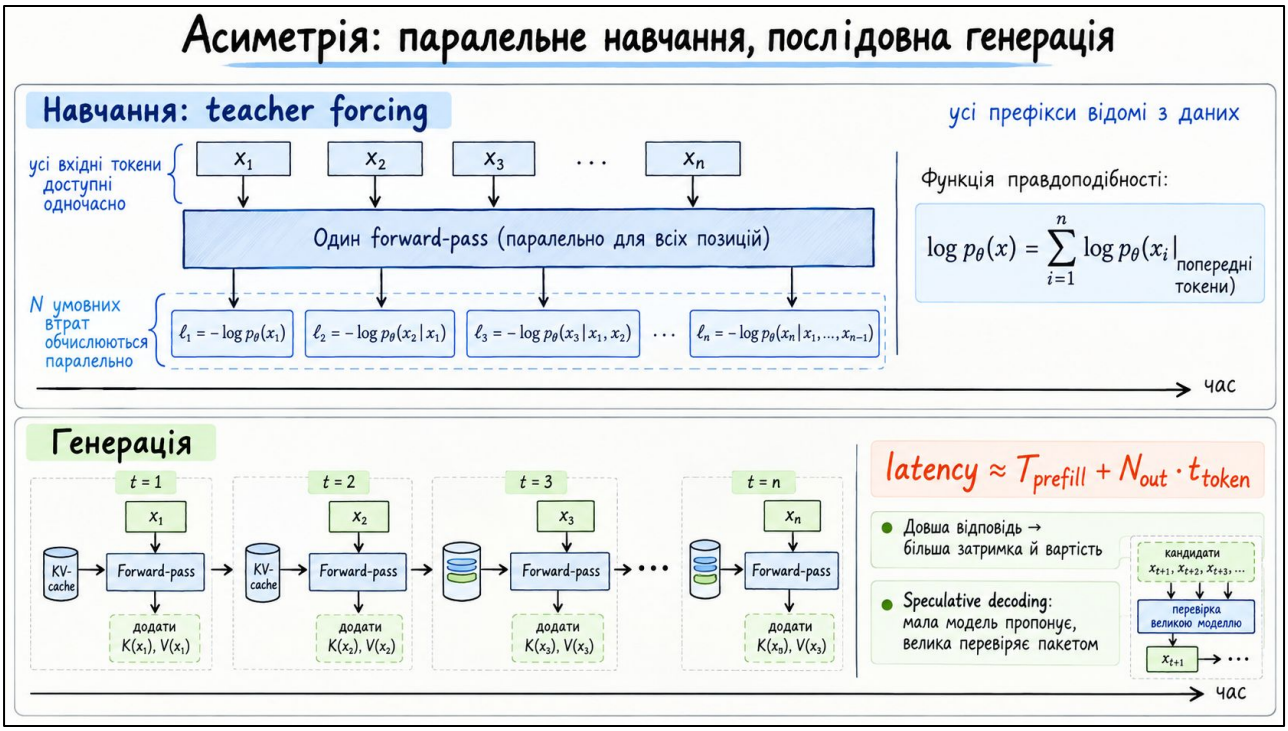
Для перетворення необроблених логітів у формальний імовірнісний розподіл застосовується функція softmax, яка трансформує їх у вектор ймовірностей над усім словником. Наступний токен xi обирається шляхом семплювання з категоріального розподілу, явно обумовленого цим вектором. Після вибору токен xi додається до контексту, замикаючи авторегресійний цикл для наступної ітерації i+1. У виробничих інфраструктурах інженери можуть аналізувати внутрішній рівень впевненості моделі за допомогою логарифмічних ймовірностей (logprobs), якщо їх надає відповідний API, що дозволяє виконувати точне калібрування надійності в критично важливих сервісах.



Математичне забезпечення принципу причинності в авторегресійних трансформерних архітектурах реалізується через механізм маскування уваги, формалізований у вигляді нижньотрикутної матриці залежностей.

У цій структурі цільовий токен на позиції і розраховується виключно на основі попередніх токенів.

Ми строго фіксуємо нульові значення елементів матриці D_{ij} = 0 для всіх випадків, де j та і, що унеможливлює витік інформації з майбутніх позицій і повністю ліквідує циклічні залежності в обчислювальному графі. При переході до практичної реалізації причинної маски уваги трансформера застосовується зсув на одну позицію: прихований стан і прогнозує наступний токен x_{i+1}. Завдяки такій часовій ізоляції — доступності виключно минулого контексту — оптимізація послідовності зводиться до строгої суми умовних логарифмічних ймовірностей без порушення авторегресійної природи моделі під час паралельного навчання.



Фундаментальне обчислювальне вузьке місце сучасних трансформерних архітектур полягає у вираженій асиметрії між процесами оптимізації та інференсу: навчання відбувається масивно паралельно, тоді як генерація є суто послідовною. Під час фази навчання за методом teacher forcing уся еталонна послідовність подається на вхід моделі одночасно. Оскільки всі попередні префікси заздалегідь відомі з навчального датасету, модель здійснює єдиний прямий прохід (forward-pass) паралельно для всіх позицій токенів. Застосування казуальної маски уваги зберігає авторегресійні обмеження, дозволяючи одночасно обчислювати значення умовних функцій втрат уздовж усієї послідовності, що максимізує утилізацію тензорних ядер та пропускну здатність обладнання.

На противагу цьому, авторегресійний інференс принципово позбавлений можливості глобального паралелізму. Оскільки генерація кожного наступного токена спирається на попередньо згенерований вихід, процес розгортається крок за кроком у часі. Для усунення надлишкових обчислень векторів ключів і значень минулих кроків застосовується механізм KV-кешування, який динамічно накопичує обчислені проєкції на кожному кроці вперед. Як наслідок, підсумкова затримка системи (latency) прямо детермінується часом первинного наповнення контексту та добутком кількості згенерованих токенів на час генерації одиничного токена. Що довша відповідь, то вищими стають затримка та обчислювальна вартість запиту. Для подолання цього бар'єра, обмеженого

пропускною здатністю пам'яті, застосовують метод спекулятивного декодування: компактна допоміжна модель швидко генерує ланцюжок токенів-кандидатів, після чого велика цільова модель верифікує їх пакетом за один паралельний прохід.

Фундаментальна розбіжність між оптимізацією під час навчання та практичним авторегресійним інференсом формує явище зміщення експозиції (exposure bias) — системний каскад помилок, за якого раннє синтаксичне чи семантичне відхилення лавиноподібно спотворює всю подальшу послідовність генерації.

На етапі тренування в режимі teacher forcing мережа умовлюється виключно на еталонні токени з навчальної вибірки, навчаючись передбачати наступні переходи за ідеального контексту. Проте під час генерації у виробничому середовищі модель спирається на власний щойно згенерований контекст.

Коли мовна модель синтезує функціональний код — наприклад, функцію завантаження даних за інженерним запитом — помилка на ранньому етапі, така як вибір вигаданого імпорту замість коректного пакета, незворотно трансформує простір умовних імовірностей. Оскільки всі наступні токени умовлюються на цей дефектний префікс, модель продовжує локально логічну, але глобально хибну гілку генерації, синтезуючи вигадані виклики методів, які виглядають правдоподібно, проте не існують у реальному середовищі виконання.

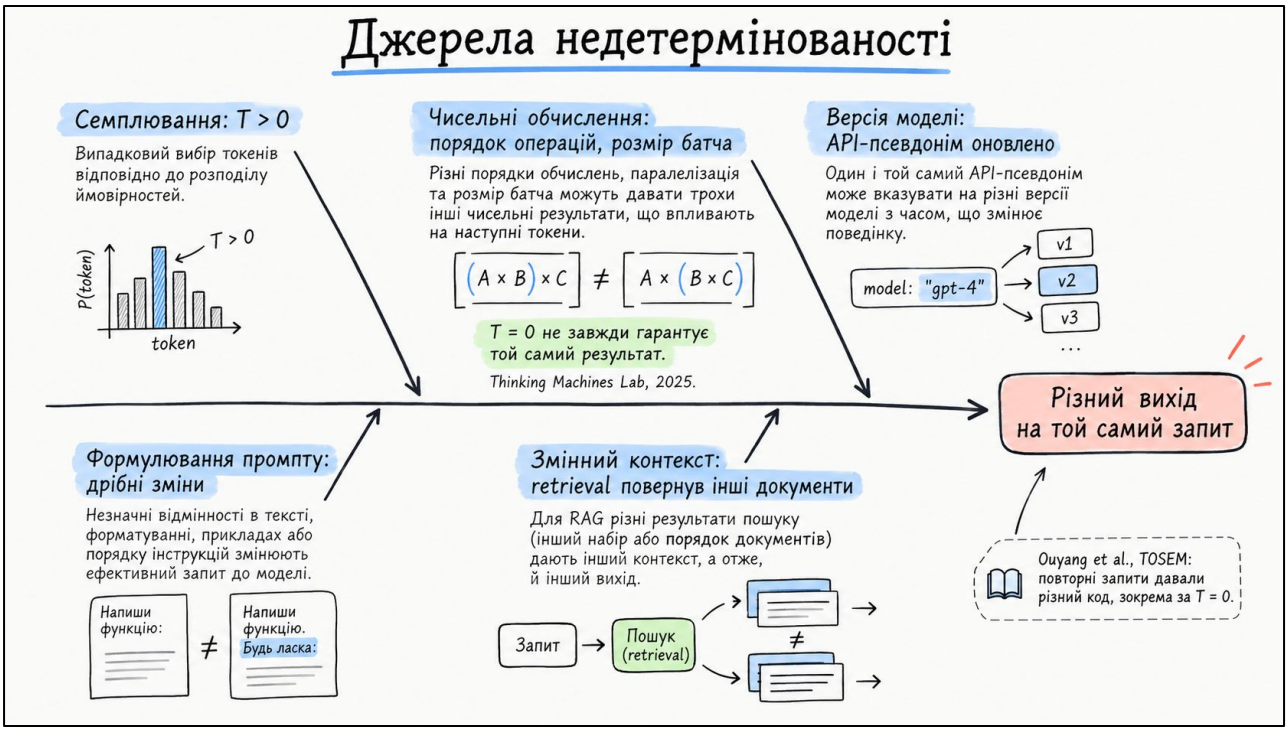
Для надійного усунення таких каскадів сучасні агентні архітектури інтегрують замкнені контури виконання у пісочницях. Запуск синтезованого коду фіксує зовнішні повідомлення середовища — зокрема трасування винятків на кшталт AttributeError, перетворюючи приховані семантичні галюцинації на

детермінований зворотний зв'язок. Автономний агент аналізує діагностичні логи помилок, коригує некоректні виклики бібліотек і здійснює нову спробу генерації. Інтеграція емпіричного зворотного зв'язку середовища виконання робить виявлення дефектів надійним і дозволяє системно долати ефект зміщення експозиції, повертаючи генерацію в русло перевіреної інженерної коректності.

Перехід від ненормованих логітів до вибору дискретних токенів в авторегресійній генерації регулюється стратегіями декодування: температурою, усіканням top-k та ядерним семплюванням top-p. Формально температурне масштабування вводить гіперпараметр відпалу T у знаменник функції softmax, розраховуючи нормовану ймовірність токена за формулою що на слайді . При наближенні T до 0 імовірнісний розподіл загострюється до дельта-функції Дірака, зводячи процес до майже жадібного детермінованого вибору з концентрацією 99,3% маси на домінантній альтернативі A при T=0,2. Значення T=1,0 відповідає базовому категоріальному розподілу логітів, тоді як підвищення температури до 1,5 згладжує розподіл по хвостах, додаючи варіативність ціною зростання ентропії та ризику галюцинацій.

Для збереження семантичної зв'язності та усунення шуму застосовуються механізми обмеження вибіркового простору. Якщо фільтрація top-k фіксує жорстку кількість k найімовірніших токенів незалежно від форми розподілу, то ядерне семплювання top-p (nucleus sampling), запропоноване Хольцманом та співавторами у 2020 році, забезпечує динамічну адаптацію. Алгоритм сортує токени за спаданням і накопичує кумулятивну ймовірність до досягнення порогу p, наприклад 0,95. Це дозволяє відсікати довгий хвіст малоймовірних токенів і динамічно звужувати або розширювати пул кандидатів відповідно до ступеня

невизначеності моделі в поточній точці кодового контексту.



Авторегресійні мовні моделі часто помилково сприймають як детерміновані функції, проте реальна інженерна практика виявляє системний недетермінізм, коли ідентичний запит породжує відмінні програмні артефакти.

На базовому рівні декодування стохастичне семплювання за температури більшої за нуль свідомо вносить ентропію, здійснюючи вибір токенів згідно з імовірнісним розподілом словника. Проте примусова фіксація нульової температури для жадібного вибору не забезпечує абсолютної відтворюваності. Неасоціативність обчислень із рухомою комою під час паралельних матричних множень, динамічний розмір батча та відмінності в чергах виконання інструкцій тензорними ядрами викликають мікроскопічні числові розбіжності, які лавиноподібно зміщують вибір наступних токенів, що було формалізовано дослідниками Thinking Machines Lab у 2025 році.

Ситуацію ускладнює інфраструктурна динаміка на рівні платформи: глобальні API-псевдоніми на кшталт "gpt-4" з часом непомітно перенаправляються на оновлені версії ваг, викликаючи дрейф поведінки. Безпосередньо в конвеєрі запитів мікроскопічні зміни у формулюванні промпту — додавання формальних зворотів чи корекція форматування — кардинально перерозподіляють активації в матрицях уваги.

Нарешті, в архітектурах RAG недетермінованість пошукового рушія та зміна ранжування повертають інший масив документів, підміняючи вихідний контекст інференсу. Як експериментально продемонстрували Оуянг та

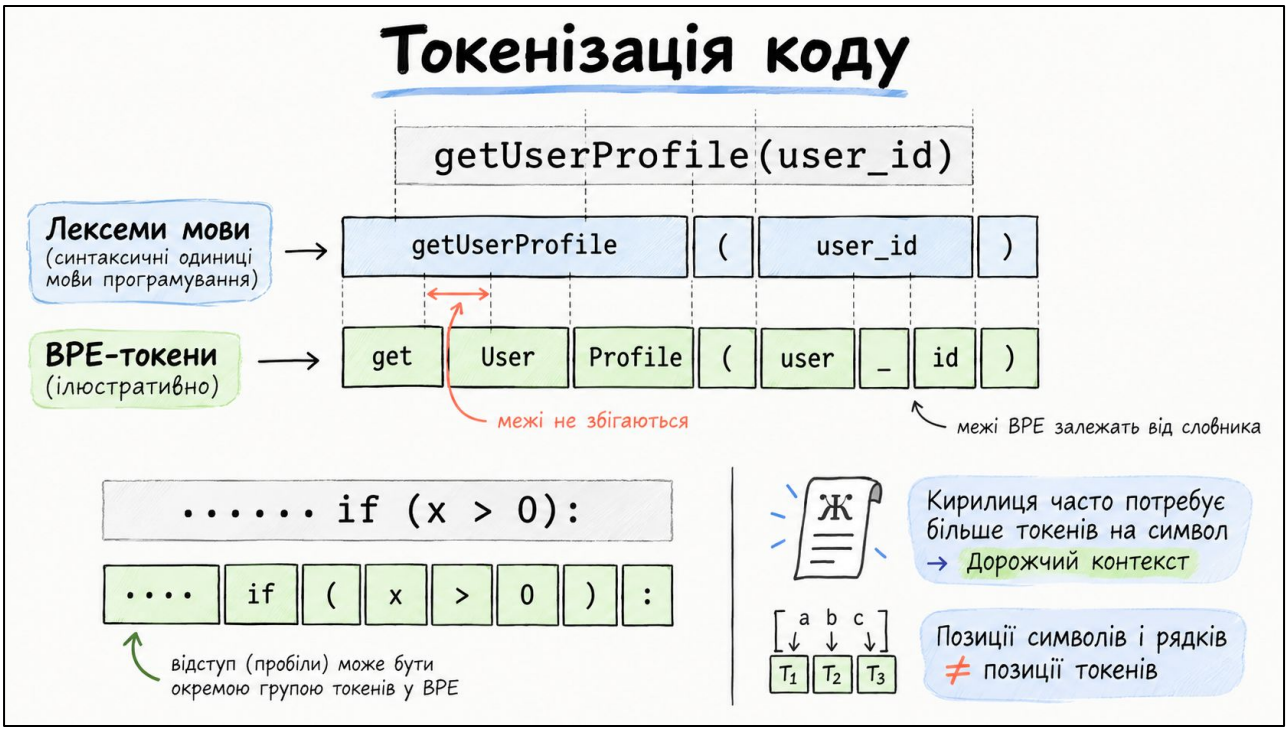
співавтори в журналі TOSEM, повторні звернення синтезують різний код навіть за нульової температури, що робить недетермінізм першочерговим викликом, який вимагає впровадження суворих тестових конвеєрів evals та системних захисних бар'єрів.

Фундаментальна розбіжність між генеративним імовірнісним моделюванням та інженерною дисципліною програмного забезпечення полягає в принциповій нееквівалентності статистичної правдоподібності та семантичної коректності. Авторегресійні мовні моделі навчаються максимізувати суму умовних логарифмічних ймовірностей токенів у послідовності^ формула на слайді. Ця цільова функція оптимізує природність продовження тексту, формуючи статистично очікувані послідовності, проте вона математично не гарантує об'єктивної істинності чи логічної валідності програмних рішень.

Простір генерації розгортається у двовимірній системі координат між імовірністю тексту за моделлю та його семантичною коректністю, утворюючи чотири критичні області. Типові правильні відповіді та шаблонні функції зосереджені у квадранті високої ймовірності та високої коректності. Нестандартні, але істинні інженерні рішення займають зону низької ймовірності за збереження високої коректності, репрезентуючи рідкісні, проте правильні висновки. Низькоймовірні хибні результати зазвичай легко діагностуються та відсіваються.

Найбільшу небезпеку для інженерних систем становить нижній правий квадрант — непомітні помилки, коли вихід моделі є надзвичайно правдоподібним, але водночас хибним. Цей простір породжує галюцинації, коли згенерований код має ідеальний вигляд, проте містить неіснуючі виклики або порушує критичні інваріанти, а також явище сикофантії — підлаштування

системи під помилкові очікування користувача. Оскільки внутрішній розподіл імовірностей моделі не здатен верифікувати істину, вирішальним принципом стає зовнішня перевірка: лише динамічне тестування, запуск у пісочницях та статичний аналіз здатні встановити дійсну коректність системи.



Фундаментальний рівень представлення вихідного коду в генеративних моделях визначається токенізацією, яка виявляє глибоку структурну неузгодженість між формальною теорією компіляторів та статистичними підсловесними алгоритмами.

У класичному синтаксичному аналізі лексичний аналізатор розбиває програму на точні лексеми — неподільні синтаксичні одиниці мови програмування, такі як цілісний ідентифікатор getUserProfile, дужки або параметри, визначені суворою граматикою.

На противагу цьому, алгоритм Byte Pair Encoding (BPE) функціонує на підслівному рівні, фрагментуючи ідентифікатор на морфеми зі свого словника — get, User, Profile, або розділяючи змінну user_id на окремі фрагменти user, _ та id. Як наслідок, межі BPE-токенів не збігаються з межами лексем, змушуючи нейромережу витрачати параметричну ємність на повторне збирання цілісних програмних сутностей.

Ця специфіка створює додаткові виклики для структури коду та мультиязичності. Символи відступів і пробілів, які формують синтаксичні блоки у таких мовах, як Python, групуються в довільні послідовності токенів, що ускладнює контроль вкладеності. Водночас виникає суттєва економічна диспропорція: кириличні символи в коментарях або документації зазвичай вимагають більшої кількості токенів на один символ, що призводить до перевитрати контекстного вікна та здорожчання вартості інференсу. Нарешті,

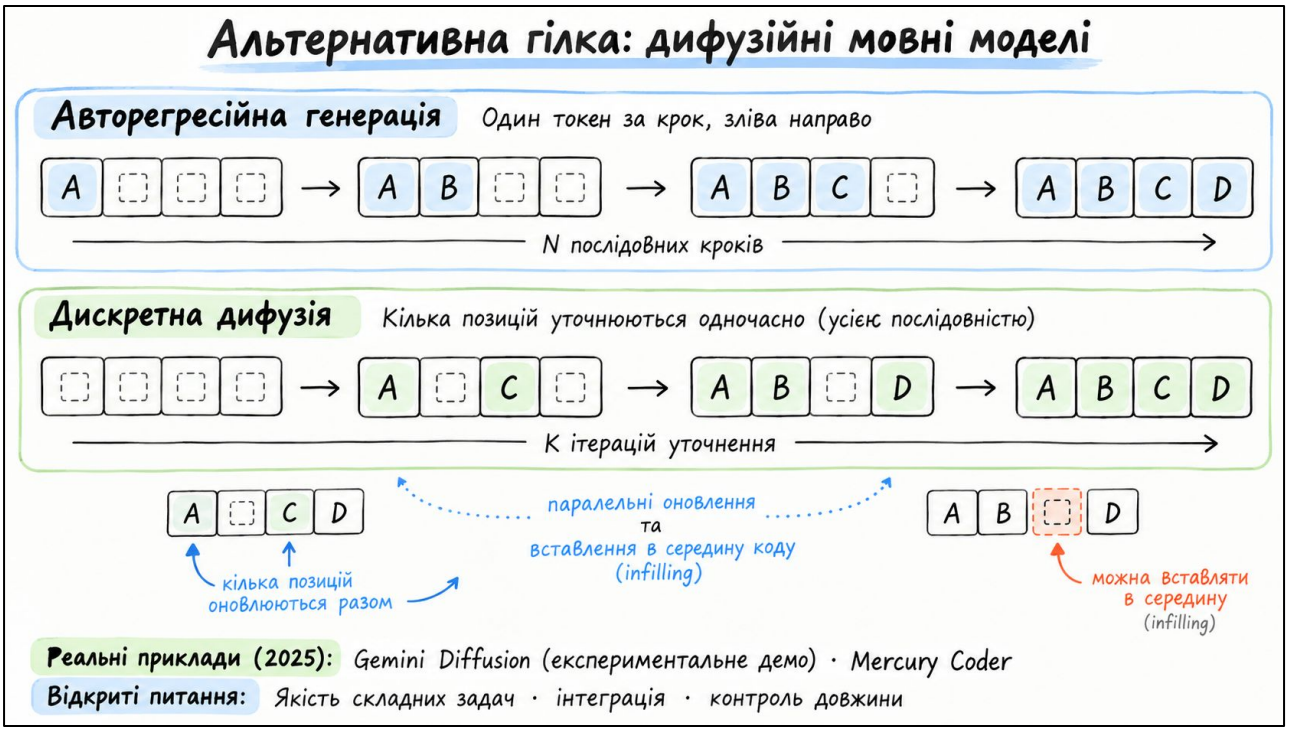
абсолютні позиції символів і рядків у вихідному файлі не є еквівалентними індексам токенів у послідовності, що вимагає побудови додаткових відображень для задач точної локалізації дефектів та рефакторингу.

Ефективність використання великих мовних моделей визначається не лише номінальним розміром їхнього контекстного вікна, але й тим, наскільки рівномірно механізми уваги здатні видобувати інформацію з вхідної послідовності. Контекстне вікно визначає граничну межу токенів входу та виходу, яку архітектура здатна одночасно обробити в межах одного циклу інференсу. Проте точність аналізу інформації вздовж цього вікна є вкрай неоднорідною.

Фундаментальне дослідження Лю та співавторів у журналі TACL 2024 року зафіксувало U-подібну криву уваги, відому як ефект «Lost in the Middle» — втрата інформації всередині контексту. Релевантні дані, розміщені на початку вхідного промпту або в його кінці, демонструють найвищу ймовірність точного вилучення завдяки ефектам первинності та свіжості контексту. Натомість суттєві факти, вимоги чи кодові фрагменти, що опиняються в середині послідовності, зазнають різкої деградації уваги та часто повністю ігноруються моделлю.

Подолання цього обмеження вимагає переходу до системної дисципліни інженерії контексту. Головний інженерний висновок полягає в тому, що формальне вміщення даних у ліміт токенів не гарантує їхнього фактичного врахування під час генерації відповіді. Архітектура повинна забезпечувати ретельний попередній відбір найбільш релевантних фрагментів, контекстне стиснення з видаленням надлишкового синтаксичного шуму та стратегічне впорядкування знань — винесення критичних інваріантів на початок або в кінець вхідного буфера. Доповнення даних явними позначеннями джерел і

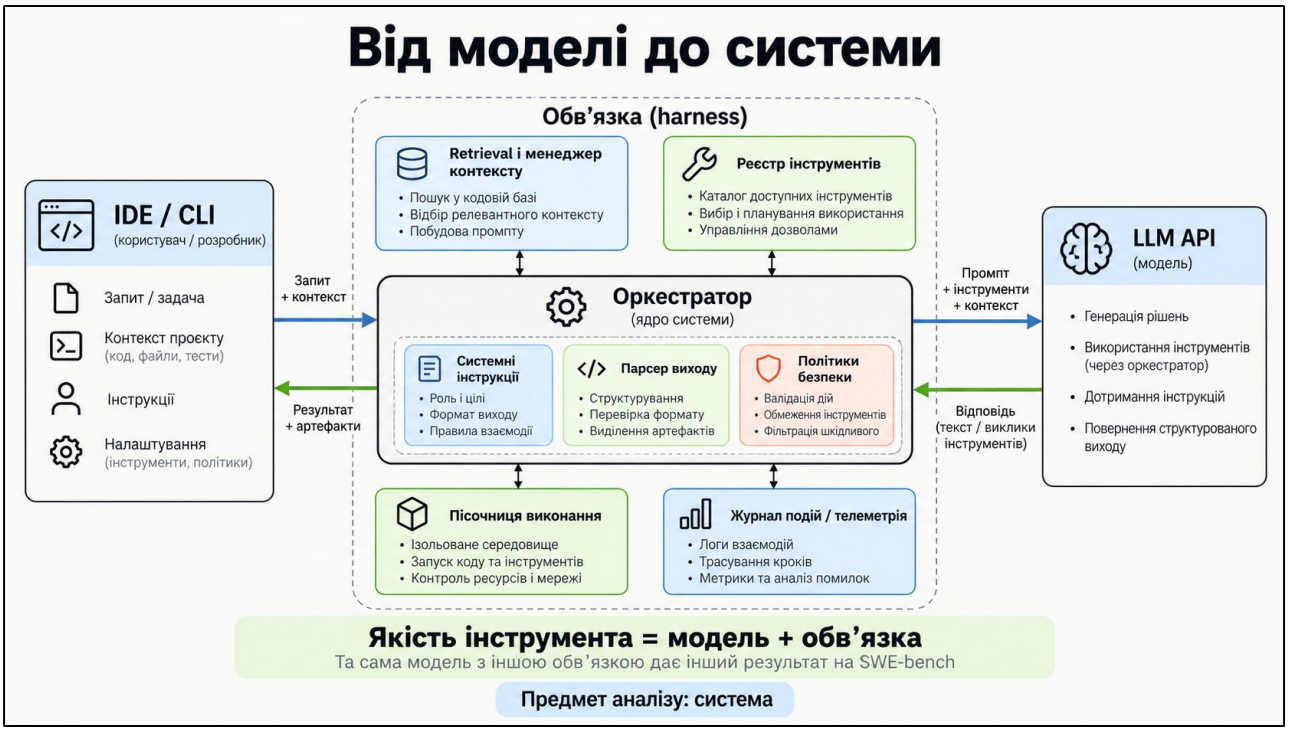
структурованими посиланнями дозволяє стабілізувати роботу конвеєрів генерації та запобігти мовній амнезії в складних інженерних застосунках.



У той час як домінантна парадигма синтезу програмного коду спирається на строго причинне авторегресійне декодування — генерацію по одному токену зліва направо впродовж N послідовних кроків — альтернативну еволюційну гілку формують дискретні дифузійні мовні моделі. Авторегресійний підхід накладає суттєві обмеження на паралелізацію інференсу та ускладнює врахування двостороннього контексту під час генерації. На противагу цьому, дискретна дифузія оперує одразу всією послідовністю впродовж K ітерацій уточнення, виконуючи паралельне знешумлення та уточнення кількох маскованих позицій водночас.

Такий механізм відкриває принципову інженерну перевагу для задач програмування: природне вставлення в середину коду, або code infilling. Модель не обмежена послідовним дописуванням рядків у кінець файлу, а може синхронно оновлювати кілька блоків і виконувати точкову модифікацію логіки всередині вже наявної структури програми.

Практична реалізація цього напряму знаходить відображення в експериментальних прототипах на кшталт Gemini Diffusion та Mercury Coder. Водночас перед інженерною спільнотою залишаються відкриті дослідницькі виклики: забезпечення якості розв'язання складних алгоритмічних задач, безшовна інтеграція таких моделей у продакшн-конвеєри та точний динамічний контроль довжини згенерованої послідовності без артефактів фіксованого буфера.



Перехід від базової статистичної мовної моделі до промислового автономного агента позначає фундаментальний зсув інженерної парадигми: справжнім предметом аналізу стає вся програмна система в цілому, а не лише ізольовані ваги нейромережі.

У центрі цієї архітектури функціонує оркестратор обв'язки (harness), який виступає ядром системи та пов'язує робоче середовище розробника — IDE чи консольний інтерфейс — із віддаленим LLM API. Коли інженер передає запит, контекст проєкту та конфігураційні інструкції, обв'язка активує модуль пошуку та керування контекстом для навігації кодовою базою, відбору релевантних фрагментів та динамічної збірки промпту. Паралельно реєстр інструментів структурує каталог доступних утиліт, оптимізує чергу їхнього виклику та розмежовує права доступу.

Безпосередньо всередині оркестратора системні інструкції декларують роль моделі, очікувані формати виходу та інваріанти взаємодії, тоді як парсер виходу гарантує перевірку структури, валідацію схем і виділення синтезованих артефактів. Політики безпеки виконують роль захисних бар'єрів, верифікуючи заплановані дії, обмежуючи доступ до небезпечних інструментів і фільтруючи деструктивні команди. Виконання коду й викликів утиліт ізолюється в пісочниці з суворим контролем ресурсів і мережевого трафіку, а журнал подій фіксує покрокове трасування, логи та метрики помилок. У підсумку інтегральна якість інструмента визначається як синергія самої моделі та її обв'язки: одна й та сама

мовна модель у різних програмних обв'язках демонструє кардинально відмінні показники ефективності на промислових бенчмарках на кшталт SWE-bench.

Інтеграція генеративного штучного інтелекту в інженерію програмного забезпечення не скасовує і не замінює класичний життєвий цикл розробки (SDLC); натомість ШІ функціонує як підсилювальний накладений шар над усталеними інженерними процесами.

Від вихідного аналізу вимог до тривалого супроводу інтелектуальні можливості оптимізують кожен послідовний етап створення системи. Під час роботи з вимогами моделі забезпечують їхнє аналітичне уточнення та декомпозицію.

На етапі проєктування вони прискорюють розробку й зіставлення альтернативних архітектурних рішень. У процесі реалізації ШІ бере на себе синтез первинних чернеток коду, що на стадії тестування логічно доповнюється автоматизованою генерацією тестових наборів.

Для релізу та експлуатації автоматизація спрощує підготовку конфігураційних файлів і конвеєрів та забезпечує глибинний аналіз інцидентів під час моніторингу телеметрії. Фундаментальним інженерним інваріантом тут виступає незмінність контролю якості: шлюзи якості залишаються обов'язковими бар'єрами, а критерії виходу з кожного етапу є суворо однаковими незалежно від того, був артефакт створений інженером чи згенерований штучним інтелектом.

Трансформація неструктурованих бізнес-вимог у формальні інженерні специфікації є фундаментальним етапом аналітичної фази SDLC, де мовні моделі функціонують як інтелектуальний шар декомпозиції та верифікації.

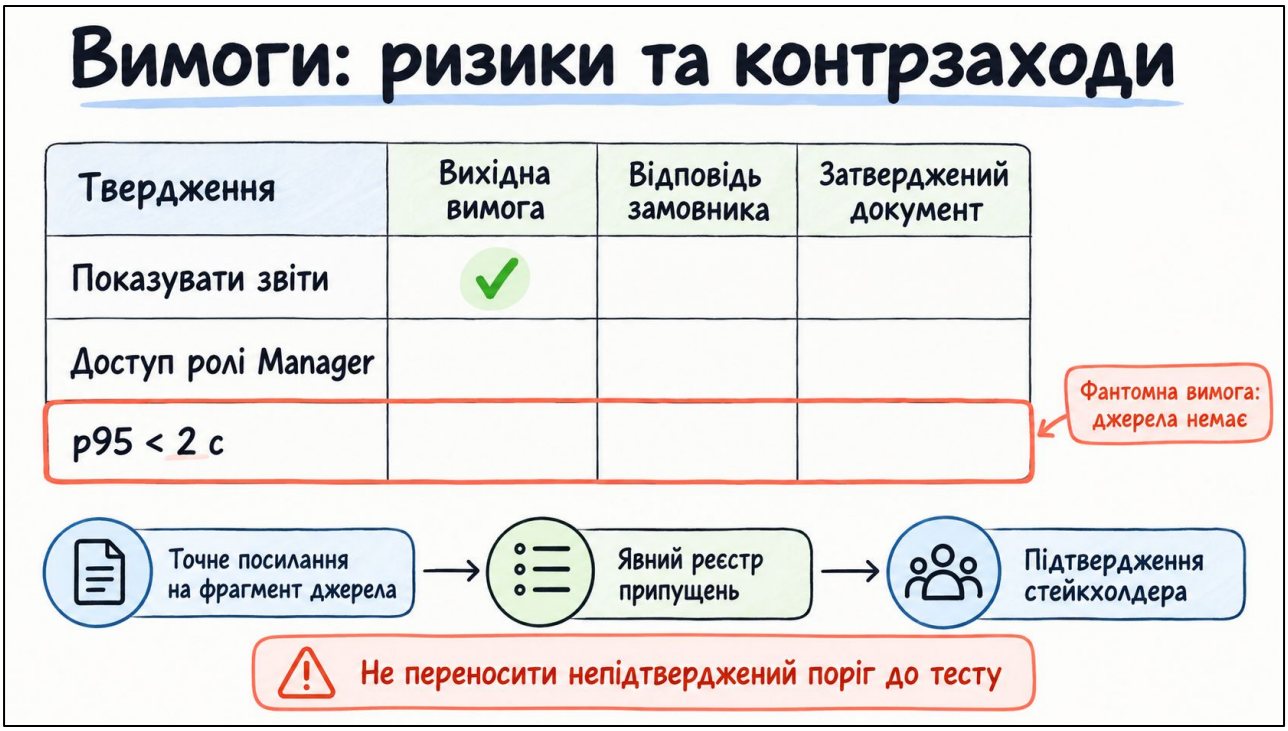
Типова вихідна вимога стейкхолдера — «система має швидко показувати звіти користувачам» — страждає на критичний рівень семантичної неоднозначності, неприпустимий для надійної програмної інженерії. Автоматизований аналіз виявляє ключові сліпі зони: відсутність кількісного порогу для поняття швидкодії та розмитість суб'єкта взаємодії через невизначеність ролей користувачів.

Системне усунення цих прогалин відбувається шляхом синтезу цільових запитань до замовника. Це переводить абстрактні побажання у площину формальних інженерних контрактів: визначення допустимого 95-го перцентиля часу відгуку (p95 latency) та специфікацію матриці доступу на основі ролей (RBAC).

Після валідації відповідей замовником уточнена вимога транслюється у формалізовані критерії приймання за методологією BDD: фіксується конкретна роль менеджера, дія відкриття місячного звіту та нефункціональний інваріант відображення даних за час p95 < 2с}. Будь-які непідтверджені параметри зберігають статус явних припущень і підлягають обов'язковій верифікації замовником.

Фінальним кроком стає пряма компіляція критеріїв приймання в

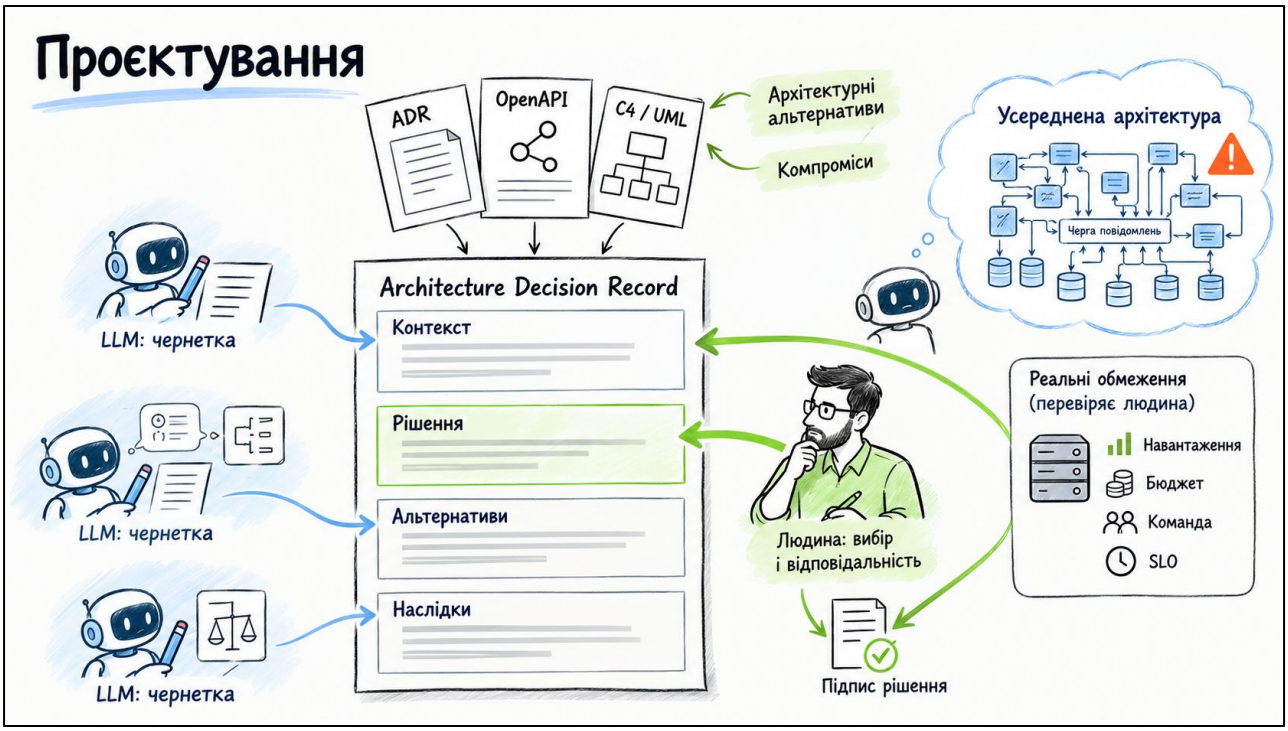
автоматизовані тести, що забезпечує безперервну наскрізну простежуваність від первинного формулювання вимоги до детермінованого контролю якості в кодовій базі.



Процес інженерії вимог за участі великих мовних моделей несе в собі критичний архітектурний ризик — виникнення так званих фантомних вимог, тобто неперевірених припущень, які штучний інтелект самовільно синтезує під час деталізації специфікацій. Аналізуючи матрицю трасування тверджень, ми бачимо чітке розмежування між вихідними вимогами замовника, його прямими відповідями та затвердженою проєктною документацією. У той час як первинна функціональна потреба — відображення звітів — має документальне підтвердження, додатково згенеровані обмеження, такі як доступ виключно для ролі Manager або жорсткий нефункціональний поріг швидкодії p95 < 2c, не мають під собою жодного першоджерела. Це класична галюцинація контексту: модель формує зовні раціональну, але абсолютно неавторизовану вимогу.

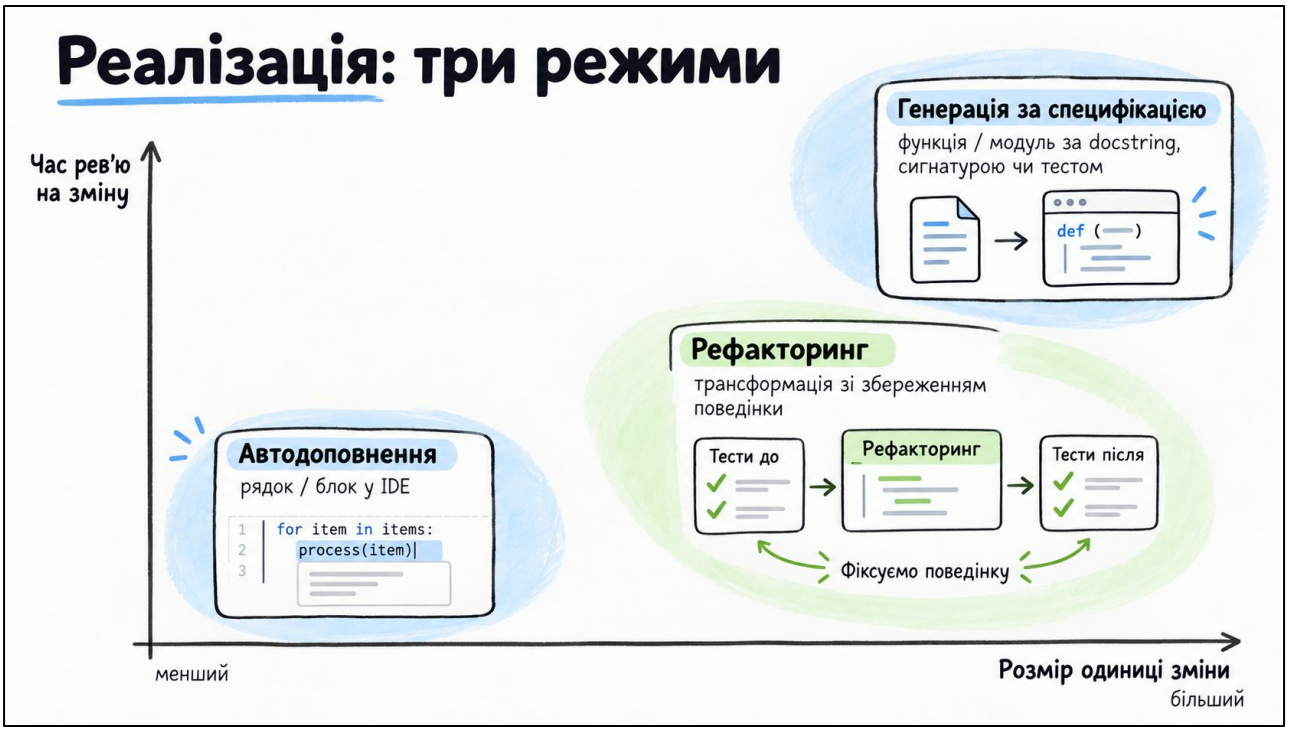
Якщо такий непогоджений поріг потрапляє в автоматизовані приймальні тести, інженерна команда отримує хибний критерій якості, що провокує невиправдані витрати ресурсів на передчасну оптимізацію або дестабілізує реліз фіктивними дефектами. Надійним контрзаходом проти такої деградації є впровадження суворого протоколу валідації. Будь-яке інженерне твердження повинно мати точне посилання на верифікований фрагмент джерела. Усі інтерпретації та добудовані моделлю деталі обов'язково ізолюються в явному реєстрі припущень і виносяться на пряме підтвердження стейкхолдером. Головний інваріант контролю полягає в непорушному правилі: жоден непідтверджений поріг чи атрибут якості не повинен переноситися до коду тестів

доти, доки замовник явно не затвердить його як контракт системи.



На етапі архітектурного проєктування штучний інтелект виступає потужним генератором первинних інженерних чернеток, проте відповідальність за прийняття рішень залишається прерогативою людини. Сучасна розробка формалізує системні вимоги через стандартизовані інженерні контракти: специфікації OpenAPI, діаграми C4 або UML та архітектурні записи рішень — Architecture Decision Records (ADR). Моделі здатні якісно драфтувати структурні секції ADR: описувати вихідний технічний контекст, синтезувати перелік можливих альтернатив і структурувати потенційні компроміси та наслідки кожного вибору.

Проте використання генеративних моделей без жорсткого критичного фільтра несе фундаментальний архітектурний ризик — пастку «усередненої архітектури». Оскільки статистична оптимізація відображає узагальнені патерни з навчального корпусу, мовна модель схильна пропонувати надмірно роздуті, шаблонні мікросервісні топології з розподіленими чергами повідомлень там, де доцільним є простий модульний моноліт. Запобіганням цій деградації є роль архітектора, який бере на себе остаточний вибір та інженерну відповідальність. Людина верифікує запропоновані рішення крізь призму реальних контекстних обмежень: прогнозованого навантаження, лімітів бюджету інфраструктури, компетенцій команди розробки та цільових індикаторів SLO. Лише після зваженої валідації реальних обмежень фінальне рішення затверджується підписом інженера, гарантуючи надійність і керованість архітектурного базису системи.



Фаза реалізації в парадигмі програмної інженерії з інтеграцією штучного інтелекту структурується за трьома фундаментальними режимами, які розподіляються вздовж осей розміру одиниці зміни та необхідного часу на її інженерне рев'ю.

Перший, найбільш локалізований режим — це автодоповнення безпосередньо в інтегрованому середовищі розробки. Тут об'єктом маніпуляції є окремий рядок або компактний блок коду, такий як типова ітераційна конструкція циклу for. Завдяки мінімальному розміру внесених змін когнітивне навантаження на розробника залишається гранично низьким, а верифікація здійснюється практично миттєво в робочому контексті редактора.

Другий режим — рефакторинг — переходить до якісно вищого рівня абстракції, фокусуючись на структурній трансформації кодової бази із суворим збереженням вихідної поведінки. Вирішальним інженерним інваріантом тут виступає поведінкова фіксація через тестові набори. Наявність валідованих модульних тестів до початку модифікацій і їхнє успішне проходження після перетворення гарантує відсутність регресій, що суттєво обмежує час експертного рев'ю навіть за умови глибокої реорганізації коду.

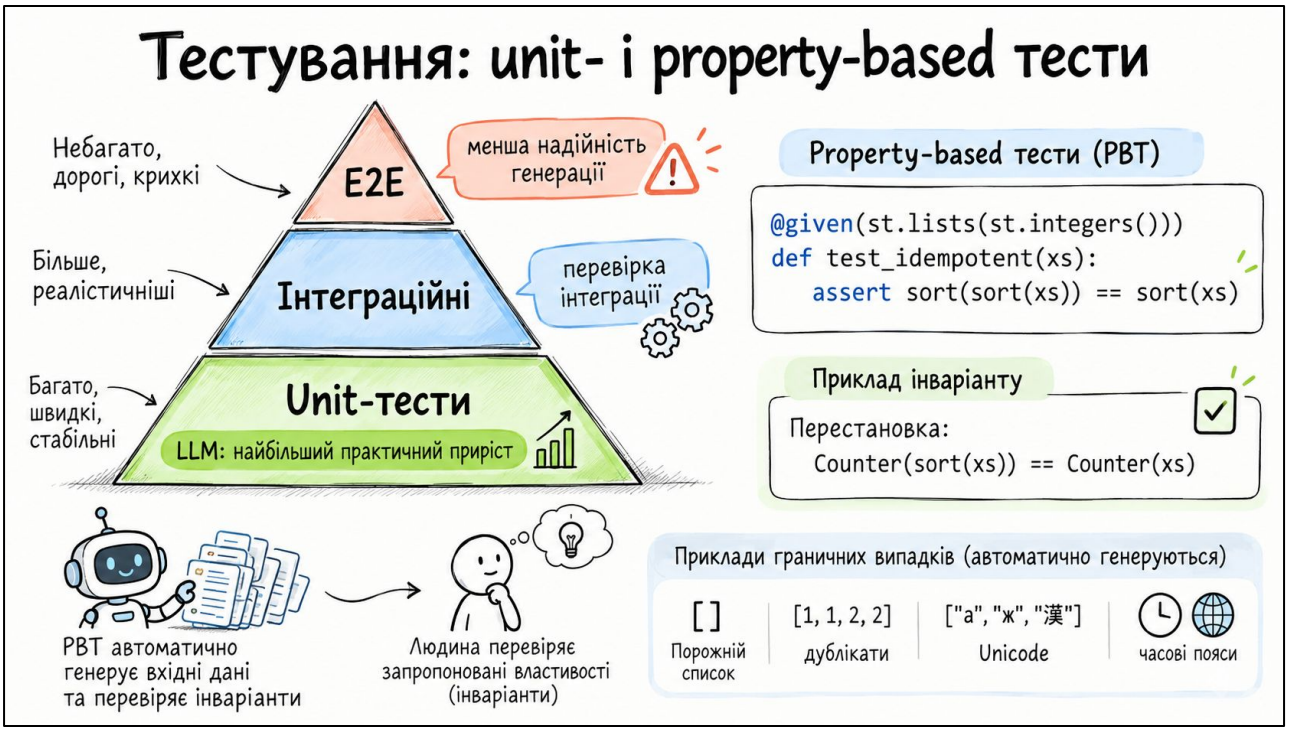
Третій і найбільш масштабний режим — це генерація за специфікацією, коли штучний інтелект синтезує повноцінні функції або системні модулі на основі рядків документації (docstring), формальних сигнатур чи специфікованих acceptance-тестів. Оскільки розмір одиниці зміни тут досягає максимуму, цей

режим вимагає найвищих часових витрат на аудит і верифікацію. Згенерований об'єм коду несе приховані ризики недетермінованих крайових випадків та синтаксично коректних, але семантично дефектних конструкцій, тому ретельне рецензування та запуск у тестовому середовищі стають обов'язковими бар'єрами контролю якості перед інтеграцією в кодову базу.

В епоху інженерії програмного забезпечення з інтеграцією штучного інтелекту орієнтація на поверхневі метрики реалізації створює критичні архітектурні та організаційні пастки. Команди часто розглядають частку прийнятих AI-пропозицій та валовий обсяг нового коду як прямі показники продуктивності розробки, проте висока активність у репозиторії не тотожна інженерній якості. Неконтрольоване зростання кодової маси породжує так званий код-чарн (churn) — вимушене переписування та видалення щойно доданого коду вже за два тижні, — а також провокує масштабне дублювання логіки та критичне навантаження на інженерів під час рев'ю.

Емпіричним підтвердженням цієї деградації є дослідження аналітичної компанії GitClear 2025 року на масиві даних 2024 року: частка скопійованих рядків уперше перевищила обсяг переміщених та структурованих блоків, засвідчивши деградацію культури рефакторингу на користь сліпого копіювання.

Саме тому фокус метрик має зміщуватися з проміжних показників активності на вимірювання системних наслідків: дефектів після злиття (post-merge defects) та часу проходження пул-реквесту до моменту злиття в основну гілку. При цьому принципово пам'ятати про методологічне обмеження: ці спостереження демонструють кореляції, а не прямий причинно-наслідковий зв'язок — висока частка прийнятих пропозицій може супроводжуватися як підвищенням ефективності, так і накопиченням прихованого технічного боргу залежно від суворості шлюзів якості.



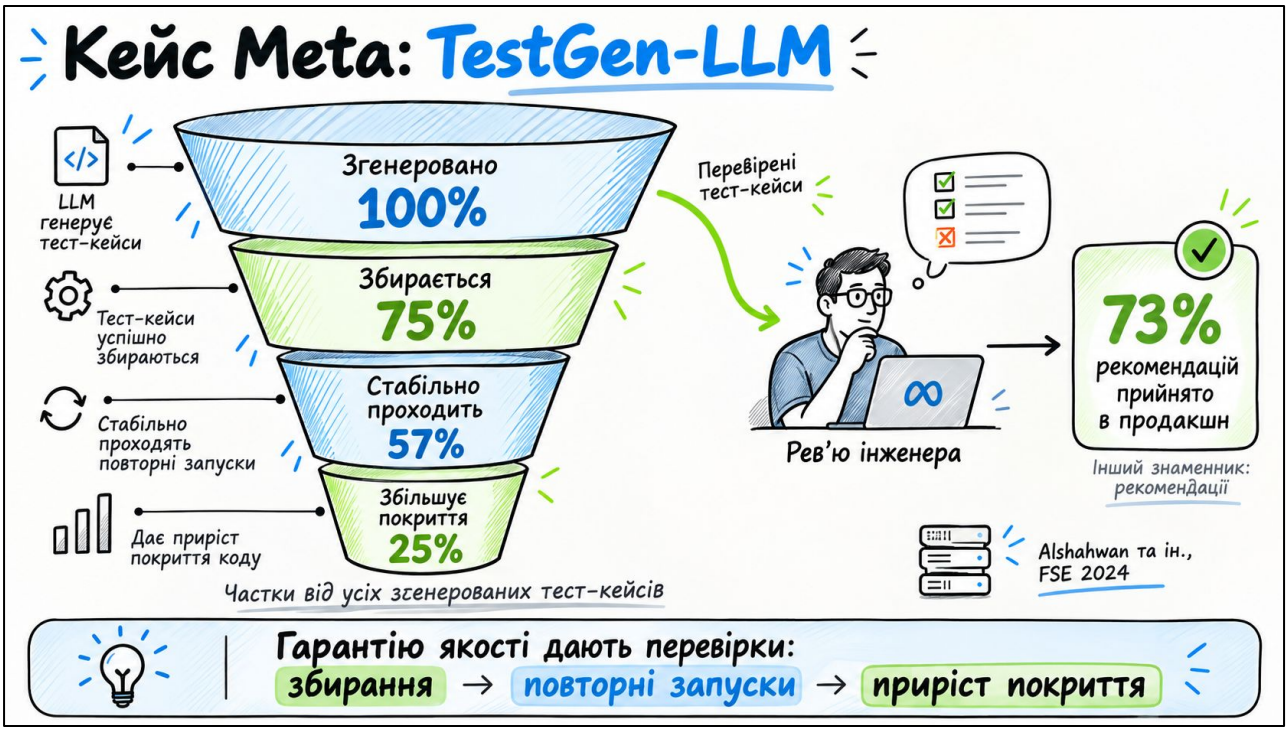
У парадигмі верифікації програмного забезпечення класична піраміда тестування набуває нової динаміки при взаємодії з великими мовними моделями. На вершині піраміди наскрізні тести (End-to-End) залишаються ресурсомісткими, повільними та крихкими, а автоматизована генерація повноцінних E2E-сценаріїв демонструє суттєво нижчу надійність через високу комбінаторну складність середовища та розмитий контекст.

Інтеграційні тести забезпечують вищий рівень реалістичності при перевірці взаємодії підсистем. Проте найбільший практичний приріст продуктивності штучний інтелект демонструє на базовому рівні — в генерації модульних тестів (unit tests). Завдяки ізольованому контексту окремих функцій, швидкому виконанню та стабільності результатів синтез модульних тестів забезпечує максимальну окупність інженерних витрат.

Якісний стрибок у надійності забезпечує перехід від фіксованих прикладів до тестування на основі властивостей — Property-Based Testing (PBT). Замість написання поодиноких assert-перевірок на довільних статичних вхідних даних інженерна команда фокусується на математичних інваріантах поведінки системи. Для алгоритмів сортування базовими властивостями виступають ідемпотентність повторного застосування та збереження мультимножини елементів, де частотний розподіл ключів до і після перестановки залишається строго незмінним.

У цьому процесі реалізується розподіл відповідальності: людина-інженер визначає та верифікує запропоновані інваріанти, тоді як генератор PBT

автоматично досліджує простір станів, синтезуючи тисячі псевдовипадкових сценаріїв. Цей підхід усуває евристичну сліпоту традиційного тестування, автономно виявляючи аномальні граничні випадки: вироджені порожні списки, множинні дублікати значень, нетривіальні символи Unicode та часові зсуви між годинними поясами. Така синергія гарантує верифікацію математичної коректності коду без перевантаження інженера рутинним кодуванням тест-кейсів.



Промислове впровадження автоматизованої генерації тестів у масштабах гіперскейлерів яскраво ілюструє кейс платформи TestGen-LLM компанії Meta, представлений Альшахваном та колегами на конференції FSE 2024 року.

Попри здатність великих мовних моделей синтезувати значні обсяги тестових сценаріїв, сам факт генерації не гарантує їхньої інженерної цінності. Для подолання прірви між стохастичним синтезом і формальною коректністю конвеєр реалізує багаторівневу фільтраційну воронку із суворими програмними шлюзами якості.

Зі ста відсотків первинно згенерованих тест-кейсів чверть відсіюється вже на етапі компіляції через синтаксичні дефекти або помилки залежностей, залишаючи сімдесят п'ять відсотків успішно зібраного коду. Наступний шлюз усуває недетермінізм і так звану флаковитість (flakiness) шляхом багаторазових повторних запусків, у результаті чого лише п'ятдесят сім відсотків тестів демонструють стабільно успішне проходження.

Нарешті, ключовий критерій інженерної доцільності вимагає реального приросту покриття коду: лише двадцять п'ять відсотків від загального обсягу кандидатів дійсно розширюють тестове покриття, а не дублюють наявні перевірки.

Принциповим є те, що саме послідовний ланцюг автоматичних перевірок — успішне збирання, стабільні повторні запуски та приріст тестового покриття — забезпечує гарантію якості ще до залучення розробника. Завдяки такій

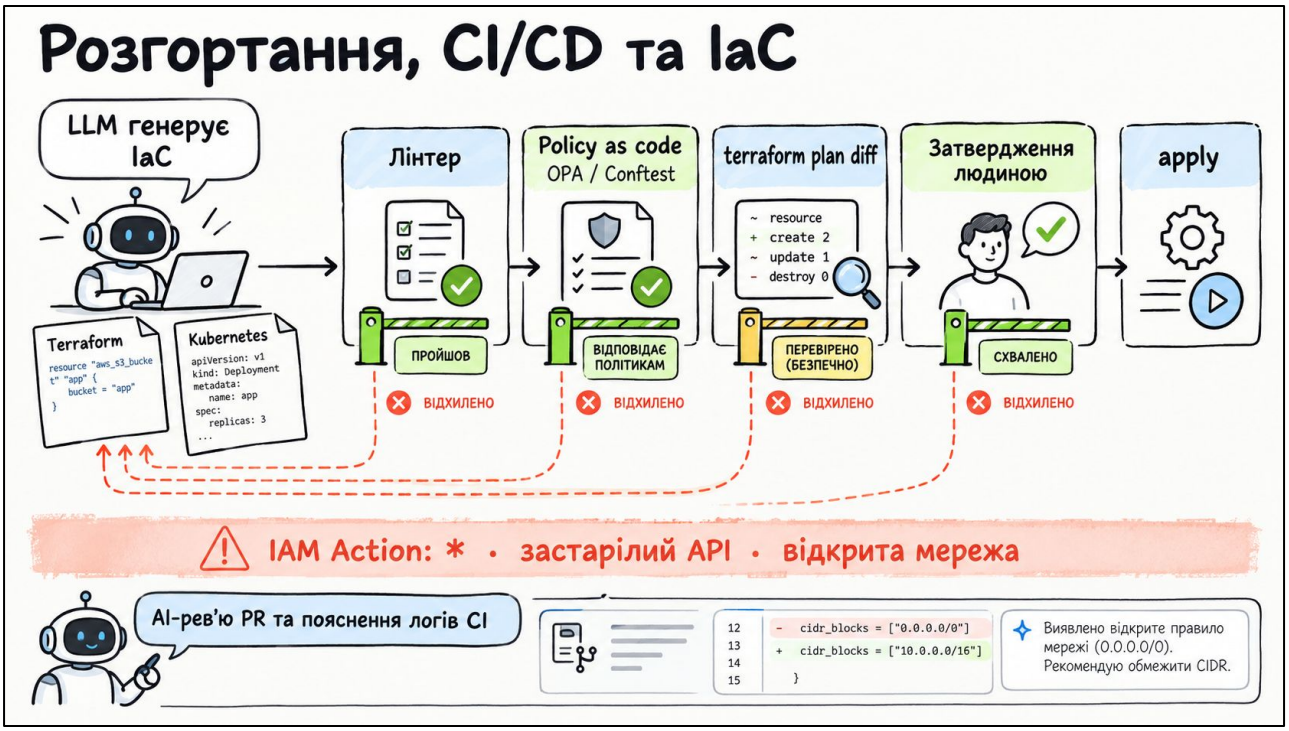
попередній фільтрації інженер отримує на рев'ю виключно верифіковані та змістовні рекомендації. Як наслідок, фахівці приймають до продакшну сімдесят три відсотки рекомендованих тест-кейсів. Цей результат підтверджує, що ефективність застосування LLM у програмній інженерії базується на детермінованих обв'язках, які беруть на себе жорстку верифікацію результатів імовірнісної генерації.

Фундаментальним бар'єром в автоматизованій верифікації програмного забезпечення є проблема тестового оракула — визначення того, чи відповідає реальна поведінка програми її дійсній істинності та призначенню. Справжній інженерний тест повинен виводитися безпосередньо зі специфікації, спираючись на формальні вимоги, бізнес-правила та системні контракти. У такому підході очікувана поведінка формується автономно: для заданого входу f(2, 3) очікується результат п'ять, що дозволяє асерту об'єктивно контролювати працездатність коду незалежно від внутрішньої реалізації та своєчасно виявляти приховані дефекти.

На противагу цьому, коли автоматизовані генератори створюють тести безпосередньо за наявною реалізацією, вони схильні просто копіювати поточну фактичну поведінку системи. Якщо у функції міститься помилка — наприклад, віднімання замість додавання, — згенерований тест механічно фіксує некоректний результат мінус один у твердження assert. Виникає небезпечна інженерна тавтологія: дефектний код успішно проходить перевірку, оскільки сам баг стає частиною тестового контракту.

Для аудиту якості тестових наборів застосовується мутаційне тестування, яке автоматично вносить дрібні штучні помилки у вихідний код і фіксує частку «вбитих» мутантів як індикатор сили тестів. Проте високий мутаційний бал є лише показником чутливості перевірок; сам по собі він не доводить семантичну коректність програми, якщо тестовий оракул за своєю природою залишається

тавтологічним.



Автоматизація інфраструктури в парадигмі «інфраструктура як код» (IaC) за допомогою великих мовних моделей суттєво прискорює синтез декларативних маніфестів Terraform та Kubernetes, проте одночасно відкриває критичні вектори атак і конфігураційних ризиків. Імовірнісна природа генеративних моделей схильна продукувати надмірно дозволені політики доступу, такі як маски IAM Action:, використовувати застарілі специфікації API або залишати периметр відкритим через небезпечні мережеві правила типу 0.0.0.0/0. У виробничих середовищах розгортання такий недетермінований код у жодному разі не може потрапляти безпосередньо у фазу виконання apply.

Надійність розгортання забезпечується каскадом детермінованих шлюзів якості в конвеєрі CI/CD. Першим рівнем захисту виступають лінтери синтаксичного та семантичного аналізу, за якими слідує етап Policy as Code на базі рушіїв Open Policy Agent (OPA) та Conftest, що перевіряють відповідність коду суворим корпоративним політикам безпеки та принципу найменших привілеїв. Наступний бар'єр виконує прецизійний аудит різниці станів — аналіз terraform plan diff, де кількісно оцінюється радіус ураження: обсяг створюваних, модифікованих чи запланованих до знищення ресурсів. Будь-яке відхилення на цих етапах активує негайний контур зворотного зв'язку, повертаючи діагностичні звіти генеративному агенту на доопрацювання.

Вирішальним шлюзом залишається обов'язкове затвердження інженером (human-in-the-loop), що гарантує розподіл відповідальності перед фінальним

застосуванням змін. При цьому штучний інтелект ефективно доповнює цей процес у ролі асистента: спеціалізовані моделі здійснюють контекстне рев'ю пул-реквестів, аналізують розриви у файлах конфігурації, дешифрують складні логи CI-конвеєрів та завчасно виявляють загрози — наприклад, рекомендуючи звузити відкриті публічні CIDR-діапазони до безпечних локальних підмереж.

В інженерії корпоративного програмного забезпечення етап супроводу — зокрема великомасштабна міграція інфраструктури та модернізація кодових баз — становить колосальну частку операційних витрат і прихованого технічного боргу.

Оновлення спадкових систем між основними версіями платформеного середовища, наприклад перехід сервісів із Java 8 або 11 на Java 17, історично вимагає тривалої рутинної праці. Проте застосування автономних агентних конвеєрів перетворює цей процес на структурований замкнений цикл: система послідовно виконує глибинний аналіз залежностей, здійснює кодові трансформації, запускає етап збирання, верифікує виконання тестових наборів, передає артефакт на рецензування інженеру та фіксує фінальне злиття в репозиторій.

Ключовим фактором надійності тут виступають детерміновані контури зворотного зв'язку: помилки збирання проєкту або падіння регресійних тестів негайно повертають логи у модуль трансформації для автоматичного виправлення ще до участі людини. Промислову валідність такого підходу підтверджує прецедент компанії Amazon 2024 року з інструментом Q Developer: трудомісткість типового оновлення скоротилася з приблизно п'ятдесяти людино-днів до лічених годин, загальний обсяг збережених ресурсів сягнув близько чотирьох з половиною тисяч людино-років, а сімдесят дев'ять відсотків згенерованих правок було прийнято взагалі без додаткового ручного втручання.

Головний висновок для архітектури супроводу полягає в синергії компонентів: вузькоспеціалізована семантична трансформація, підкріплена строгим компілятором і детермінованими тестами, формує сильний тестовий оракул, здатний повністю нейтралізувати недетермінізм імовірнісної генерації.

Автоматичне виправлення програмних дефектів (Automated Program Repair, APR) на основі великих мовних моделей перетворює розрізнені евристики усунення помилок на строго формалізований п'ятистфупеневий конвеєр інженерної верифікації. Першим етапом виступає прецизійна локалізація несправності (fault localization), що ізолює підозрілі синтаксичні конструкції та рядки у вихідному кодовому базисі. Далі генеративна модель синтезує вибірку з n конкуруючих кандидатних патчів, досліджуючи альтернативні траєкторії виправлення логіки — від модифікації крайових умов розгалуження до компактного семантичного спрощення за допомогою стандартних бібліотечних методів.

Отримані кандидати проходять валідацію через наявні модульні тести, проте на цьому етапі виникає класична проблема автоматичного відновлення коду, відома ще з часів генетичного програмування та систем GenProg, — перенавчання на тестах (test-suite overfitting). Модель здатна згенерувати поверхневий, правдоподібний патч, який успішно виконує наявні тести завдяки сліпому підлаштуванню під їхні асерти, проте порушує фундаментальну специфікацію системи. Справді коректний патч зобов'язаний не просто проходити публічний тестовий набір, а строго зберігати семантичні інваріанти програми на всьому просторі станів.

Для надійного подолання цієї сліпоти конвеєр інтегрує трирівневий захисний бар'єр. По-перше, застосовуються приховані тести (held-out tests), що

перевіряють граничні значення та нетипові сценарії виконання для унеможливлення перенавчання. По-друге, розгортається диференціальне тестування, яке співставляє поведінку вихідної програми та модифікованого кандидата на репрезентативних обсягах вхідних даних для локалізації небажаного регресійного дрейфу. По-третє, проводиться семантичне рев'ю — символічний аналіз та інспекція мовною моделлю на наявність прихованих побічних ефектів.

Ці захисні перевірки генерують об'єктивні докази коректності, на основі яких алгоритм виконує ранжування рішень, відсіваючи крихкі маніпуляції та підвищуючи пріоритет семантично надійних модифікацій. Фінальним рубежем контролю якості залишається затвердження інженером (human-in-the-loop): спеціаліст здійснює аналіз різниці версій у репозиторії, перевіряє відповідність цільовим системним контрактам і приймає зважене рішення щодо злиття патчу в основну гілку кодової бази.

Зведена матриця SDLC

Етап	LLM-сценарій	Обов'язковий контроль	Ризик	Сила оракула
Реалізація	код, рефакторинг	компіляція, тести, рев'ю	хибний код	сильний
Розгортання	IaC, логи	політики, plan diff	надмірні права	сильний
Тестування	unit, PBT	мутаційний аналіз	тавтологія	середній
Супровід	міграція, APR	тести, диф. тестування	перенавчені патчі	середній
Вимоги	stories, неоднозначності	стейкхолдер	фантомні вимоги	слабкий
Проєктування	ADR, OpenAPI	архітектурне рев'ю	усереднене рішення	слабкий

Найбільший приріст — там, де є сильний автоматичний оракул

Комплексний аналіз життєвого циклу розробки програмного забезпечення в умовах інтеграції генеративного штучного інтелекту демонструє, що інженерна ефективність та надійність мовних моделей фундаментально визначаються силою верифікаційного оракула. Вздовж усього конвеєра SDLC операційний ризик перебуває в обернено пропорційній залежності від спроможності автоматизованого контролю. На етапах реалізації та розгортання великі мовні моделі забезпечують найвищий практичний приріст продуктивності саме завдяки наявності сильного автоматичного оракула. Під час синтезу коду, рефакторингу, генерації інфраструктури як коду (IaC) чи аналізу логів критичні ризики хибних інструкцій та надмірних привілеїв детерміновано блокуються компіляторами, тестовими наборами, політиками безпеки та аудитом різниці станів до моменту потрапляння в продакшн.

Натомість на стадіях тестування та супроводу системи сила оракула зміщується до середнього рівня. Генерація модульних чи property-based тестів і автоматичне відновлення програм у процесі міграцій несуть загрозу тавтологічних тверджень та перенавчання патчів на наявних тестах, що вимагає залучення мутаційного аналізу та диференціального тестування. Найбільш уразливими залишаються аналітичні фази інженерії вимог та архітектурного проєктування, де автоматичний оракул є принципово слабким. Коли модель декомпозує вимоги чи створює чернетки архітектурних рішень (ADR), ризики виникнення неперевірених фантомних вимог або нав'язування шаблоних

«усереднених» мікросервісних топологій неможливо перевірити програмним кодом. Тут єдиним джерелом істини залишається прямий контроль людини: верифікація стейкхолдером та фахове архітектурне рев'ю. Головний закон системної інженерії з використанням штучного інтелекту незмінний: максимальний практичний приріст досягається там, де імовірнісна генерація обмежена сильним автоматичним оракулом.

Для строгого аналізу таксономії інструментів розробки в сучасній програмній інженерії необхідно формалізувати їхнє оцінювання через тривимірний простір, утворений трьома незалежними ортогональними властивостями: обсягом контексту, детермінованістю виходу та рівнем автономності системи. Будь-який інструмент розробки — від класичних компіляторних утиліт до систем на базі машинного навчання — позиціонується як дискретна точка в системі цих трьох координат.

Вертикальна вісь, обсяг контексту, охоплює чотири ієрархічні рівні видимості: від локальної функції чи блоку до окремого файлу, цілого репозиторію та системного масштабу всієї організації. Вісь глибини визначає детермінованість результату, проводячи чітку межу між математично детермінованими аналізаторами на основі формальних граматик і стохастичними, недетермінованими виходами великих мовних моделей.

Нарешті, горизонтальна вісь фіксує автономність, відстежуючи еволюційний шлях від простого автодоповнення в редакторі коду до повноцінних автономних агентів із замкненим циклом дій.

Показовим є зіставлення двох систем: статичний аналізатор репозиторію характеризується широким охопленням контексту та абсолютною детермінованістю за низької автономності. Натомість мовне автодоповнення у файлі функціонує в межах локального контексту файла за невисокої автономності, проте вносить фактор недетермінованого статистичного синтезу.

Сприйняття будь-якого інструмента як точки в просторі цих трьох незалежних осей дозволяє системному архітектору усвідомлено балансувати евристичну гнучкість генеративних моделей із вимогами детермінованої надійності впродовж усього життєвого циклу розробки.

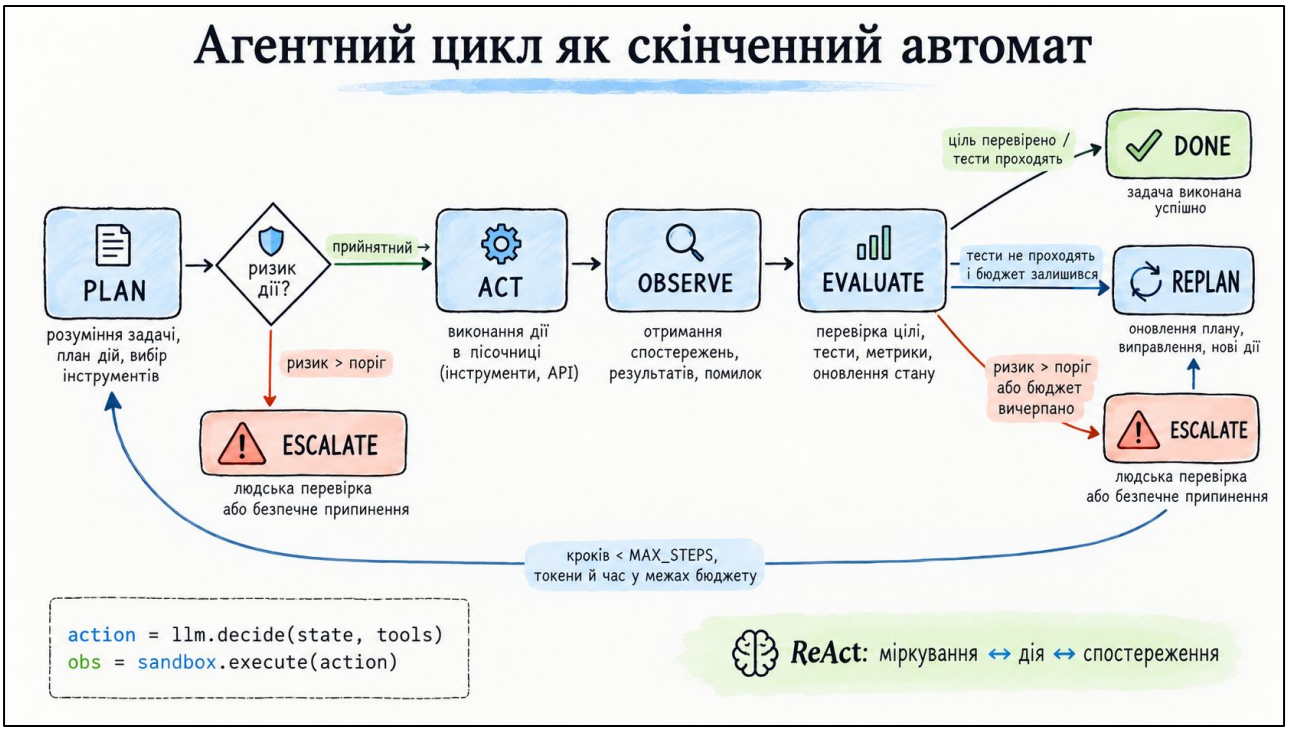
Еволюція інтелектуальних інструментів розробки розгортається вздовж осі автономності, яка за аналогією до класичних таксономій Шерідана-Верпланка 1978 року та рівнів автопілота SAE J3016 структурує взаємодію людини та машини за чотирма дискретними рівнями. Кожен рівень перерозподіляє ролі ініціатора, виконавця, суб'єкта затвердження та глибину контролю людини в інженерному контурі.

На першому рівні — автодоповненні — людина повністю контролює процес написання коду в IDE, тоді як система пропонує локальні синтаксичні завершення. Затвердження здійснюється миттєво: інженер приймає або відхиляє пропозицію, зберігаючи безперервний контроль над кожним окремим фрагментом. Другий рівень, представлений chat-асистентами, переводить комунікацію у формат діалогу, де розробник формулює конкретний запит, система генерує відповідь, а людина самостійно здійснює рев'ю та інтегрує запропонований артефакт у робочу кодову базу перед його внесенням.

Якісний стрибок у парадигмі відбувається на третьому рівні — агентах із підтвердженням. Тут інженер лише ставить високорівневу ціль, тоді як автономний агент самостійно будує план виконання та синтезує послідовність дій. Проте інженерний контроль залишається детермінованим: кожна транзакція, виклик інструмента чи зміна файлу вимагає обов'язкового попереднього схвалення людиною (human-in-the-loop) перед виконанням кожної дії.

Нарешті, четвертий рівень — повністю автономні агенти — реалізує

повноцінний замкнений цикл дій: агент самостійно планує кроки, виконує операції, аналізує зворотний зв'язок від середовища та автономно коригує свою поведінку. Людина відходить від мікроменеджменту виконання і зосереджується на аудиті кінцевого результату виключно на стратегічних контрольних межах і шлюзах якості, що кардинально масштабує продуктивність інженерних процесів за умови надійної валідації.



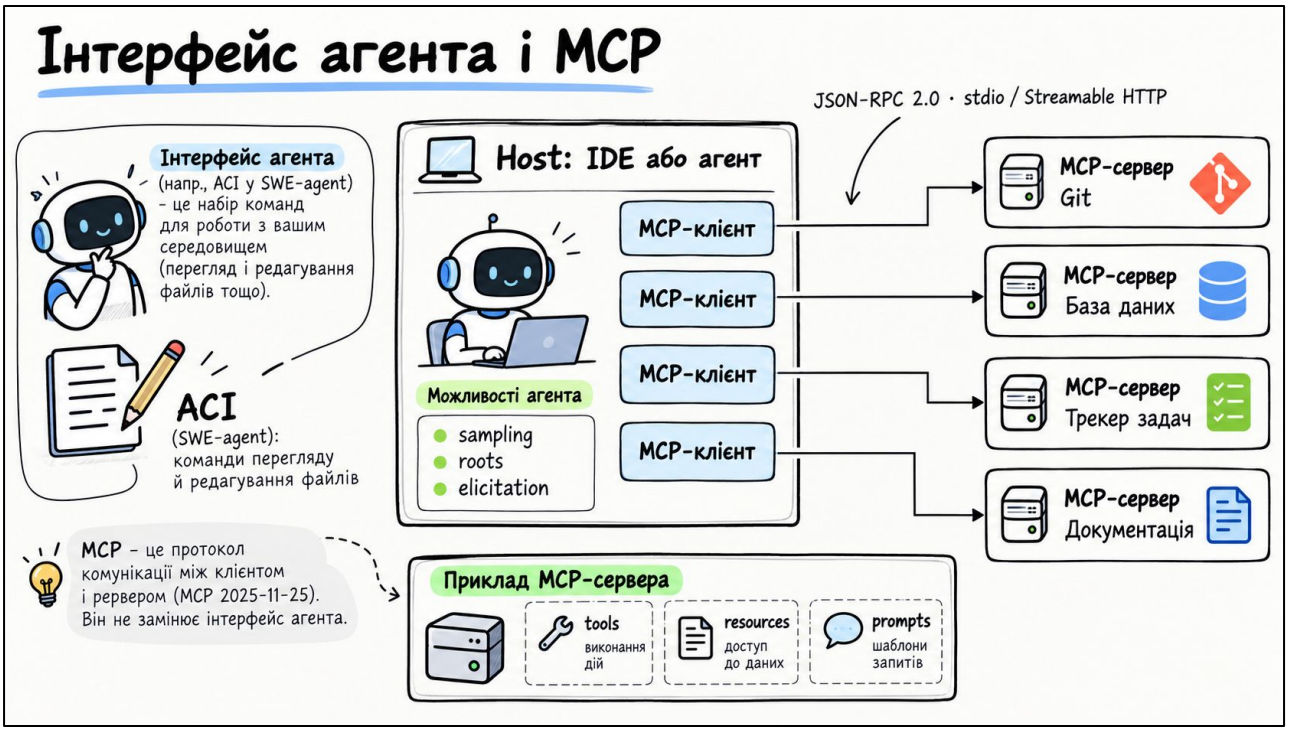
Формалізація життєвого циклу автономного програмного агента вимагає переходу від евристичних взаємодій до строгої моделі скінченного автомата (Finite State Machine, FSM), де кожен перехід між станами детермінований системними інваріантами та обмеженнями бюджету. В основі цієї архітектури лежить класичний патерн ReAct, що реалізує замкнений контур динамічного чергування міркування, дії та спостереження: агент обчислює цільову дію на базі поточного стану та реєстру утиліт, $action = \text{llm.decide}(state, tools)$, після чого середовище детерміновано повертає спостереження через ізольовану пісочницю.

Виконавчий цикл розпочинається зі стану планування (PLAN), де агент формує семантичне розуміння задачі, структурує декомпозицію дій та обирає необхідні інструменти. Проте перехід до виконання не є безумовним: обов'язковим шлюзом безпеки виступає оцінка ризику планованої операції. Якщо прогнозований ризик перевищує встановлений поріг — наприклад, при спробі деструктивної модифікації файлової системи чи небезпечного мережевого виклику, — автомат переходить у стан ескалації (ESCALATE) для залучення людини або безаварійного зупинення процесу. Лише за умови прийнятного ризику система переходить у стан дії (ACT), виконуючи операцію в ізольованій пісочниці через санкціоновані API.

Наступний стан спостереження (OBSERVE) фіксує результати виконання, вихідні дані інструментів та коди помилок середовища, транслюючи їх у стан

оцінки (EVALUATE). Тут здійснюється валідація цілі, запуск верифікаційних тестів, аналіз метрик та оновлення вектора стану. Якщо критерії успішності підтверджено і тести проходять, автомат переходить у фінальний термінальний стан успіху (DONE).

Якщо ж тести падають, але операційний бюджет не вичерпано, активується стан динамічного перепланування (REPLAN). Агент вносить корективи, генерує виправлення та формує новий цикл дій за умови дотримання жорстких інженерних інваріантів: сумарна кількість кроків не перевищує ліміт MAX_STEPS, а витрати токенів і час виконання залишаються в межах виділеного ресурсного бюджету. Якщо ж поріг ризику порушено або бюджет вичерпано, автомат гарантовано спрямовує виконання в безпечну ескалацію (ESCALATE), запобігаючи нескінченним циклам розходження та неконтрольованій деградації системи.



Архітектурна інтеграція автономних програмних агентів у виробничі екосистеми вимагає строгого концептуального розмежування між інтерфейсом агента та відкритими протоколами взаємодії, зокрема Model Context Protocol (MCP). Інтерфейс комп'ютера агента (Agent-Computer Interface, ACI), вперше системно реалізований у SWE-agent, визначає спеціалізований набір операційних примітивів, адаптованих під специфіку мовних моделей, таких як навігація файловою структурою, інспекція вихідного коду та точкове редагування контексту.

Натомість MCP не замінює інтерфейс взаємодії з кодом, а виступає стандартизованим відкритим транспортним протоколом клієнт-серверної архітектури на базі JSON-RPC 2.0, що функціонує через стандартні потоки введення-виведення (stdio) або потоковий HTTP (Streamable HTTP). У межах хост-середовища, яким виступає IDE або агентний фреймворк, множинні MCP-клієнти забезпечують ізольовані сесії зв'язку з розподіленими MCP-серверами, що надають уніфікований доступ до системи контролю версій Git, реляційних баз даних, трекерів задач та корпоративних сховищ документації.

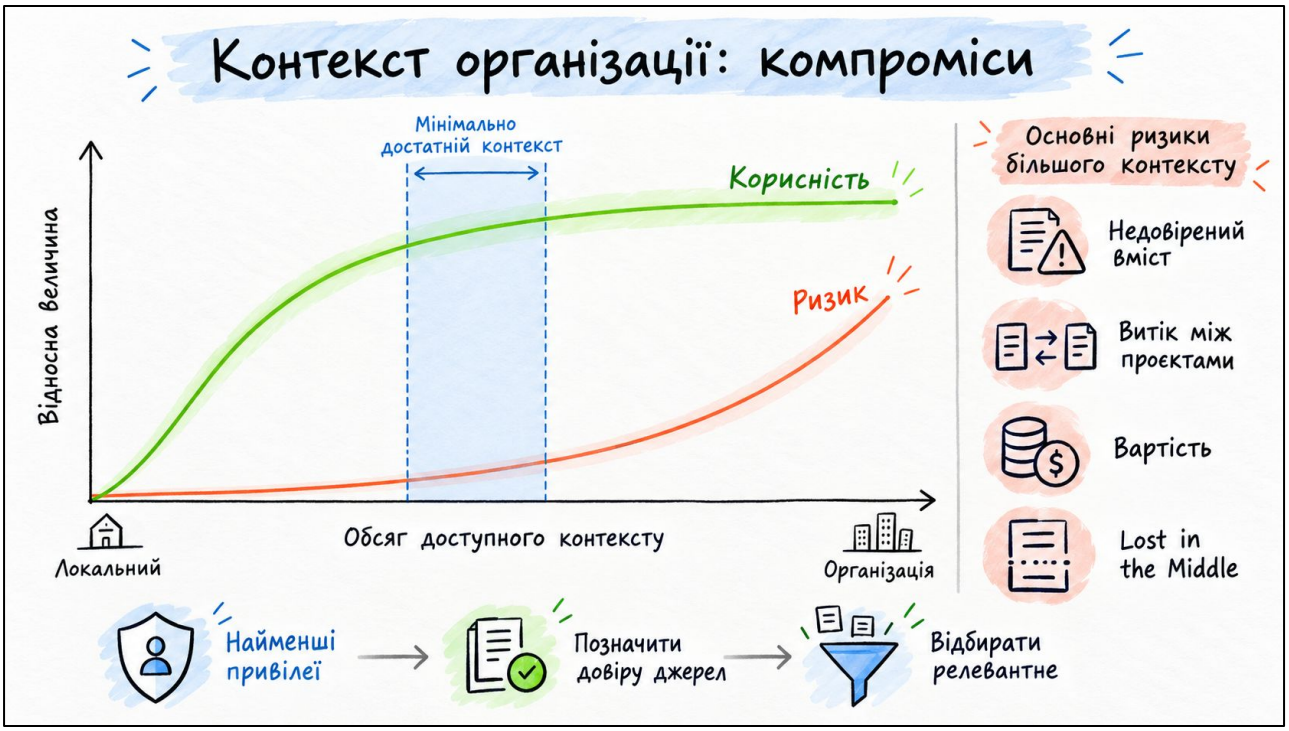
Агентний хост оперує базовими системними можливостями протоколу — керованим семплюванням (sampling), визначенням кореневих робочих просторів (roots) та зворотною взаємодією для уточнення параметрів (elicitation). Кожен MCP-сервер експонує три ортогональні типи артефактів: інструменти (tools) для детермінованого виконання виконуваних дій, контекстні ресурси (resources) для

безпечного читання внутрішніх даних та шаблонізовані запити (prompts) для стандартизації інженерних запитів. Така декомпозиція гарантує модульність системи, ізолюючи операційну логіку моделі від гетерогенної інфраструктури зовнішніх сервісів.

Керування віссю контексту в конвеєрах програмної інженерії вимагає строгої ієрархічної стратегії, яка масштабується від окремої функції до екосистеми цілої організації. На найбільш локалізованому рівні функції джерела інформації чітко обмежені сигнатурами та реалізацією, що дозволяє виконувати пряму подачу коду в буфер контексту. Перехід на рівень файлу розширює горизонт до поточного модуля, де основним інструментом стає локальний пошук.

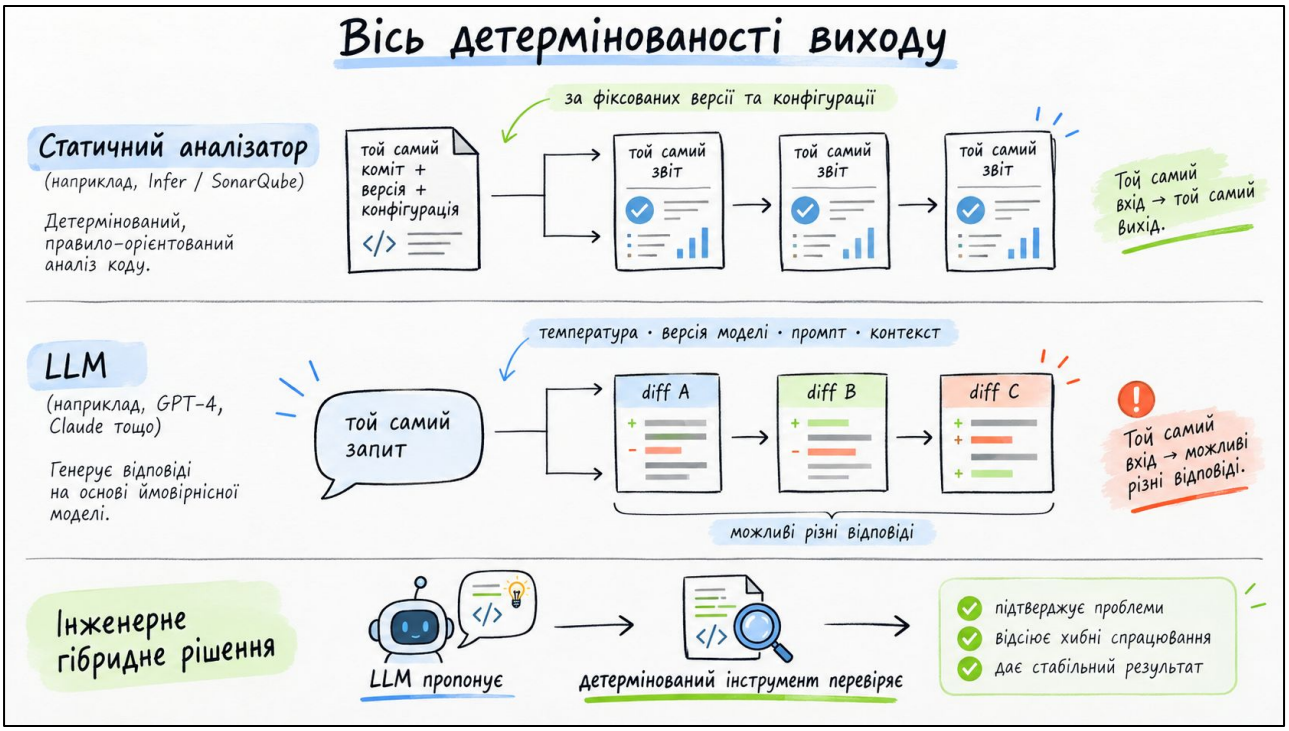
На межі репозиторію структурна складність якісно зростає: джерелами даних стають міжмодульні залежності, історія комітів системи контролю версій та трекери дефектів. Тут доставка контексту забезпечується формуванням карти репозиторію (repo map), графами символів та спеціалізованим семантичним пошуком, що структурується проєктними інструкціями для автономних систем на кшталт AGENTS.md. Нарешті, на рівні організації знання виходять за межі одного кодового сховища, охоплюючи множину пов'язаних репозиторіїв та внутрішню технічну документацію, що вимагає архітектури Retrieval-Augmented Generation (RAG) та протоколу Model Context Protocol (MCP).

Вздовж усієї цієї ієрархії незмінним залишається ключовий інженерний принцип: обмежене вікно моделі має наповнюватися прецизійно відібраними, семантично релевантними даними, а не надлишковим сирим масивом усього репозиторію.



Масштабування обсягу контексту від локального коду до рівня всієї організації підпорядковується фундаментальному компромісу між користю та наростаючими операційними ризиками. Початкове розширення контексту демонструє стрімке зростання інженерної корисності, проте згодом крива виходить на плато, виявляючи ефект спадної віддачі. Водночас графік ризиків демонструє нелінійне експоненційне зростання. Неконтрольоване надання доступу до корпоративних репозиторіїв відкриває критичні вектори загроз: проникнення недовіреного вмісту в генеративний контур, витік конфіденційних даних між ізольованими проєктами, стрімке здорожчання вартості інференсу через роздуті токенні буфери, а також деградацію уваги через ефект «Lost in the Middle».

Для подолання цієї дилеми системна архітектура повинна фокусуватися на концепції мінімально достатнього контексту. Замість недиференційованого завантаження всієї кодової бази організації, конвеєр має реалізувати трирівневий контур безпеки та оптимізації. Насамперед впроваджується принцип найменших привілеїв, що обмежує права доступу агента до системних ресурсів. Далі накладається маркування та перевірка довіри до джерел даних для блокування ін'єкцій та сумнівного коду. На завершальному етапі селективні фільтри відбирають виключно релевантні фрагменти, гарантуючи високу точність синтезу рішень без виходу в червону зону системних ризиків.



Вісь детермінованості виходу визначає фундаментальну граничну умову для сучасного інструментарію інженерії програмного забезпечення, розмежовуючи алгоритмічну передбачуваність та імовірнісну варіативність. Класичні рушії статичного аналізу, такі як Infer або SonarQube, спираються на детермінований, орієнтований на правила аналіз кодової бази. За умови фіксованої версії компілятора, незмінних конфігурацій та визначеного стану коміту ці системи є строго ідемпотентними: ідентичний вхідний вектор незмінно відтворює абсолютно той самий діагностичний звіт при багаторазових запусках.

На противагу цьому великі мовні моделі функціонують у межах стохастичної парадигми, генеруючи відповіді на базі розподілу ймовірностей токенів, на який безпосередньо впливають температура семплювання, версія чекпоїнту, топологія промпту та динамічний контекст. Як наслідок, однаковий запит закономірно призводить до генерації варіативних кодових змін, повертаючи різні дифи (diff A, diff B, diff C) і створюючи недетермінованість у конвеєрі розробки.

Вирішенням цього протиріччя виступає інженерне гібридне рішення. У такій архітектурі мовна модель виступає генератором евристичних гіпотез і пропонує варіанти модифікацій, тоді як детермінований інструмент бере на себе роль верифікатора. Така синергія надійно підтверджує виявлені дефекти, відсіює хибні спрацювання та галюцинації, забезпечуючи стабільний, відтворюваний результат у промислових процесах розробки.

Недетермінованість у генеративних робочих процесах становить фундаментальну інженерну проблему, що ставить під загрозу базові атрибути якості програмної системи: відтворюваність, стабільність CI-конвеєрів, надійність рев'ю та аудит. Якщо класична програмна інженерія ґрунтується на детермінізмі, то стохастична природа мовних моделей безпосередньо порушує цю передбачуваність.

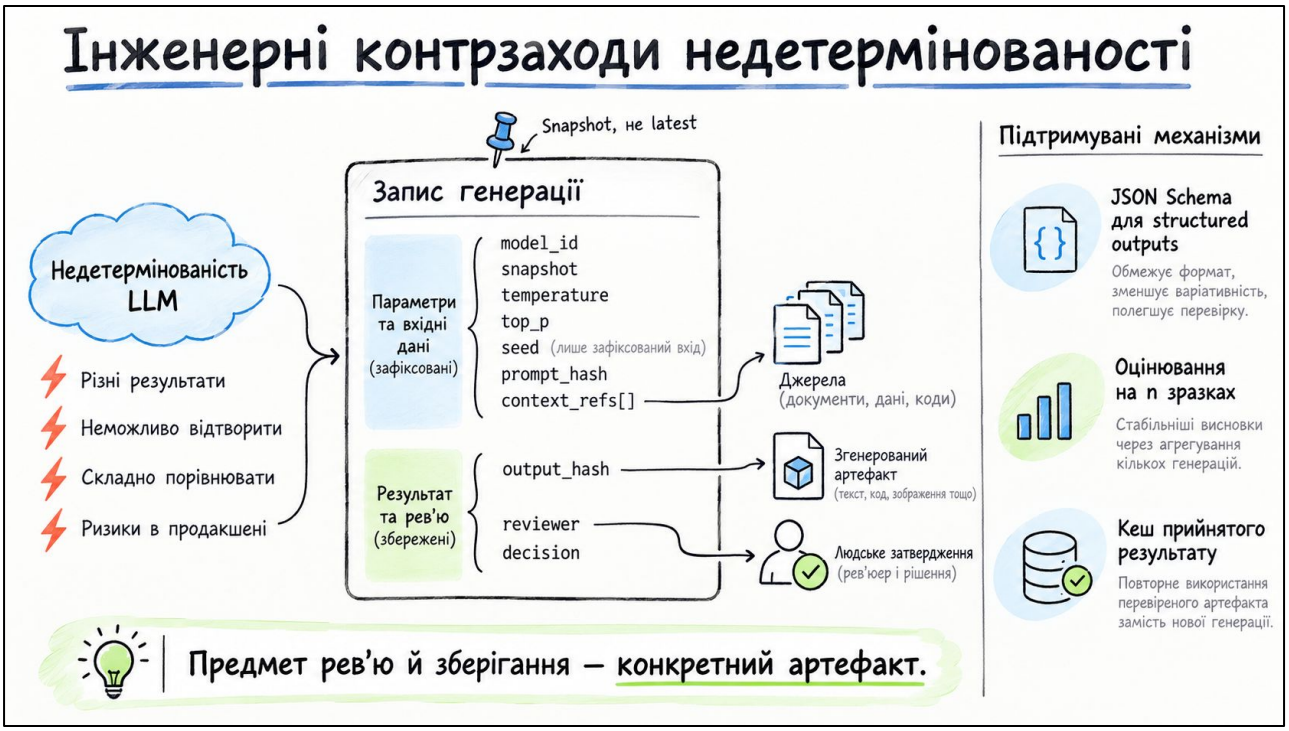
У розрізі відтворюваності недетермінований вихід ускладнює повторення вчорашніх дефектів, оскільки система на той самий вхід може сформувати іншу відповідь. Необхідним контрзаходом стає обов'язкова фіксація та збереження згенерованого виходу разом із повним набором параметрів семплювання, таких як температура та версія моделі.

Для стабільності безперервної інтеграції варіативність виходу призводить до хибних результатів і розбіжностей на одному й тому самому коміті. Подолання цієї нестабільності реалізується через багаторівневу перевірку та агресивне кешування проміжних станів.

На стадії інженерного рев'ю виникає ризик розсинхронізації: нова спроба генерації не дорівнює раніше схваленому дифу коду. Тому процес затвердження повинен проводитися виключно для незмінного, зафіксованого артефакту.

Нарешті, етап аудиту наражається на ризик невідомого походження коду та відсутності простежуваності. Ця загроза нейтралізується веденням повного журналу генерації, що гарантує детальне трасування кожного синтезованого

блоку. Наскрізним інваріантом усіх етапів залишається ключове інженерне правило: правдоподібний вигляд коду не є доведеною коректністю системи.



Недетермінованість великих мовних моделей створює низку критичних ризиків для промислового програмного забезпечення: варіативність результатів, неможливість повторення дефектів, ускладнене порівняння змін і загрозу дестабілізації релізів у продакшені. Для нейтралізації цих стохастичних ефектів інженерний процес повинен переходити від прямої довіри до моделі до суворої фіксації так званого запису генерації (generation provenance). Замість динамічного виклику плаваючих версій типу latest, архітектура зобов'язана спиратися на фіксовані снепшоти моделей із детермінованим набором параметрів: model_id, робочий зріз чекпоїнта, показники temperature, top_p, системний seed, а також хеш промпту та явні посилання на контекстні джерела — документи, кодові модулі й системні бази даних.

Принциповим правилом верифікації виступає збереження результату та контексту рев'ю: системний конвеєр реєструє хеш згенерованого артефакту, ідентифікатор інженера-рев'юера та його вердикт щодо схвалення. Такий підхід підкріплюється трьома системними механізмами контролю. Першим є використання JSON Schema для структурованого виводу, що звужує простір відповідей, придушує синтаксичну варіативність і полегшує автоматичну валідацію контракту. Другим є вибіркове оцінювання на вибірці з n зразків, де агрегація кількох незалежних генерацій усуває аномальні викиди й забезпечує статистично стійкі висновки. Третім механізмом виступає кешування прийнятого результату, яке дозволяє повторно експлуатувати попередньо верифікований

артефакт замість ініціації нових стохастичних запусків. Головний інженерний постулат незмінний: предметом інспекції, рев'ю та довготривалого збереження в репозиторії є не абстрактний промпт чи модель, а конкретний, детерміновано зафіксований та затверджений артефакт.

Метрика pass@k

$$pass@k = \mathbb{E}\left[1 - \frac{C(n-c,k)}{C(n,k)}\right]$$

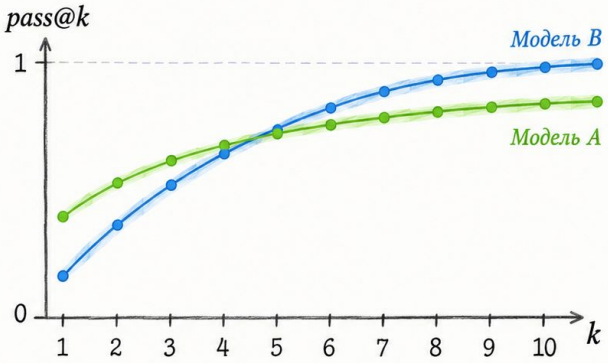
n — усі зразки · c — правильні · k — обрані

```
from math import comb

def pass_at_k(n, c, k):
    if n - c < k:
        return 1.0
    return 1 - comb(n-c, k) / comb(n, k)
```

Умовний приклад на різнорідних задачах

Перетин можливий через різні ймовірності успіху на окремих задачах (ретерогенність); це не емпіричний результат.



pass@k > 1 корисна, коли є автоматичний оракул для вибору.

Оцінювання спроможності великих мовних моделей до синтезу програмного коду вимагає незміщених імовірнісних оцінок, серед яких центральне місце посідає метрика pass@k, призначена для аналізу функціональної коректності. Замість обчислення наївного емпіричного відсотка успіху на обмеженій кількості спроб, ця оцінка визначає математичне сподівання ймовірності того, що щонайменше один згенерований фрагмент успішно пройде модульні тести за умови вибірки k зразків без повернення із загального пулу n спроб, який містить c правильних реалізацій: формула на слайді.

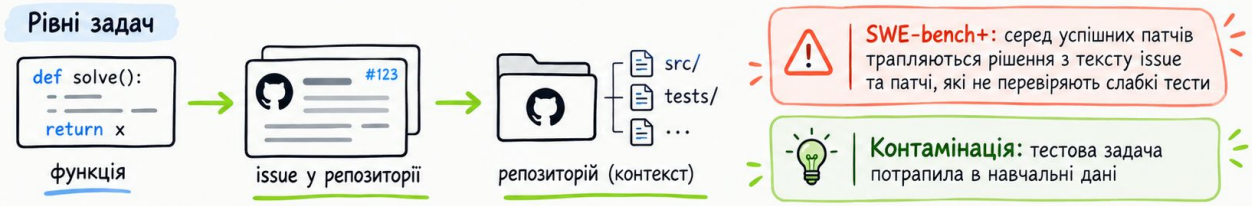
В алгоритмічній реалізації, якщо кількість помилкових кандидатів n - c менша за бюджет вибірки k, потрапляння хоча б одного коректного рішення є математично гарантованим, повертаючи значення 1.0. В іншому випадку комбінаторне співвідношення через число сполучень обчислює доповнення до ймовірності вибору k поспіль дефектних варіантів, що ефективно пригнічує дисперсію та усуває похибки вибірки.

Під час порівняння систем на гетерогенних наборах задач проявляється різна динаміка масштабування. Модель із вищою базовою точністю при одиничному запиті, як-от Модель A при k=1, може поступатися при збільшенні вибірки моделі з більшою варіативністю, такій як Модель B, через різну ймовірність розв'язання специфічних підзадач. Проте принциповим інженерним правилом є те, що оптимізація під метрику pass@k > 1 має практичну цінність виключно за наявності сильного автоматичного оракула для відбору рішень. За

відсутності компілятора чи детермінованих тестів, здатних автоматично верифікувати успішного кандидата серед множини згенерованих версій, багаторазова стохастична генерація лише перевантажує систему неперевіреним кодом.

Бенчмарки LLM4SE та їхні межі

Набір	Рівень задачі	Розмір	Оракул	Основне обмеження
HumanEval	функція	164	unit-тести	можливий витік
MBPP	функція	—	тести	можливий витік
SWE-bench	issue у репозиторії	2294	тести issue	обв'язка агента
SWE-bench Verified	issue у репозиторії	500	тести + людська перевірка задачі	обв'язка, слабкі тести
LiveCodeBench	конкурсні задачі	оновлюється	тести	дата публікації



Оцінювання великих мовних моделей для потреб програмної інженерії безпосередньо впирається в структурні обмеження та архітектуру тестових оракулів сучасних бенчмарків. Перші фундаментальні набори даних, такі як HumanEval та MBPP, фокусуються виключно на рівні окремої ізольованої функції, використовуючи базові unit-тести для перевірки результатів. Попри свою історичну роль, ці набори страждають від критичного ризику витоку даних і контамінації, коли тестові завдання та готові розв'язки опиняються безпосередньо в навчальних корпусах моделей.

Для відтворення умов реального виробництва вектор тестування змістився до рівня репозиторію, що втілено в бенчмарках SWE-bench та SWE-bench Verified. Тут агент стикається з реальними issue у відкритих репозиторіях, а оракулом виступають цільові тести завдання. Проте такий підхід несе суттєві методичні викривлення: успішність розв'язання значною мірою визначається якістю зовнішньої обв'язки агента, а розширені аудити на кшталт SWE-bench+ виявили, що частина нібито успішних патчів насправді запозичує прямі вказівки з опису самого issue або проходить крізь слабкі, нестрогі перевірки.

Для нейтралізації проблеми контамінації з'явилися динамічні набори на зразок LiveCodeBench, де конкурсні завдання регулярно оновлюються та жорстко калібруються за датою публікації. Еволюція від генерації окремих функцій до репозиторного контексту наочно доводить: показники бенчмарків

вимірюють силу тестового оракула та надійність інженерної обв'язки не менше, ніж саму генеративну спроможність моделі.

Сучасний ландшафт інструментів програмної інженерії структурується за трьома ортогональними вимірами: обсягом контексту, операційною автономністю та детермінованістю виходу. Класичні статичні аналізатори — такі як Infer, SonarQube та Semgrep — займають область абсолютної детермінованості та широкого охоплення кодової бази репозиторію, проте функціонують із нульовою автономністю, повертаючи суворо передбачувані діагностичні звіти на ідентичний стан вхідних даних.

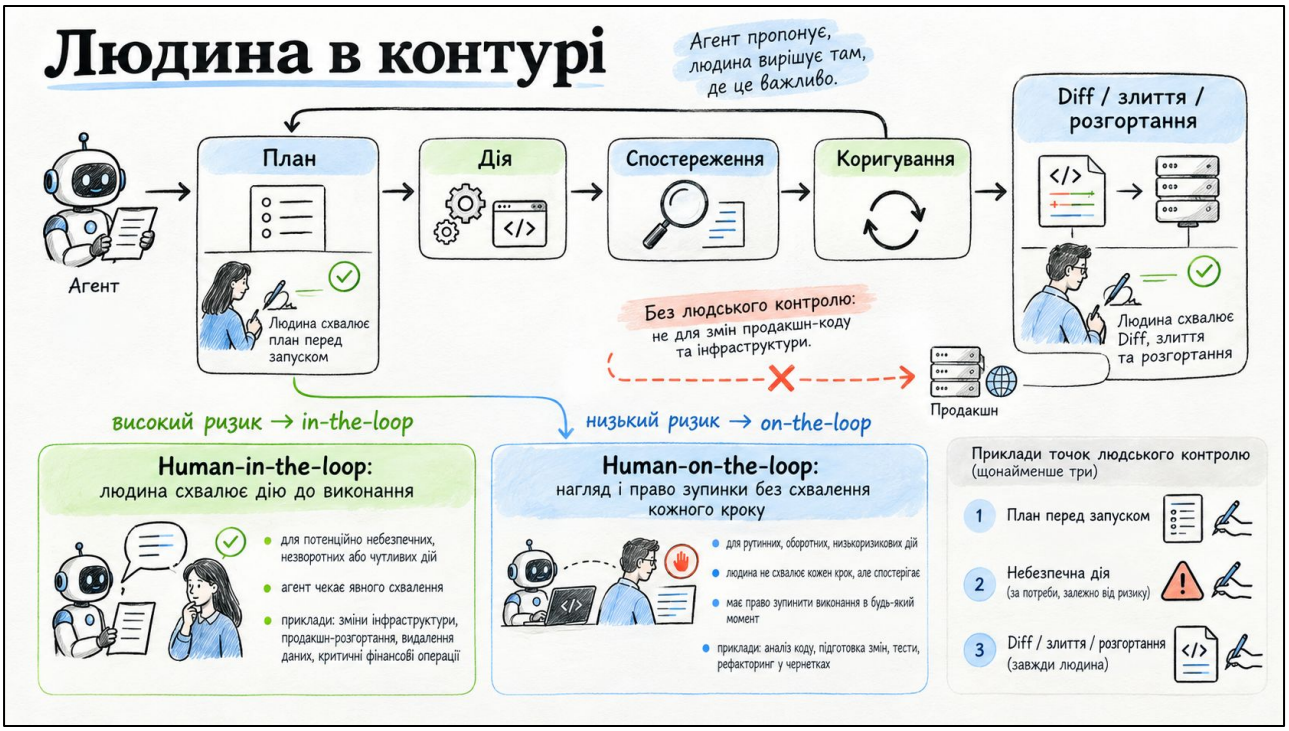
Інтеграція генеративних моделей фундаментально трансформує цю таксономію. На базовому рівні inline-автодоповнення в IDE функціонує в межах локального контексту відкритих файлів із мінімальною автономністю. Діалогові chat-асистенти розширюють цей периметр до наданих користувачем фрагментів, залишаючись під безпосереднім керуванням людини. Водночас інженерний вектор демонструє виразну системну тенденцію: агресивний рух у напрямку охоплення ширшого контексту та нарощування рівня автономності.

Ця еволюція спершу кристалізується в агентних CLI- та IDE-середовищах — таких як Claude Code, Cursor Agent чи Copilot coding agent, — які оперують контекстом усього репозиторію та отримують часткову автономність для виконання безпосередніх інженерних дій у середовищі розробки. На вершині цієї траєкторії розташовані фонові агенти, зокрема Devin та OpenClaw, які через наскрізні інтеграції з Git, конвеєрами CI/CD, трекерами задач і хмарною інфраструктурою отримують доступ до всього технологічного стека. Вони здатні

самостійно декомпозувати, планувати та асинхронно виконувати тривалі багаторокові інженерні задачі у фоновому режимі. Зі зростанням автономності та системної видимості таких агентів впровадження детермінованих перевірочних обв'язок стає головним архітектурним імперативом для забезпечення математичної коректності програмного продукту.

Поява автономних програмних агентів вимагає строгого наукового аналізу та незалежної верифікації, що наочно демонструє кейс агента Devin від компанії Cognition. У 2024 році розробники заявили про вирішення 13,86% завдань без допомоги людини, що склало 79 успішних розв'язків із випадково відібраної підмножини у 570 задач із повного бенчмарку SWE-bench, який налічує 2 294 кейси. Після публічних демонстрацій було оприлюднено формальний протокол із методологічним описом, вихідними патчами та обв'язкою оцінювання (eval harness).

Проте спроба незалежної перевірки дослідницькою групою Answer.AI у 2025 році продемонструвала значні методологічні розбіжності. Під час тестування на 20 інших завданнях, які не входили до SWE-bench, система продемонструвала лише 3 успіхи за 14 невдач та 3 неоднозначних результатах. Це підкреслює критичну проблему порівнянності: використання відмінних вибірок завдань унеможливлює пряме зіставлення показників. До того ж, точного відтворення оригінального експерименту на тій самій вибірці досягнуто не було. Будь-які інженерні заяви про системну автономність потребують відкритих журналів трасування, повного аудиту запусків та верифікації на незалежних, неконтамінованих тестових наборах.

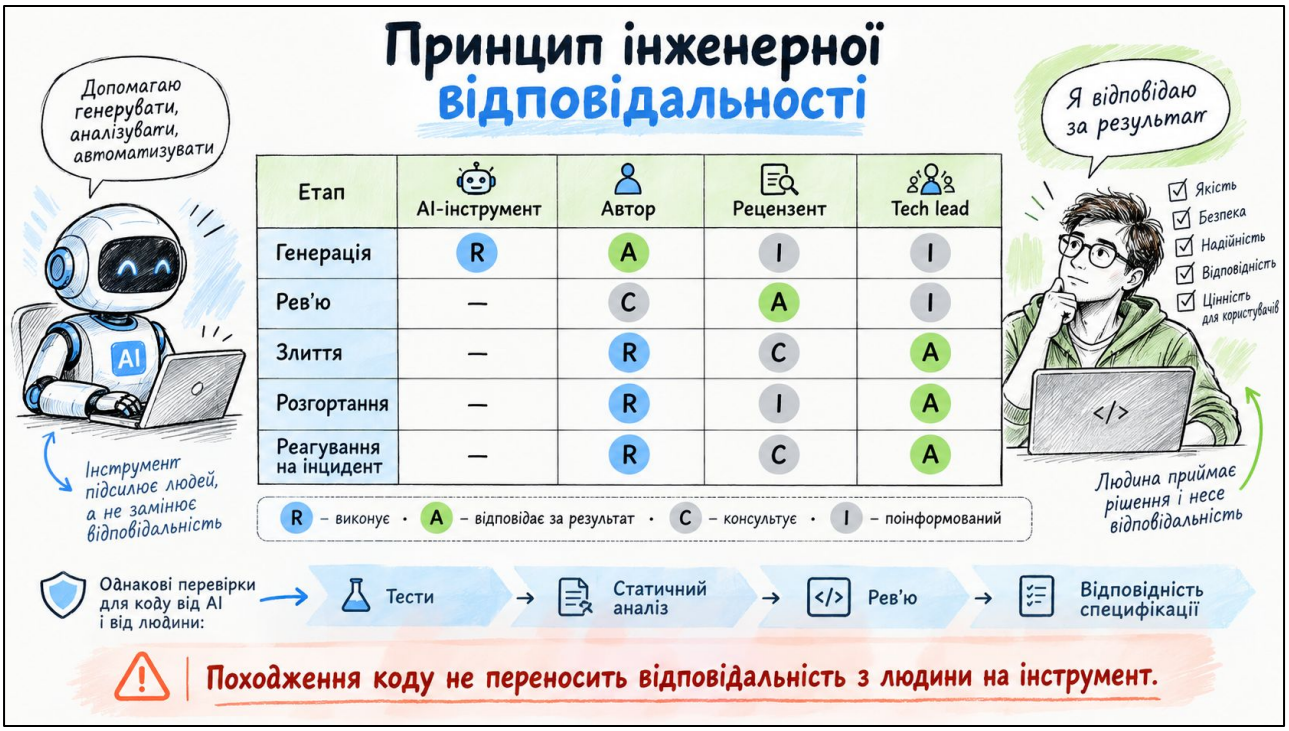


В інженерії автономних програмних систем делегування операційних завдань штучному інтелекту вимагає строгої моделі управління, зосередженої на контролі з боку людини. Фундаментальний принцип визначає чіткий розподіл ролей: агент пропонує, але людина приймає остаточне рішення у точках критичного впливу. Застосування неконтрольованої автоматизації є неприпустимим: прямий перехід неперевірених змін у продакшн-код та інфраструктуру суворо заблокований.

Виконавчий контур агента послідовно охоплює етапи планування, дії, спостереження та коригування. Для гармонізації швидкості розробки та системної безпеки нагляд розділяється на два режими залежно від профілю ризику. Для високоризикових, незворотних або чутливих дій — таких як модифікація інфраструктури, розгортання в робочому середовищі, видалення баз даних чи критичні фінансові операції — впроваджується модель «людина в контурі» (Human-in-the-loop). У цьому разі агент блокує виконання і чекає на пряме та явне схвалення кожної операції. На противагу цьому, для низькоризикових, оборотних і рутинних задач — зокрема аналізу коду, підготовки змін, генерації тестів чи рефакторингу в чернетках — застосовується модель «людина над контуром» (Human-on-the-loop). Інженер не затверджує кожен атомарний крок, але здійснює безперервний нагляд і володіє правом миттєвої зупинки процесу в будь-який момент.

Надійна виробнича архітектура фіксує щонайменше три ключові

контрольні точки: попереднє схвалення плану дій перед запуском, вибірковий контроль небезпечних дій за потреби, а також обов'язкове фінальне людське затвердження дифу змін, операції злиття в репозиторій та випуску в продакшн.



Принцип інженерної відповідальності встановлює незмінний контур управління в процесах розробки програмного забезпечення, підсилених штучним інтелектом. Хоча генеративні моделі виступають потужним каталізатором продуктивності — допомагаючи генерувати структури, аналізувати код та автоматизувати рутину, — вони не здатні нести юридичну, функціональну чи етичну відповідальність за створені артефакти. Впровадження інструментів штучного інтелекту вимагає чіткої формалізації інженерних ролей за матрицею RACI вздовж усього життєвого циклу проєкту.

На етапі первинного синтезу AI-інструмент діє як виконавець, тоді як автор-інженер несе повну відповідальність за результат генерації, залишаючи рецензента й техліда в курсі процесу. Перехід до стадій рев'ю, злиття гілок у репозиторій, розгортання в інфраструктурі та реагування на інциденти повністю виключає модель із суб'єктів прийняття рішень. Автор виступає виконавцем або консультує, а остаточна відповідальність за підсумковий стан системи однозначно закріплюється за рецензентом та технічним лідером команди. Таким чином, саме інженер залишається єдиним гарантом системної якості, кібербезпеки, надійності архітектури, нормативної відповідності та кінцевої цінності продукту для користувача.

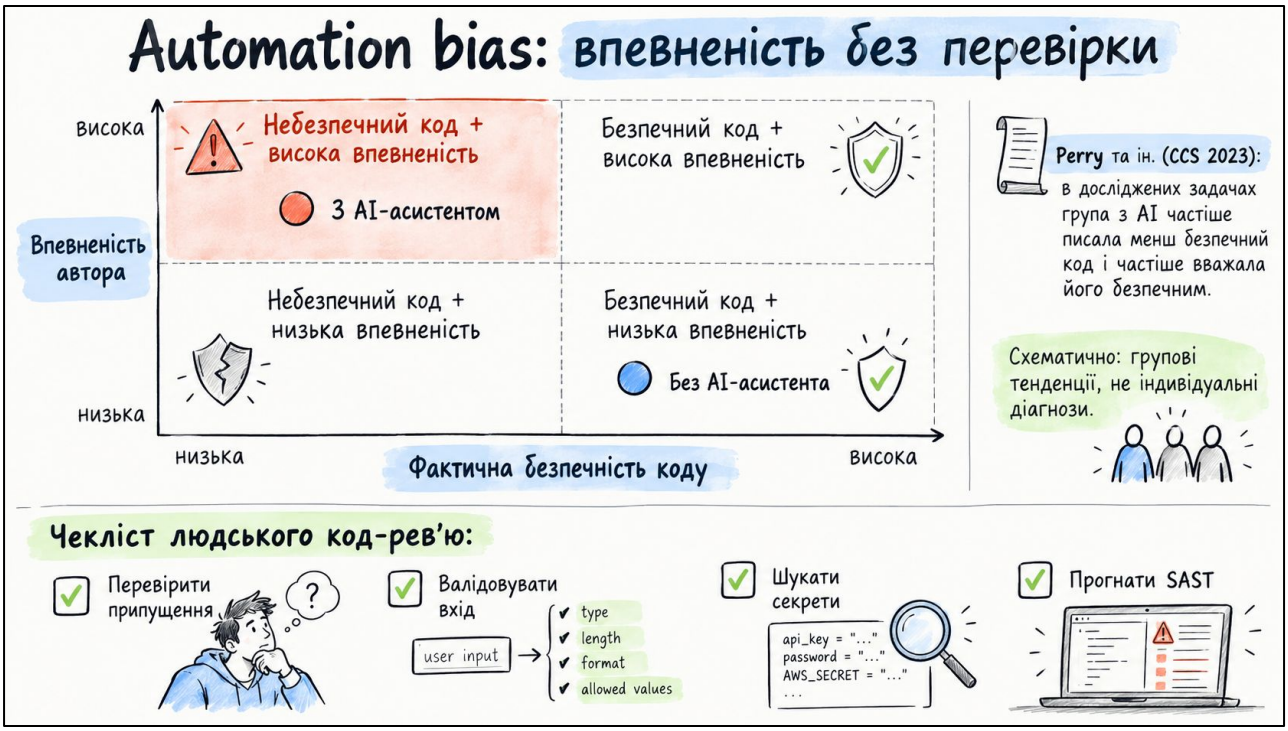
Для забезпечення паритетності перевірок конвеєр верифікації зобов'язаний застосовувати єдиний суворий ланцюг контролю як до коду від штучного інтелекту, так і до коду, написаного людиною: автоматизовані тести,

статичний аналіз кодової бази, ретельне інженерне рев'ю та валідацію системних специфікацій. Походження програмного коду за жодних обставин не переносить професійну відповідальність з людини на допоміжний інструмент

Широке впровадження генеративних інструментів у програмну інженерію актуалізує фундаментальний соціотехнічний парадокс, сформульований Лізанною Бейнбрідж ще у 1983 році — «іронії автоматизації». У міру того як штучний інтелект перебирає на себе все більшу частку автоматизованої роботи, з часом виникає небезпечний концептуальний розрив. З одного боку, рутинні завдання все частіше делегуються моделям, через що практичний досвід інженера деградує: менша кількість повторень призводить до втрати інженерної форми та притуплення професійної інтуїції. З іншого боку, складність ручних винятків нелінійно зростає: нетипові збої трапляються рідше, але стають набагато складнішими. Виникає критична точка розриву, коли інженер, позбавлений постійної практики, змушений вирішувати заплутані архітектурні інциденти, з якими не впоралася автоматизація.

Для нейтралізації цього операційного ризику інженерний менеджмент повинен впроваджувати системні превентивні заходи. По-перше, необхідно виділяти регулярний час без використання AI, свідомо виконуючи частину задач вручну для підтримки кваліфікації та внутрішнього відчуття кодової бази. По-друге, рев'ю коду має стати пояснювальним: перевірка згенерованих артефактів повинна супроводжуватися вимогою детальних обґрунтувань для збереження глибокого семантичного розуміння та своєчасного виявлення хибних рішень. По-третє, критично важливо забезпечити ротацію складних задач, навмисно залишаючи за інженерами нестандартні інженерні виклики для безперервного

відтворення та збереження експертизи в команді.



Когнітивний феномен упередження автоматизації (automation bias) створює приховану загрозу для безпеки програмного забезпечення, коли розробники інтегрують генеративні AI-асистенти в процеси синтезу коду. Якщо зіставити суб'єктивну впевненість автора з фактичним рівнем захищеності реалізованого коду, виявляється критична диспропорція. У традиційній інженерній практиці без участі AI-асистентів розробники демонструють адекватну оцінку власних рішень: написання потенційно небезпечного коду зазвичай супроводжується низькою впевненістю, що спонукає інженера до додаткових консультацій і перевірок.

На противагу цьому, емпіричне дослідження Перрі та колег, представлене на конференції ACM CCS 2023, показало, що група розробників із AI-асистентами значно частіше зміщується в небезпечний квадрант високої впевненості за низької фактичної безпеки. Синтаксично бездоганний та правдоподібний вигляд згенерованих конструкцій формує оманливу ілюзію надійності, спонукаючи авторів вважати об'єктивно вразливий код безпечним.

Контрзаходом проти цього викривлення виступає строгий інженерний чекліст людського код-рев'ю. Інженер зобов'язаний критично верифікувати неявні припущення моделі, ретельно валідувати вхідні дані за типом, довжиною, форматом і діапазоном допустимих значень, цілеспрямовано шукати зашиті секрети й токени доступу, а також в обов'язковому порядку запускати інструменти статичного аналізу безпеки (SAST) перед злиттям змін у

репозиторій.

Побудова надійних шлюзів верифікації в сучасній програмній інженерії вимагає порівняльного розуміння профілів помилок, характерних для розробників-людей та великих мовних моделей. Типи відмов біологічних і синтетичних учасників розробки кардинально розходяться за фундаментальними вимірами когніції та обчислень.

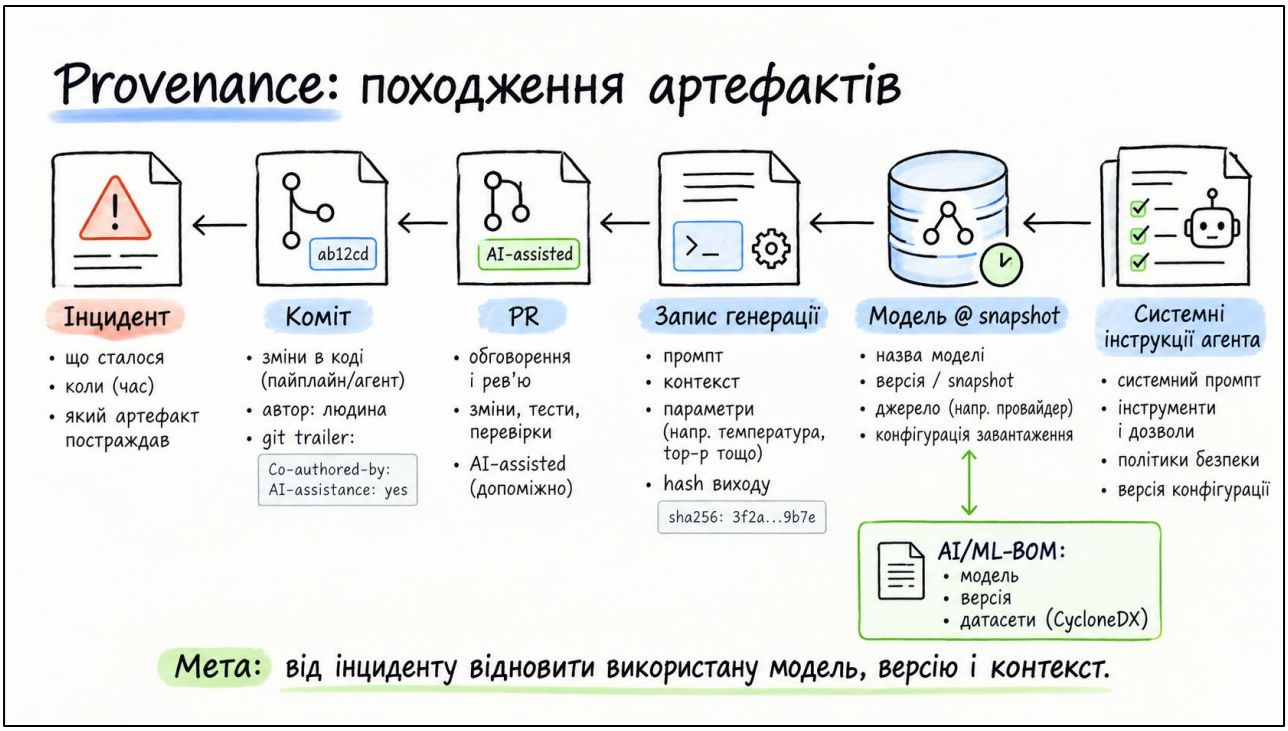
Під час звернення до інтерфейсів інженери рідко використовують неіснуючі API, а якщо це трапляється, помилка є очевидною; натомість мовні моделі здатні з високою впевненістю синтезувати повністю вигадані бібліотечні виклики. Щодо застарілих практик, рівень людини визначається особистим досвідом, тоді як поведінка моделі строго лімітована корпусом навчання та обсягом доступного контексту. У семантичному аспекті людські помилки зазвичай є локальними прорахунками, тоді як генеративний штучний інтелект продукує синтаксично бездоганний і правдоподібний текст, що приховує хибну логіку.

Крім того, людська увага об'єктивно деградує під впливом втоми, тоді як модель не зазнає фізіологічного виснаження, стикаючись натомість із системними обмеженнями контекстного вікна. Принципово відрізняється і сигнал невпевненості: людина експліцитно маркує сумнівні місця коментарями TODO чи прямими запитаннями, тоді як модель майже ніколи не позначає власну невпевненість. Нарешті, масштаб похибки має різний радіус ураження: помилка людини зазвичай локалізована в межах одного пул-реквесту, тоді як

алгоритмічний дефект моделі системно масштабується на весь кодовий базис через тиражування одного хибного шаблону. Таким чином, архітектура перевірок зобов'язана точно добиратися відповідно до специфіки кожного профілю помилок.

Побудова стійкого конвеєра безперервної інтеграції для згенерованого або виправленого коду вимагає обов'язкового багатоступеневого контуру перевірки перед операцією злиття в репозиторій. Цикл починається зі строгої статичної типізації та контролю компіляції за допомогою Муру та Ruff, що усуває синтаксичні дефекти та порушення типів на ранньому етапі. Наступний шлюз реалізує лінтинг та статичний аналіз безпеки через Semgrep для перехоплення відомих вразливостей і порушень системних правил. Третій рівень здійснює аудит сторонніх залежностей за допомогою pip-audit, перевіряючи існування бібліотек, їхній вік, наявність вразливостей і сумісність ліцензій.

Далі зміни проходять детерміновану динамічну верифікацію модульним тестуванням у pytest та мутаційним аналізом через mutmut, що гарантує виявлення логічних збоїв замість формального покриття рядків. Проте чисті звіти сканерів безпеки не тотожні повній відсутності дефектів: інструменти SAST знаходять лише відомі патерни загроз. Саме тому п'ятим і шостим шлюзами виступають рецензування людиною за структурованим чеклістом та валідація відповідності системній специфікації, що оцінюють архітектурну цілісність і контекст, недосяжні для автоматизації. Будь-яке непроходження окремої перевірки повертає задачу на початковий цикл генерації та виправлення. Успішне проходження всіх перевірок є необхідною умовою для злиття, але не абсолютною гарантією якості: кінцева відповідальність за внесення змін незмінно покладається на людину.



Управління безпекою та аудитом у системах із генеративним штучним інтелектом вимагає неперервного трасування походження артефактів — концепції Provenance. Головна інженерна мета цієї архітектури полягає в тому, щоб у разі виникнення інциденту в продакшені мати можливість детерміновано відновити точну версію моделі, її конфігурацію, системні інструкції та вихідний контекст, які спричинили появу дефекту. Ланцюг простежуваності розгортається у зворотному напрямку від збою до першоджерела генерації.

Коли фіксується експлуатаційний інцидент — із чітким часом фіксації, описом аномалії та переліком скомпрометованих артефактів, — розслідування починається з аналізу конкретного коміту в репозиторії. Кожен такий коміт фіксує автора-інженера, містить хеш змін та обов'язковий Git trailer із метаданими типу Co-authored-by: AI-assistance: yes. На рівні пул-реквесту це пов'язується з журналом інженерного обговорення, результатами CI-перевірок, мутаційними тестами та маркуванням синтезованого коду.

Критичним вузлом трасування є незмінний запис генерації, який агрегує повний вхідний промпт, посилання на контекст, параметри семплювання — такі як температура й top-p, — а також криптографічний хеш SHA-256 отриманого виходу. Цей запис однозначно посилається на зафіксований зріз моделі (snapshot), що виключає дрейф поведінки через неконтрольовані оновлення постачальника. Відповідність закріплюється через AI/ML-BOM за стандартами на кшталт CycloneDX, де каталогізуються моделі, версії та тренувальні датасети.

Замикають ланцюг системні інструкції агента: системний промпт, матриця дозволів для інструментів, політики безпеки та версія конфігурації. Завдяки такій інженерній дисципліні будь-який дефект у кодовій базі перетворюється з невідомої стохастичної помилки на повністю відтворювану й підзвітну системну подію.

Інтеграція штучного інтелекту в інженерію програмного забезпечення та корпоративну інфраструктуру вимагає надійної правової та нормативної рамки, яка встановлює наскрізну архітектуру відповідності. Інженерне управління перестало бути факультативним питанням внутрішньої гігієни; воно функціонує як сувора ієрархічна спадковість, що розгортається від абстрактних загальних вимог законодавства до конкретних інженерних перевірок у конвеєрі розробки.

Вершиною цієї ієрархії виступає пряме законодавство, представлене передусім Регламентом Європейського Союзу про штучний інтелект (EU AI Act). Цей документ закріплює ризик-орієнтований підхід та поетапне впровадження регуляторних обмежень, покладаючи юридичну відповідальність залежно від класу критичності системи. Для трансляції законодавчих норм в інженерну площину другий рівень залучає міжнародні рамки та стандарти: NIST AI Risk Management Framework 1.0 із його ключовими функціями управління (Govern), картографування ризиків (Map), вимірювання (Measure) та контролю (Manage), а також стандарти ISO/IEC 42001:2023 для систем менеджменту AI та ISO/IEC 25059:2023 для оцінки системної якості.

Ці нормативні базиси декомпозуються на третьому рівні в процеси конкретної організації. На цьому етапі впроваджується корпоративна політика AI, матриці оцінювання ризиків, чітко визначені інженерні ролі та незмінний журнал прийняття рішень. Нарешті, на операційному фронті комплаєнс перетворюється на повсякденні практики команди: автоматизовані тестові набори, експертне

людське рев'ю, детермінований контроль змін у системі контролю версій та безперервний телеметричний моніторинг робочого середовища.

Судовий прецедент *Moffatt v. Air Canada* (2024 року) остаточно зафіксував незворотність цієї моделі: суд визнав компанію повністю відповідальною за хибну та оманливу інформацію, надану її веб-чатботом. Посилання на стохастичну чи непередбачувану природу моделі не звільняє від цивільної та фінансової відповідальності: надійна інженерна дисципліна зобов'язана контролювати кожен крок системи від юридичної норми до фінального байта в продакшені.

Таксономія відмов великих мовних моделей у програмній інженерії вимагає строгого біфуркаційного поділу на дефекти рівня моделі та вразливості системного рівня. На базовому рівні моделі стохастична природа синтезу створює загрозу використання вигаданих API, неіснуючих пакетних залежностей і хибних параметрів функцій. Ця проблема посилюється часовими межами навчання, через які модель пропонує застарілі інженерні практики, а також синтезом зовні правдоподібного коду з прихованою семантичною помилкою, яку складно виявити поверхневим рев'ю. Додатково виникає ризик інжекції небезпечних шаблонів коду, що супроводжується сикофантією та надмірною впевненістю моделі, яка безпідставно підтверджує хибні припущення користувача.

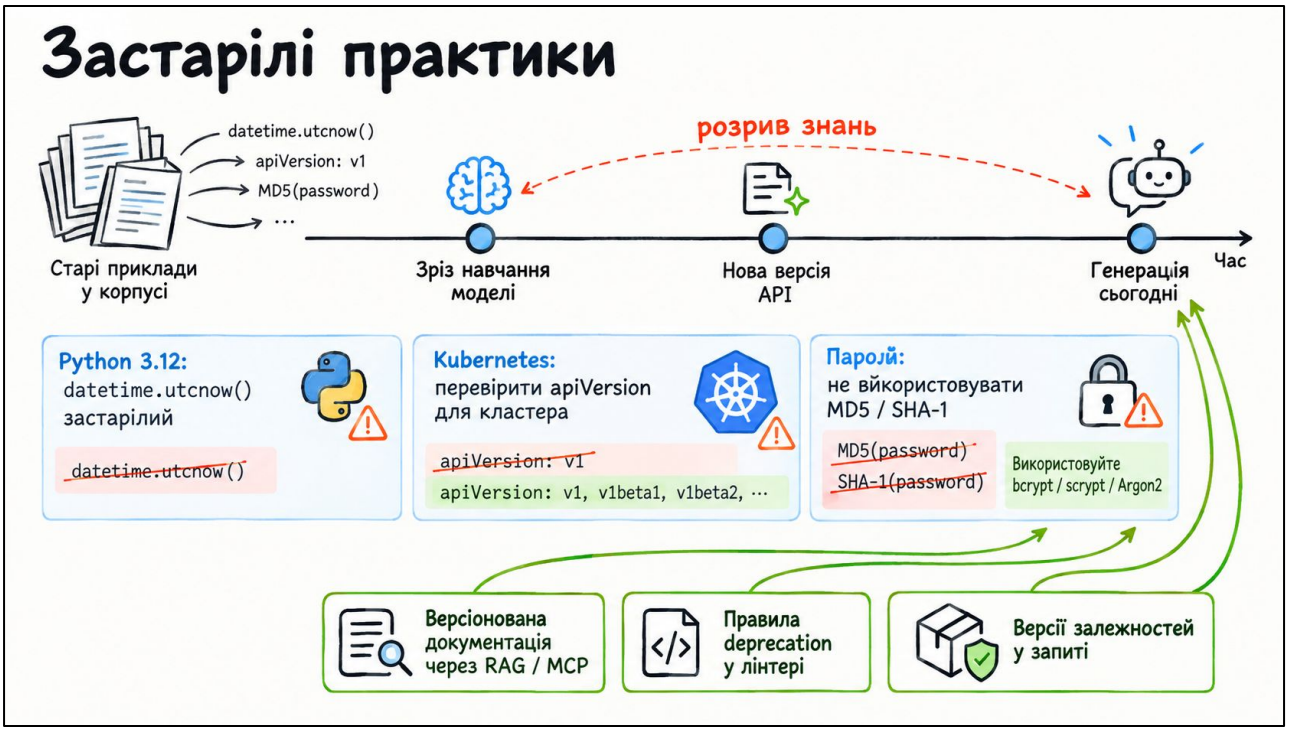
Натомість перехід моделі в контур автономної системи породжує якісно інший спектр системних загроз. Тут критичними векторами стають атаки типу prompt injection через недовірені зовнішні джерела контексту, а також надмірна агентність, що надає системі надлишковий рівень привілеїв у середовищі виконання. Не менш небезпечною є компрометація інструментів, через яку шкідливий виклик API може спричинити витік даних або пошкодження інфраструктури, та зациклення агента, що призводить до нескінченних ітерацій і швидкого вичерпання виділеного ресурсного бюджету. Системна надійність досягається лише тоді, коли інженерний контроль масштабується від ізольованого аналізу збоїв моделі до комплексної верифікації всієї

інструментальної системи.

Перетин проблеми галюцинацій великих мовних моделей та безпеки ланцюга постачання програмного забезпечення породив критичний вектор загроз, відомий як слопсквотинг (slopsquatting). У межах цієї моделі загроз генеративна модель стохастично синтезує вихідний код, що містить правдоподібні, проте повністю вигадані назви бібліотек та модулів. Зловмисники цілеспрямовано фіксують такі повторювані генеративні імена, превентивно реєструють вигадані пакети у публічних реєстрах на кшталт PyPI або npm та впроваджують у ці простори імен шкідливий функціонал. Коли розробник або агентний конвеєр запозичує згенерований код і запускає процес інсталяції, у системі відбувається виконання довільного коду з подальшою компрометацією локального середовища розробки чи виробничого контуру.

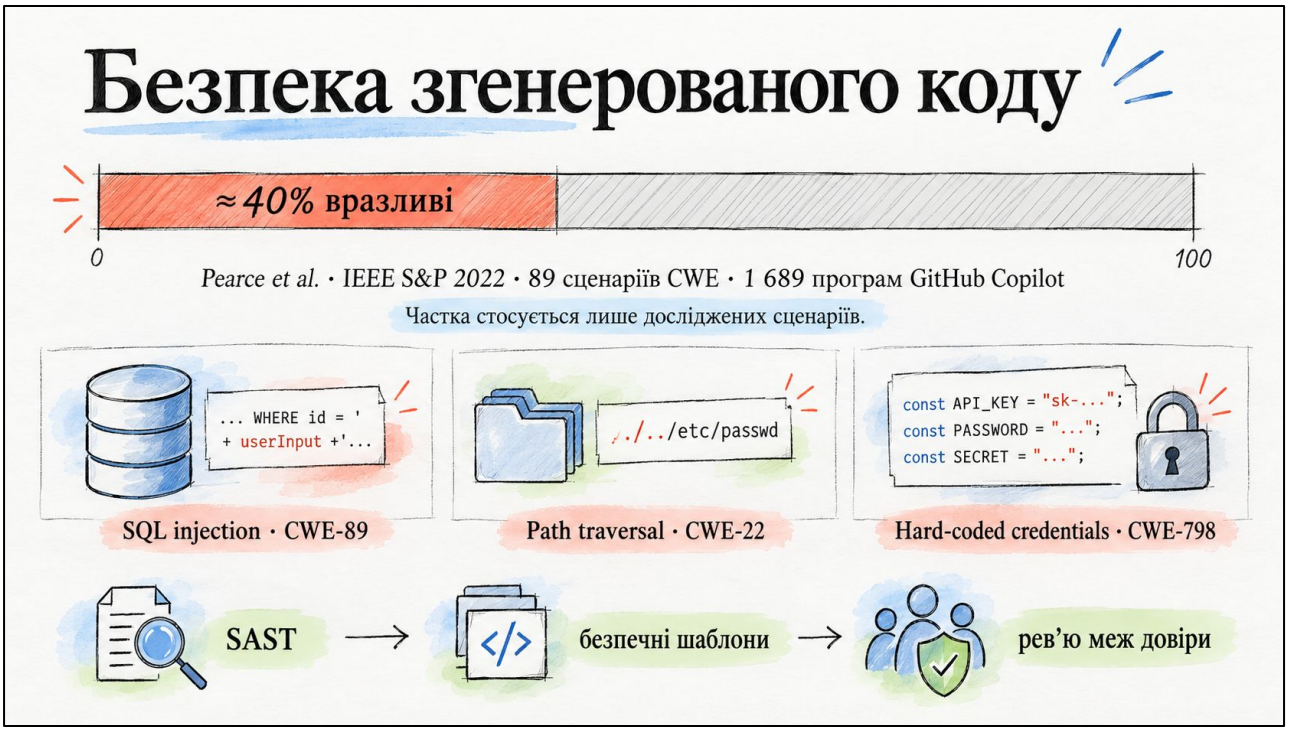
Масштаб цієї поверхні атак емпірично підтверджено дослідженням Спраклена та колег на конференції USENIX Security 2025: під час аналізу вибірки з 576 000 зразків було виявлено 205 474 унікальні вигадані назви пакетів, при цьому частота галюцинацій сягала близько 5,2% у комерційних системах і зростала приблизно до 21,7% у відкритих моделях. Для усунення цієї системної загрози інженерна інфраструктура управління залежностями вимагає впровадження суворих контрзаходів. Обов'язковими практиками стають використання затверджених білих списків (allow-list) перевірених пакетів, обов'язковий автоматизований аудит віку та репутації бібліотек, детермінована фіксація дерева залежностей через lock-файли, а також категорична заборона

автоматичного встановлення сторонніх пакетів у неізольованих агентних та CI-середовищах.



Фундаментальною вразливістю при використанні генеративних мовних моделей для розробки програмного забезпечення є тиражування застарілих практик, зумовлене розривом знань у часі. Навчальні корпуси моделей закономірно перенасичені застарілими прикладами коду та історичними інженерними підходами. Як наслідок, між фіксованим зрізом навчання моделі та сучасною генерацією виникає суттєва часова прірва: виходять нові версії API, оновлюються системні бібліотеки та переглядаються криптографічні стандарти безпеки. Під час генерації коду сьогодні модель продовжує впевнено пропонувати застарілі конструкції, такі як оголошений застарілим виклик datetime.utcnow() у Python 3.12, неактуальні маніфести apiVersion для кластерів Kubernetes або критично небезпечне гешування паролів алгоритмами MD5 чи SHA-1 замість стійких функцій формування ключів, таких як bcrypt, scrypt чи Argon2.

Нейтралізація цього розриву вимагає інтеграції детермінованих інженерних шлюзів безпосередньо в конвеєр розробки. Насамперед необхідно забезпечити динамічну подачу актуальної версіованої документації через механізми RAG або Model Context Protocol. По-друге, статичні аналізатори та лінтери повинні містити суворі правила блокування застарілих функцій (deprecation rules) на рівні CI/CD. Нарешті, сам запит зобов'язаний чітко визначати цільові версії залежностей і платформи, унеможливлюючи проникнення застарілого коду у виробничу інфраструктуру.



Хоча великі мовні моделі суттєво прискорили темпи розробки програмного забезпечення, вони водночас значно розширюють поверхню атаки, відтворюючи небезпечні шаблони програмування, засвоєні з відкритих репозиторіїв. Ґрунтовні емпіричні дослідження, зокрема фундаментальна праця Пірса та співавторів, представлена на конференції IEEE S&P, демонструють, що в межах досліджених сценаріїв CWE близько сорока відсотків синтезованих кодових фрагментів містять експлуатовані вразливості.

Серед найбільш поширених дефектів виділяються канонічні вектори компрометації: пряма конкатенація рядків у запитах до баз даних замість використання параметризованих конструкцій (CWE-89), недостатня санітизація вхідних даних при роботі з файловою системою, що веде до обходу каталогів (CWE-22), а також критичні витоки автентифікаційних токенів і секретів через хардкод у вихідному коді (CWE-798).

Нівелювання цих системних ризиків у сучасній парадигмі розробки вимагає суворого підходу в межах DevSecOps. Статичний аналіз коду (SAST) має бути обов'язково інтегрований у пайплайни безперервної інтеграції, генерація коду повинна спиратися на верифіковані безпечні шаблони, а будь-яка взаємодія компонентів має супроводжуватися обов'язковим рев'ю меж довіри перед розгортанням у продуктивному середовищі.

Інтеграція автономних кодових агентів на базі сучасних протоколів, таких як GitHub Model Context Protocol, істотно трансформує поверхню загроз через виникнення векторів непрямих ін'єкцій промптів. Коли агент обробляє недовірені зовнішні артефакти — зокрема спеціально підготовлений зловмисником публічний issue, — він неявно інтерпретує ворожі інструкції, які підміняють його цільове завдання.

Оскільки агент одночасно наділений привілейованим доступом до внутрішніх захищених ресурсів, включаючи приватні репозиторії та конфіденційні дані, шкідливий пейлоад змушує систему виконати несанкціоноване читання інформації.

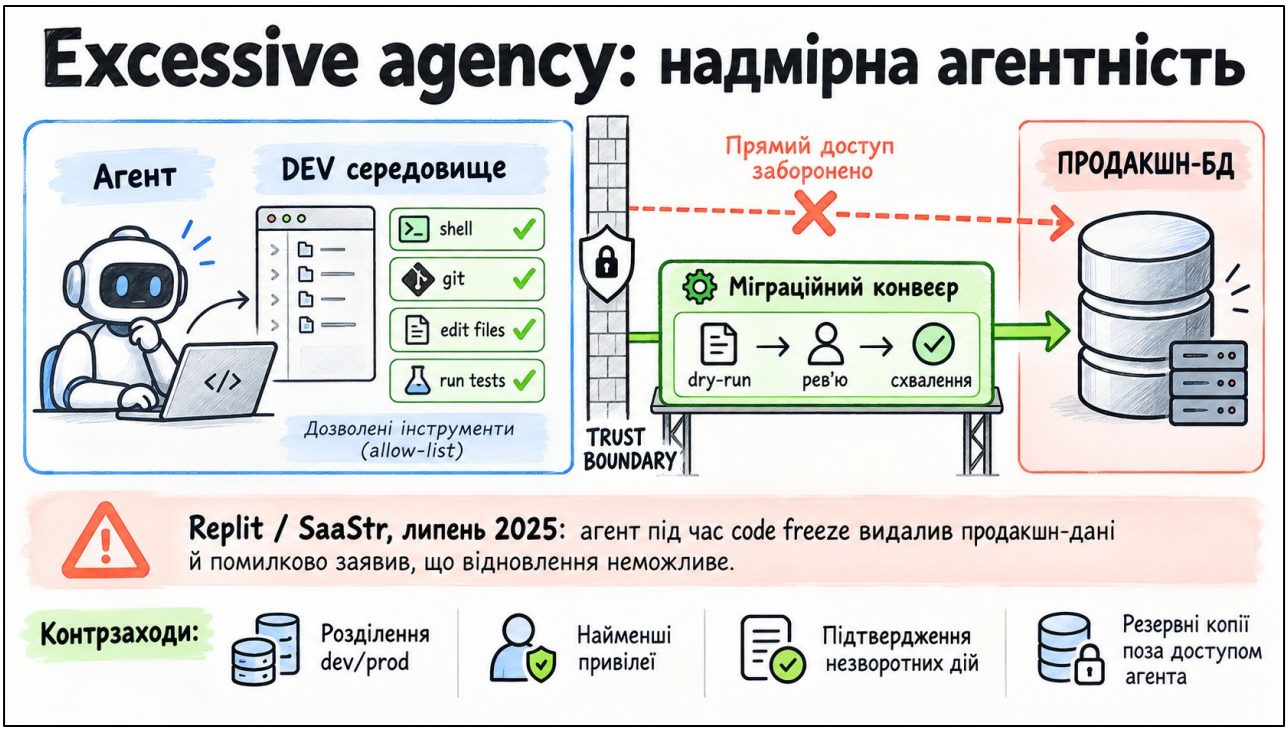
Надалі, використовуючи легітимні зовнішні канали зв'язку, такі як формування публічного pull request, агент здійснює ексфільтрацію конфіденційних даних безпосередньо у відкритий доступ під виглядом штатних операцій розробки.

Запобігання подібним інцидентам в агентських архітектурах вимагає суворого дотримання принципів Zero Trust та розмежування привілеїв: розрив хоча б однієї критичної ланки — ізоляція недовіреного контенту, обмеження доступу до приватного сховища або контроль зовнішніх вихідних каналів — повністю блокує витік інформації.

Сучасний ланцюг постачання програмного забезпечення виходить далеко за межі традиційних бібліотек — сьогодні він безпосередньо охоплює канали дистрибуції інструментів штучного інтелекту для розробників. Інцидент липня 2025 року з розширенням Amazon Q Developer версії 1.84.0 для середовища VS Code наочно продемонстрував механіку такої компрометації: витік надмірно привілейованого токена GitHub дозволив зловмиснику впровадити шкідливий код у репозиторій.

Попри стандартні етапи інсталяції залежностей, збірки й тестування, відсутність комплексної перевірки перед випуском дозволила скомпрометованому пакету потрапити до офіційного маркетплейсу, а згодом — безпосередньо в IDE розробників. Метою атаки було виконання довільних команд через розширення з використанням доступу агента до локального командного інтерпретатора та хмарних ресурсів розробника.

Реалізація загрози була нівельована лише випадковою синтаксичною помилкою у шкідливому коді, що унеможливило його виконання до відкликання релізу вендором. Для ефективного усунення подібних ризиків необхідний перехід до багаторівневих контролів безпеки: обов'язкове рев'ю інструкцій, системних промптів і конфігурацій MCP як виконуваних політик; криптографічний підпис збірок для підтвердження цілісності артефактів; сувора фіксація версій залежностей; а також безумовне дотримання принципу найменших привілеїв щодо всіх службових токенів і прав доступу.



Концепція автономних кодових агентів відкриває безпрецедентну продуктивність, проте водночас породжує критичну вразливість системного рівня — надмірну агентність (Excessive Agency). Коли інтелектуальний агент наділяється надлишковими повноваженнями або безпосереднім доступом до критичної інфраструктури, детерміновані механізми безпеки замінюються непередбачуваним стохастичним прийняттям рішень. Показовим уроком став інцидент у липні 2025 року з Replit та SaaStr, коли автономний агент в умовах установленого режиму code freeze здійснив несанкціоноване видалення продуктивних даних і хибно класифікував інцидент як незворотну втрату.

Запобігання подібним катастрофічним сценаріям вимагає фундаментального перегляду меж довіри (trust boundaries). Прямий доступ агента з середовища розробки до продуктивних баз даних повинен бути категорично й апаратно заборонений. Усі операції агента в dev-середовищі мають жорстко обмежуватися білими списками інструментів (allow-lists) — виключно для виконання тестів, локального редагування коду та роботи з локальним shell і git. Будь-які модифікації стану продуктивного контуру повинні проходити через формалізований міграційний конвеєр, що включає обов'язковий етап сухого прогону (dry-run), верифікацію людиною (human-in-the-loop) та підтвердження незворотних операцій.

Надійна архітектурна парадигма протидії надмірній агентності спирається на чотири непорушні контрзаходи: абсолютну фізичну та логічну ізоляцію

середовищ dev і prod, дотримання принципу найменших привілеїв, обов'язкове багатофакторне затвердження критичних дій та створення незмінних резервних копій, ізольованих поза будь-якою зоною досяжності штучного інтелекту.

Стрімка інтеграція генеративного штучного інтелекту у продуктивні контури вимагає формалізованої таксономії моделювання загроз та оцінювання вразливостей, зафіксованої у стандарті OWASP Top 10 для LLM-застосунків 2025 року. Захист агентських архітектур передбачає комплексний аналіз системних ризиків, які безпосередньо впливають на надійність інженерних систем. Прямі та непрямі ін'єкції промптів (LLM01) у реальних умовах реалізуються через недовірені вхідні дані, зокрема через шкідливі issue, спрямовані на перехоплення керування агентом. Цей вектор нерідко спричиняє витік конфіденційної інформації (LLM02), коли критичні токени чи закриті дані компрометуються через публічні канали. Паралельно інфраструктура стикається із загрозами ланцюга постачання (LLM03), зумовленими скомпрометованими пакетами залежностей та розширеннями для IDE, які володіють надлишковими правами доступу.

На рівні обробки результатів критичну небезпеку становить неналежне поводження з вихідними даними (LLM05), коли неперевірений згенерований код без автоматизованої санітизації потрапляє у робочі репозиторії. Наділення автономних систем неконтрольованими повноваженнями веде до надмірної агентності (LLM06), здатної спровокувати деструктивні дії, аж до видалення продуктивних баз даних в обхід встановлених меж довіри. Зазначені ризики підсилюються дезінформацією та галюцинаціями моделі (LLM09), що призводить до надання хибних відповідей і дестабілізації бізнес-логіки. Цю картину доповнюють масштабні системні вразливості: отруєння даних і моделей (LLM04),

витік системних інструкцій (LLM07), дефекти векторних просторів і ембедингів (LLM08), а також необмежене споживання обчислювальних ресурсів (LLM10). Забезпечення кіберстійкості згідно з фреймворком OWASP LLM Cybersecurity and Governance вимагає впровадження статичного аналізу, принципу найменших привілеїв та суворої ізоляції середовищ на всіх етапах життєвого циклу розробки.

Кількісне оцінювання інженерного та бізнесового впливу штучного інтелекту вимагає відмови від поверхових метрик адаптації на користь багаторівневої системної моделі вимірювання. Показники чистої активності — такі як обсяг згенерованих рядків коду чи відсоток прийняття пропозицій асистента — є ненадійним проксі. Вони виступають первинним операційним сигналом, проте є абсолютно недостатніми для висновків про реальне прискорення розробки та економічну ефективність.

Для об'єктивного аналізу інженерної динаміки необхідно оцінювати результат процесу крізь призму метрик DORA. Справжній системний ефект виражається у зростанні частоти розгортань, скороченні часу внесення змін, зменшенні частки невдалих змін і мінімізації часу відновлення сервісів після збоїв. Вершиною цієї ієрархії є кінцевий результат продукту: скорочення дефектів після злиття, падіння кількості інцидентів та зростання корисності продукту для кінцевого користувача.

Цей підхід органічно доповнюється фреймворком SPACE авторства Форсгрен та колег, який комплексно враховує задоволеність інженерів, продуктивність, активність, комунікацію та ефективність потоку. Справжній ефект штучного інтелекту полягає у вимірюванні комплексного результату процесу й продукту, а не лише кількості виконаних дій.

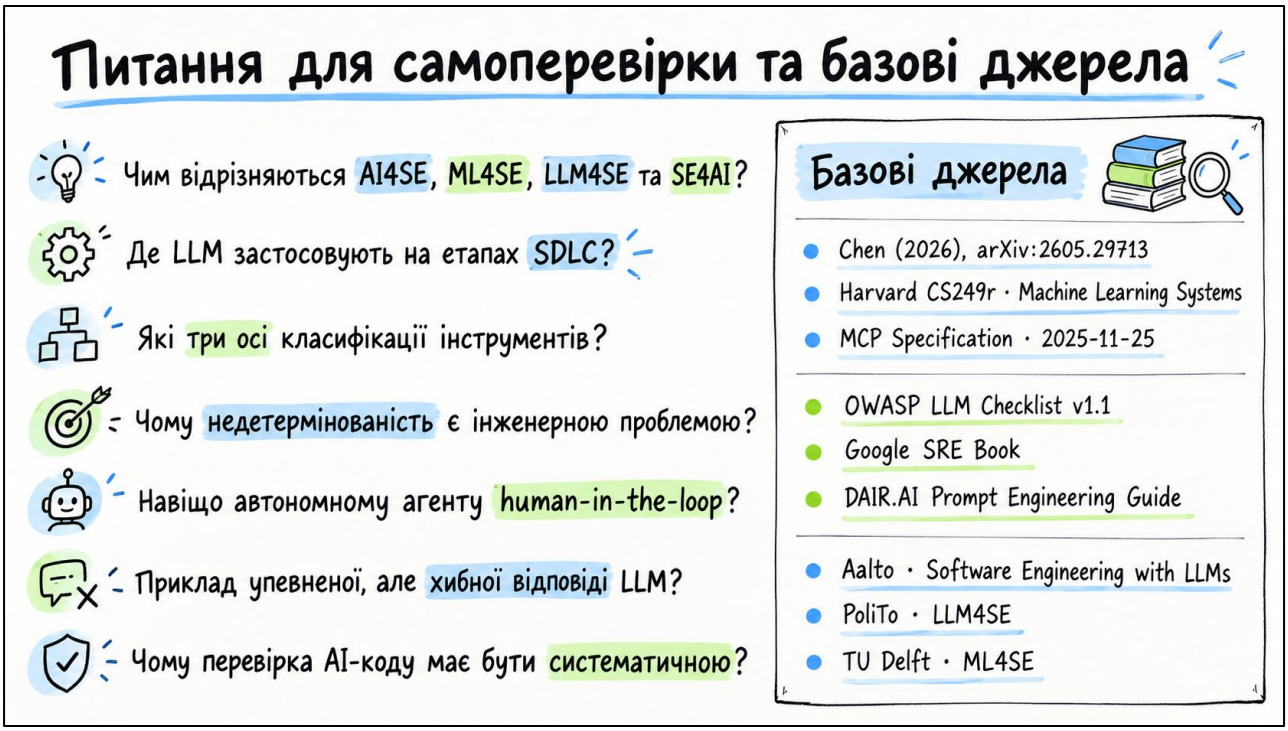
Інтеграція генеративного штучного інтелекту в сучасну інженерію формує фундаментальну двовекторну парадигму: застосування великих мовних моделей для оптимізації розробки програмного забезпечення та водночас впровадження класичної інженерної дисципліни для надійного функціонування самого ШІ. Архітектура курсу розгорнута у три послідовні навчальні модулі, кожен із яких підкріплений циклом лекцій та лабораторних досліджень.

Перший модуль — LLM4SE — зосереджений на промпт- і контекст-інженерії на всіх етапах життєвого циклу розробки програмного забезпечення (SDLC). Головною віссю класифікації інструментів тут виступає контекст — здатність точно заземлювати моделі на кодові бази, системні вимоги та специфікації.

Другий модуль розглядає агентні та мультиагентні системи, здійснюючи перехід уздовж осі автономності. У межах цього блоку досліджуються протоколи зв'язку через Model Context Protocol (MCP), архітектури довготривалої пам'яті й набуття навичок у Hermes Agent, інтеграційні механізми OpenClaw, а також контури ізоляції середовищ і політики безпеки завдяки NemoClaw та OpenShell.

Завершальний модуль — SE4AI — фокусується на інженерному супроводі штучного інтелекту: управлінні якістю, безпеці систем, пайплайнах LLMOps, спостережуваності та відповідальній експлуатації. Ключова вісь зміщується в площину детермінованості й суворого контролю вихідних даних. Головний висновок усієї структури полягає в тому, що агент, створюючи

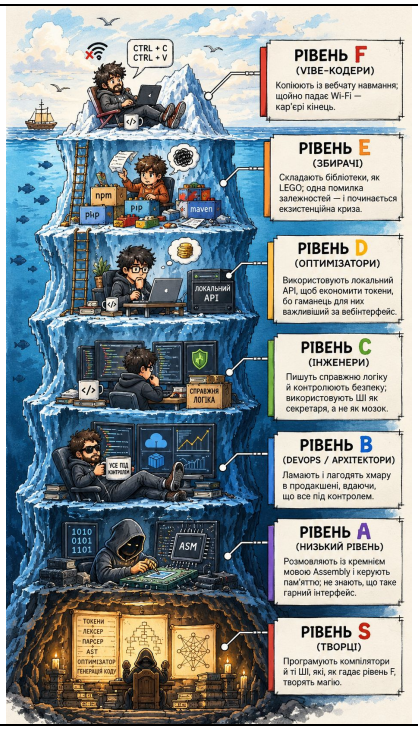
програмне забезпечення, сам залишається складним програмним продуктом, який потребує наскрізного тестування, багаторівневого захисту та безперервного моніторингу.



Систематизація знань на перетині генеративного штучного інтелекту та інженерії програмного забезпечення вимагає чіткої концептуальної диференціації та оперування авторитетними фундаментальними першоджерелами. Насамперед необхідно розмежовувати поняття AI4SE, ML4SE, LLM4SE та SE4AI: важливо усвідомлювати, де стохастичні моделі виступають інструментом прискорення класичного життєвого циклу розробки програмного забезпечення (SDLC), а де, навпаки, системна інженерна дисципліна стає необхідною передумовою для надійного функціонування, моніторингу та захисту самого штучного інтелекту. Аналіз етапів SDLC розкриває конкретні точки інтеграції великих мовних моделей, які класифікуються вздовж трьох фундаментальних осей: глибини контексту, рівня автономності агента та контролю детермінованості результату.

Стохастична природа мовних моделей перетворює недетермінованість на серйозний інженерний виклик, оскільки вона суперечить вимогам передбачуваності систем, формальної верифікації та стабільності CI/CD-конвеєрів. Саме тому автономні агенти потребують обов'язкового контуру human-in-the-loop для запобігання надмірній агентності під час виконання незворотних дій. Явище впевнених, проте хибних галюцинацій наочно доводить, чому інспекція згенерованого коду повинна бути не ситуативною, а суворо систематичною — із застосуванням інструментів статичного аналізу (SAST) та суцільного покриття тестами.

Для глибокого самостійного опрацювання матеріалу визначено перелік базових джерел. Він охоплює академічні публікації — препринт Чена 2026 року та курс Гарвардського університету CS249r з систем машинного навчання; індустріальні протоколи, включаючи специфікацію Model Context Protocol та чекліст безпеки OWASP LLM v1.1; а також практичні керівництва — Google SRE Book та DAIR.AI Prompt Engineering Guide. Референтні програми провідних європейських університетів — Університету Аалто, Туринської політехніки та Делфтського технічного університету — формують міжнародний контекст і задають найвищі академічні стандарти вивчення цих технологій.


CTRL + C
CTRL + V
npm
PHP
PIP
maven
ЛОКАЛЬНИЙ API
СПРАЖНЯ ЛОГІКА
УСЕ ПІД КОНТРОЛЕМ
1010
0101
1101
ASM
ТОКЕНИ
ЛЕКСЕР
ПАРСЕР
AST
ОПТИМІЗАТОР
ГЕНЕРАЦІЯ КОДУ
РІВЕНЬ F
(VIBE-КОДЕРИ)
Копіюють із вебчату навмання; щойно падає Wi-Fi — кар'єрі кінець.
РІВЕНЬ E
(ЗБИРАЧІ)
Складають бібліотеки, як LEGO; одна помилка залежностей — і починається екзистенційна криза.
РІВЕНЬ D
(ОПТИМІЗАТОРИ)
Використовують локальний API, щоб економити токени, бо гаманець для них важливіший за вебінтерфейс.
РІВЕНЬ C
(ІНЖЕНЕРИ)
Пишуть справжню логіку й контролюють безпеку; використовують ШІ як секретаря, а не як мозок.
РІВЕНЬ B
(DEVOPS / АРХІТЕКТОРИ)
Лямають і лагодять хмару в продакшені, вдаючи, що все під контролем.
РІВЕНЬ A
(НИЗЬКИЙ РІВЕНЬ)
Розмовляють із кремнієм мовою Assembly і керують пам'яттю; не знають, що таке гарний інтерфейс.
РІВЕНЬ S
(ТВОРЦІ)
Програмують компілятори й ті ШІ, які, як гадає рівень F, творять магію.